The Cedar Attestation Gap: Structural Security Limits of Agentic AI Authorization Policies and a Formal Robustness Framework

Anonymous submission — target venue: USENIX Security / IEEE S&P

Abstract

Cedar's authorization policy language has a structural gap at the type level: `Request.context` is a flat, untagged `Map Attr Value` with no provenance field anywhere in Cedar's type universe (`Cedar/Spec/Evaluator.lean`, cedar-spec [1]), making `context.attr_name` syntactically identical whether the attribute is caller-supplied (adversary-controlled) or PEP-attested (infrastructure-verified). We formalize this as **Security Claim 1** (§5.3), machine-checked in Lean 4 (`lean/SecurityClaim1.lean`, 337 lines, standalone, no Mathlib, zero `sorry` / `admit`) for a simplified, context-only model of Cedar (`Value := String`): any vulnerability class V whose correct-DENY requires checking an adversary-controlled context attribute α satisfies $B_{struct}(\pi, V) = 0$ for every Cedar policy π without PEP attestation of α — a direct consequence of Cedar's evaluator returning `req.context` verbatim, making forged and attested values indistinguishable at the PDP layer. A companion file (`lean/SecurityClaim2_EntityStore.lean`, 216 lines, same conventions, zero `sorry` / `admit`) machine-checks the qualitative converse for entity-store attributes — they are *not* universally forgeable the way context attributes are, since their values come from a request-independent trusted store — closing what earlier drafts left as an informal argument (§5.3 states the precise scope of both files). Security Claim 1 makes the attestation ceiling $R^{max}_{struct}(T) = \Sigma_{\text{att-dep}} w_i / \Sigma w_i$ computable for any taxonomy T without running the full empirical apparatus. We call the resulting exposure the **attestation gap**.

To measure the gap empirically, we build a dual-loop hardening framework — a *blue* loop that optimizes a Cedar policy, a *red* loop that adversarially tests it — with **R(π)**, a deterministic severity-weighted robustness metric evaluated against the official Cedar Rust engine (cedar-python 0.1.4). Nine blue-loop iterations produce three policy variants; R(π) rises from 0.2000 to 1.0000 across a 9-vector taxonomy T_9 (weight 21). The structural ceiling $R^{max}_{struct}(T_9) = 9.5/21 = 0.4524$ is achieved exactly by the partial policy — confirming Security Claim 1 constructively. An adaptive mutation engine (7 strategies) generated **9,130 mutations**, discovering 121 fingerprint-distinct bypasses in the baseline and 0 in either hardened policy, corroborated by exhaustive enumeration of all **768 well-typed Cedar request states** from the opposite direction. Exhaustive enumeration surfaces one attack class absent from V1–V8: **V9** (path_attestation_gap, $w = 1$), independently confirming Security Claim 1 across the full bypass-capable state space. The final policy π_2 is verified across **four independent modalities**: (1) exhaustive 768-state enumeration — 0 bypasses across all 76 attestation-gap states; (2) Z3 SMT syntactic lint — every permit rule UNSAT under pep_all_False in <4ms; (3) Z3 SMT semantic bypass check — no well-typed bypass achieves PERMIT; and (4) entity-store-aware consistency check — `is_owner` encoded as `(principal_id == resource_owner_id)` via Z3 integer equality (not a free Boolean), certifying π_2 UNSAT (3.95ms), finding V7 bypass witnesses in π_1, and showing via negative control that the check is not tautologically UNSAT. Language-stability: OPA/Rego on the same taxonomy reaches the identical structural ceiling $R^{max}_{struct} = 0.4524$. Post-hoc validation uncovered a **schemaless-mode silent policy degradation hazard** in Cedar: `String.contains()` on string attributes generates a runtime type error that Cedar's specification-compliant engine discards per documented semantics ("a policy that results in an error is ignored"), yielding PERMIT when a developer expects DENY. Cedar's schemaless mode accepts the ill-typed policy without parse-time rejection; callers checking only `response.allowed` receive PERMIT with no indication that a forbid rule was silently neutralized.

We additionally present a **specification-discovery measurement and an external bypass prevalence audit** (§5.6). Cedar's grammar makes `context.attr_name` syntactically identical for caller-supplied and PEP-attested attributes — what we call the **Schema-Reality Gap**. On a co-authored corpus of 25 Cedar policies across five

domains (75 context attributes), an SMT/Z3 all-OWA baseline finds 25/25 bypass scenarios in ~90ms but produces 5 false alarms (20%); an LLM-based classifier achieves F1=0.649 — barely above the trivial all-OWA baseline (F1=0.515) — because both are blocked by the same structural absence. We propose provenance annotations (`@caller_supplied`, `@pep_attested`) in Cedar schema: an OWA-discriminated SMT analysis eliminates all 5 false alarms (policy-level F1: 0.889 → 1.000, FN=0), with F1=1.000 out-of-sample on 3 unseen cedar-examples policies. An AI-simulated blind annotator pilot (21 third-party policies, 5 independent repositories, 62 attributes, §5.1) extends this out-of-sample check and surfaces the first concrete instance of the "harder case" this paper's own validity analysis names as absent from its corpus — a caller-supplied attribute and a PEP-attested attribute producing an indistinguishable permit path — though closing this gap still requires the human annotator study the pilot cannot substitute for (§8.1). To measure the gap's prevalence, we cloned `cedar-policy/cedar-examples` (Apache-2.0, Cedar team at AWS) and audited all 7 policies in cedar-example-use-cases against the Rust PDP. OWA labels come from cedar-examples README documentation, not from this paper's authors. **2/7 bypasses confirmed**: document_cloud (sole gate: `is_authenticated` boolean, DENY→PERMIT) and tax_preparer (forged `context.consent.client` EntityUID bypasses consent gate; cedar-examples README explicitly states "consent is passed in with the authorization request's context"). **2/7 OWA_COMPLEX** (streaming_service date-time functions not evaluable in cedar-python 0.1.4 schemaless mode; sales_orgs entity-store-attested attributes). **3/7 TN_NO_CONTEXT** (no context attributes). We extend this measurement to **14 production Cedar policies from 8 organizations** (Microsoft AGT, AWS AgentCore, Firma-AI, sondera-ai, fronalabs, RelayOne, scopeblind, wshobson/agents) — sampled from 1,210 `.cedar` files with `context.*` found via GitHub Code Search — using a **source-code provenance methodology** that classifies security-critical context attributes by locating the request-construction code rather than inferring from policy text. Four policies have code-verified TN provenance (`cedar/transform.rs`, `channel/manager.rs:96–100`, `build_context()` in sidecar, AWS gateway comment); 8 OWA_COMPLEX or operator-misuse cases. A Sprint 29 scan of 612 `context.* in [string_list]` hits uncovered a **second Cedar schemaless hazard**: Cedar's `in` operator is for entity-hierarchy membership only; applied to String context attributes it generates a silent type error discarded per spec, fail-open when a blanket permit exists. **Production operator-misuse bypass (Rust PDP)**: wshobson/agents `review-agent-governance.cedar` — all 5 forbid rules silently discarded + blanket permit fallthrough — deployed in 4 additional public repos (`run_sprint29_production_bypass_audit.py`, 5/5 bypasses confirmed). Sprint 30 provides the first production OWA attestation-gap bypass (Security Claim 1 confirmation): minorun365/agentcore-book `permit_amount.cedar` — agent forges `context.input.amount = 49999` to obtain Cedar PERMIT, then calls `process_refund(amount=100000)` via actual API; Cedar cannot verify the authorized value matches the actual parameter. Sprint 32 closes the loop with a **live running-system exploit** (`demo/cedar-gateway-demo/`): faithful gateway harness (FastAPI + cedar-python 0.1.4) replicating the no-interceptor deployment pattern; exploit script demonstrates forged Cedar context (49999 cents → PERMIT) → actual refund of 100000 cents ($1000) executing on a live HTTP service; Cedar not re-evaluated at execution time; $500.01 extracted beyond Cedar's authorization (`results/sprint32_live_exploit_confirmed.json`). Sprint 31 audits the official AWS Labs production deployment (awslabs/agentcore-samples `lakehouse-agent`), revealing the **correct OWA mitigation pattern**: the interceptor Lambda attests geography from Cognito JWT and overwrites any agent-supplied value — OWA forgery prevented. However, a Cedar policy coverage gap remains: `text_to_sql` is not in the EU-individual-claims forbid scope, allowing EU data access via SQL even with correct OWA defense. This establishes the reference architecture for §6 recommendations. Combined n=25: **5 confirmed OWA/misuse bypasses (20%); 6 OWA_COMPLEX; 14 TN/correctness-failure (awslabs + 13 prior)**. Two of the 5 bypasses are CEDAR_OPERATOR_MISUSE (wshobson/agents and kenhuangus/agentctl — independent repos, distinct domains, same Cedar `in`-on-String mechanism; Cedar upstream disclosure: cedar-policy/cedar#2428, filed 2026-06-24). A follow-up audit (§5.6) dynamically confirms five additional repositories that installed the wshobson policy verbatim via Claude Code's plugin/marketplace distribution channels — **seven repositories dynamically confirmed PERMIT in total, but the independent-authorship count is N=2** for the original purposive-discovery corpus; the other five are supply-chain amplification of one vulnerable policy, not

independent recurrence. Broadening the search beyond `context.*` attributes to any String-typed attribute compared via `in` (§5.6.1) later finds a **third independently-authored, dynamically-confirmed bypass** (`shoaibakthar/observeid-platform`, a separation-of-duty gate bypassed identically to wshobson/kenhuangus), raising independent bypass authorship to **N=3** — with the caveat that, unlike the first two, this repository ships a schema that Cedar's own validator would flag, though the repository shows no evidence of actually running it in CI. A repo-stratified randomized audit (78 repositories, Wilson 95% CI, seed=42) finds a raw 17.9% conditional H1 (`in`-on-String) regex-match frequency within the queried corpus; reading every flagged file rather than trusting the regex count, 5 of the 14 matches are non-deployment artifacts (3 are `cedar-policy-core`'s own vendored test fixture, 2 are mirror repos carrying only a self-labeled "not a production example" test file) and a 6th is a correlated-authorship non-instance (~650 near-identical files from one AI-agent-generated monorepo, the same de-duplication logic already applied to the wshobson case), correcting the estimate to **6–8/78 (7.7%–10.3%) production-plausible misuse** (§5.6.1); of the (separately-drawn) 10 unique repositories with H1 misuse in the file-level sample checked for CI enforcement, 0/10 invoke `cedar validate` inside an actual `.github/workflows` job (§5.6.1). Cedar's own validator, when a schema is supplied, is a complete structural fix for this class: it flags both illustrative examples and 158/158 (100%) real `in`-on-String patterns captured in the file-level random sample (§5.6.1, §5.10). Repeating the analogous randomized-audit methodology for H2 (`.contains()`-on-String) initially yields a different, more cautionary result: 10/57 repositories match a `.contains()`-on-context-attribute regex candidate, but manual review of all 10 finds **zero confirmed misuse** in that specific sample — a co-located schema definitively confirms correct `Set<String>` typing for 2 of the 10, and the rest are unresolved-but-consistent with correct usage, since unlike `in`, `.contains()` is Cedar's legitimate Set-membership operator and cannot be reliably classified by regex alone (§5.6.2). Broadening beyond `context.*` and replacing naming-based inference with systematic schema cross-referencing (§5.6.3, mirroring the same two fixes §5.6.1 applied to H1) then finds a genuine, independently-authored, dynamically-confirmed H2 bypass: `SafetyMP/FidusGate`, a real AI-agent tool-call gateway whose `.cedarschema` explicitly types the misused attributes as `String`, where a non-authorized principal can write directly to a `.env` secrets file and chain a `curl`-to-shell payload past a sandboxed-command allowlist — both because the relevant `forbid` rules are discarded by the identical type error. This is H2's first independently-authored, dynamically-confirmed bypass in this paper.

Keywords: attestation gap, Cedar, ABAC, agentic AI authorization, formal robustness metric, OWA/CWA, autonomous red teaming, PEP attestation, cedar-python, schema-reality gap, specification discovery.

---

1. Introduction

Every action that an LLM agent takes on the world — reading files, invoking tools, delegating tasks — passes through an authorization policy. These policies, typically written in attribute-based access control (ABAC) languages such as Amazon's Cedar, express fine-grained, entity-typed rules with a default-deny posture. When security teams audit these policies — or when SMT tools check them automatically — they face a systematic blind spot: Cedar's grammar makes `context.attr_name` syntactically identical whether the attribute is caller-supplied (the adversary controls it) or PEP-attested (trusted infrastructure computed it before the PDP saw it). A Z3 solver treating all context attributes as adversary-controlled generates correct bypass reports for OWA attributes — but also generates false alarms for every PEP-attested attribute in the policy, with no way to tell them apart. We call this the **Schema-Reality Gap**, and our primary empirical contribution is its measurement and a proposed fix.

Why the gap is structurally unavoidable. The gap is not an accident of Cedar's syntax. **Security Claim 1** (§5.3) characterizes it: for any Cedar authorization schema, any vulnerability class whose correct-DENY depends on an adversary-controlled context attribute satisfies $B_{struct}(\pi, V) = 0$ for every Cedar policy π without PEP attestation of that attribute. This follows from Cedar's evaluator returning `req.context` verbatim — `Request.context` is a flat, untagged `Map Attr Value` with no provenance field anywhere in the type universe (`Cedar/Spec/Evalu-`

`ator.lean`, cedar-spec [1]). Cedar's PDP cannot distinguish `context.ownership_verified = true` submitted by a legitimate PEP from the same value forged by a malicious caller — and no additional policy rule can create that distinction. The mitigation is not a policy rule; it is architectural: a PEP that attests the attribute before handing it to Cedar, and a schema annotation that makes provenance visible to static analysis.

Our apparatus. To instantiate and measure this bound empirically, we build a dual loop over a Cedar policy, inspired by the AutoResearch pattern [3]. A **blue agent** proposes structured policy modifications and commits them only when robustness improves. A **red agent** generates adversarial requests targeting the current policy and surfaces bypasses. An **orchestrator** coordinates the loops and detects cross-loop meta-patterns that neither agent finds alone. Both agents are implemented with Google ADK2 2.2.0 (LoopAgent, SequentialAgent) and evaluate policies against the official Cedar Rust engine via cedar-python 0.1.4. The framework is the empirical validation instrument for Security Claim 1: we measure whether the actual partial policy achieves exactly the attestation ceiling the theorem predicts — and it does.

Our metric. We make robustness a scalar by defining

$$R(\pi) = \frac{\sum_{i=1}^{8} w_i \cdot B(\pi, v_i)}{\sum_{i=1}^{8} w_i}$$

where π is the policy under test, v_i a vulnerability vector, w_i its severity weight, and $B(\pi, v_i) \in \{0.0, 0.5, 1.0\}$ a bypass score (active, partial, closed). R(π) is deterministic for a fixed policy and vector set, making it a *reproducibility-oriented* fitness function and the concrete instrument with which Security Claim 1's ceiling prediction ($R^{max}_{struct}(T_8) = 9.5/20 = 0.4750$) is tested. The 8-vector taxonomy comprises 7 DENY-expected attack vectors (V1–V7) and 1 PERMIT-expected over-blocking check (V8), preventing a trivial deny-all policy from achieving $R(\pi) = 1.0$.

Contributions.

1. **Security Claim 1 — the Cedar attestation gap** (§5.3): a schema-independent formal characterization, proved in Lean 4 (`lean/SecurityClaim1.lean`, 337 lines), that for any Cedar schema S, any vulnerability class V that requires checking an adversary-controlled context attribute α satisfies $B_{struct}(\pi, V) = 0$ for every Cedar policy π without PEP attestation of α — a direct consequence of Cedar's CWA evaluation semantics [1] making forged and attested values indistinguishable at the PDP layer. The attestation ceiling $R^{max}_{struct}(T)$ is computable for any taxonomy T without running the empirical framework. For our taxonomy T_8: attestation-dependent vectors (V5–V7, V3.2) carry $\approx 52.5\%$ of total risk weight, giving $R^{max}_{struct}(T_8) = 9.5/20 = 0.4750$, confirmed constructively by the partial policy ($R(\pi_1) = 0.4750$). Under T_9: $R^{max}_{struct}(T_9) = 9.5/21 = 0.4524$. The gap magnitude is domain-specific (Cedar S2: 0.3889) but language-stable (Cedar S1 and OPA/Rego reach identical ceilings on the same taxonomy).
2. **The Schema-Reality Gap, an external bypass prevalence audit using actual cedar-examples files plus a production ecosystem extension, and a provenance annotation fix** (§5.6): we cloned `cedar-policy/cedar-examples` (Apache-2.0, Cedar team at AWS, release/4.11.x) and audited all 7 cedar-example-use-cases policies against the Rust PDP. OWA labels come from cedar-examples README documentation, not from this paper's authors. **2/7 (29%) bypasses confirmed** on actual policy files: (a) document_cloud — sole gate `is_authenticated` boolean (inline request context), DENY→PERMIT; (b) tax_preparer — forged `context.consent.client` EntityUID bypasses consent gate; cedar-examples README explicitly states "consent is passed in with the authorization request's context." We then extend the audit to **14 production policies from 8 organizations** — Microsoft (agent-governance-toolkit IFC policy), AWS (AgentCore healthcare policies), Firma-AI, sondera-ai, fronalabs, RelayOne, scopeblind, wshobson/agents — sampled from **1,210 Cedar files with `context.*`** found via GitHub Code Search. A **source-code provenance methodology** classifies each security-critical context attribute by reading the request-construction code (the function that calls `Con-`

`text::from_json_value(...)`). Four policies have code-verified TN provenance: sondera-ai's YARA+IFC harness (`cedar/transform.rs`), fronalabs/frona's channel DB (`manager.rs:96–100`), Firma-AI's sidecar RuntimeSignals (`build_context()` in `cedar_evaluator.rs`), and AWS AgentCore's gateway-documented timestamp. Seven production policies are OWA_COMPLEX (financial: spending-authority `context.approval_method`, payment threshold `context.input.amount`; healthcare: `context.input.patient_id`; agent-governance: `context.command_pattern`; messaging: `context.now`; escalation: `context.ticket_severity`). **Combined n=21: 2 confirmed bypasses (10%), 9 OWA_COMPLEX (43%), 10 TN (48%); upper bound 11/21 = 52%** have at least one OWA-reachable permit path. (Post-Sprint 33: a second independent CEDAR_OPERATOR_MISUSE_BYPASS confirmed at kenhuangus/agentctl, bringing the operator-misuse bypass class to n=2 independent instances across distinct domains; total corpus n=25, 5 confirmed bypasses — see §5.6 Extended Production Corpus.) On a co-authored corpus of 25 Cedar policies (75 context attributes), an SMT/Z3 all-OWA baseline finds 25/25 bypass scenarios in ~90ms but produces 5 false alarms (20%); an LLM-based classifier achieves F1=0.649 — marginally above the trivial all-OWA baseline (F1=0.515) — because both are blocked by the same structural absence. We propose Cedar schema annotations (`@caller_supplied`, `@pep_attested`) and empirically validate that OWA-discriminated SMT analysis eliminates all 5 false alarms (policy-level F1: 0.889 → 1.000, FN=0). On 3 unseen cedar-examples policies, F1=1.000 out-of-sample. A companion static PEP code analyzer shows annotation emission can be automated (CI/CD linter, 0.4ms/policy) under clean write-pattern conventions.

3. **R(π):** a deterministic, reproducibility-oriented robustness metric for Cedar authorization policies, evaluated against the official Cedar Rust PDP. R(π) operationalizes Security Claim 1: the metric's attestation ceiling is exactly $R^{max}_{struct}(T) = \sum_{i \notin att} w_i / \sum w_i$, measurable for any taxonomy T.

4. A **9-class vulnerability taxonomy** ($T_9 = V1\text{–}V9$): seven DENY-expected attack vectors (V1–V7), one PERMIT-expected over-blocking check (V8), and one new class V9 (path_attestation_gap, $w = 1$) identified by exhaustive 768-state enumeration as the sole $(role, action, resource\_type)$ group absent from V1–V8. V9 is fully attestation-dependent; the final policy closes all 9 vectors ($R(\pi_2, T_9) = 1.0000$). We make no claim of completeness; §5.2 and §6.4 enumerate attack classes the taxonomy omits.

5. A **dual-loop execution**: 9 blue-loop iterations producing a three-stage Cedar policy trajectory, with R(π) as the commit criterion. An end-to-end re-run with the official Rust PDP as the commit oracle (`scripts/run_sprint18_rust_pdp_loop.py`) converged identically to R(π) = 1.0000 in 6 greedy iterations with zero rollbacks, confirming no oracle-engine divergence affects the hardening trajectory (§4.3).

6. A **Cedar schemaless-mode silent policy degradation hazard**: `String.contains()` on string attributes generates a runtime type error that Cedar's spec-compliant engine discards per documented policy-error semantics, yielding fail-open PERMIT. Cedar's schemaless mode accepts the ill-typed policy at load time without rejection; the hazard is the intersection of (1) no parse-time type checking in schemaless mode and (2) Cedar's runtime policy-error discard semantics — not a spec divergence, but a documented behavior that creates a silent developer trap for callers checking only `response.allowed`.

7. A **zero false-positive rate**: all three policy variants correctly permit all four V8 legitimate-access requests (FP rate = 0%), confirming no over-restriction is introduced by the hardening trajectory.

8. A **Security Claim 1 domain-stability replication** (§5.7): a second Cedar schema (S2: API Gateway/Microservices, T'_{S2} taxonomy, 7 vectors, weight 18) confirms the attestation gap in a distinct application domain. The Rust PDP verifies $R'^{max}_{struct}(T'_{S2}) = 7/18 = 0.3889$ and $R'(\pi_{att}) = 1.0$. The gap magnitude is domain-specific; the structural claim is domain-stable.

9. A **Security Claim 1 language replication** (§5.8): OPA/Rego evaluation (OPA 0.68.0) confirms the attestation gap in a second policy language. The Rego structural ceiling $R^{max}_{struct}(T_{9\text{-OPA}}) = 9.5/21 = 0.4524$ is **identical to Cedar S1** — when the structural policy applies all closeable defenses, both languages reach the same bound. The structural claim of Security Claim 1 is language-stable.

10. A **construction lint check** (§5.9): Z3 SMT confirms that every permit rule in all three attested policies (Cedar S1, S2, Rego S1) requires at least one @pep_attested attribute to be True (UNSAT under pep_all_False, < 4 ms). A sanity check (pep=all_True) confirms the encoding is not trivially overconstrained. This mechanizes a development-time construction property: catching permit rules accidentally written without any attestation gate.

---

2. Background

2.1 Cedar Authorization Language

Cedar [1] is an entity-typed ABAC language. Policies are sets of `permit` and `forbid` rules over typed principals, actions, and resources, with optional `when` / `unless` conditions over entity and `context` attributes. Evaluation is **default-deny** and **forbid-overrides**: if any `forbid` matches the request, the decision is DENY regardless of permits; otherwise, if any `permit` matches, the decision is PERMIT; otherwise DENY. Cedar's design lends itself to automated reasoning — AWS provides SMT-backed equivalence and reachability checks [9] — which proves properties against a *fixed specification*. Our work is complementary: rather than proving a policy equivalent to a spec, we empirically search for the most robust specification under adversarial pressure.

2.2 Agentic AI Governance

Gibbs's Four-Pillar model [2] frames production agent governance as registration/inventory, runtime policy enforcement, continuous observability, and automated compliance verification. The runtime-enforcement pillar assumes a *dual-engine* authorization stack (relationship-based OpenFGA plus attribute-based Cedar) behind a Policy Enforcement Point. Critically, the PEP is responsible for *attesting* the context attributes a policy reasons over — a requirement that, as we show, is not a deployment detail but the load-bearing security boundary, quantifiable by the R(π) metric.

2.3 OWA vs. CWA in Policy Systems

The formal setting for Security Claim 1 (§5.3) is grounded in a decisive tension between evaluation models. Under the **Closed-World Assumption (CWA)**, anything not explicitly true is false; a missing positive flag means the corresponding guard fails closed. Cedar, however, evaluates only what is *stated* in the request: a `forbid` of the form `when { context.flag == false }` fires only when the attacker-supplied flag is literally `false`. The adversary does not delete the attribute (which might trip a fail-closed default); they simply forge `context.flag = true`, and the negative-condition forbid does not fire while the permit does. Cedar therefore operates under the CWA — it treats every context value as authoritative — while the adversary operates under the **Open-World Assumption (OWA)**, free to set any context field to any value. Negative-condition forbids are exploitable unless an external authority attests the value before it reaches Cedar. This is the attestation gap: **for any Cedar schema, any vulnerability class requiring the policy to reason about an adversary-controlled context attribute is uncloseable by structural Cedar rules alone** — a consequence of Cedar's evaluation semantics, not of any particular schema or weight distribution (Security Claim 1, §5.3).

2.4 The AutoResearch Pattern

Karpathy's AutoResearch [3] is a minimal autonomous-experimentation loop: an agent edits a single file, a scalar metric is measured, and the change is committed if the metric improves or reverted otherwise. The three universal ingredients are an editable artifact, a scalar metric, and a fixed cycle. We map these directly onto policy hardening: the `.cedar` file is the artifact, R(π) is the metric, and R(π)-improvement is the commit criterion. The dual-loop framework we build (§3) is the empirical instrument for testing Security Claim 1's predictions across the hardening trajectory.

---

3. Framework

3.1 Threat Model

We make the adversary explicit, since R(π) is only meaningful relative to a stated attacker. **Adversary capability.** The attacker is a compromised or malicious agent that fully controls the *request* presented to the Cedar PDP: the `(principal, action, resource, context)` tuple, including all attacker-reachable entity and `context` attributes. The attacker may set any context flag to any value (this is precisely what makes negative-condition forbids exploitable, §2.3). **Trust boundary.** We assume the Cedar PDP itself, the policy artifact, and the PEP's attestation channel are *not* under attacker control: when a context attribute is *attested* by the PEP (§4.5), the attacker cannot forge it. The security of attestation-dependent vectors (V5–V7) rests entirely on this assumption; if the PEP-to-PDP channel is compromised, R(π) makes no guarantee. **Out of scope.** We do not model attacks on the LLM agent's prompt or reasoning (indirect prompt injection [6,7] is a complementary layer, §7), side channels, denial-of-service against the PDP, or collusion between the attested PEP and the adversary. R(π) measures only whether the *policy*, as evaluated by the production engine, admits a PERMIT that a correct policy would DENY for a given request under this model.

3.2 Dual-Loop Architecture

Blue loop. The blue agent proposes a Cedar policy modification as structured output containing a `cedar_diff` field. The orchestrator applies the diff, evaluates R(π) via the online Cedar oracle (the pure-Python evaluator; see §4.3 for why this differs from the production Rust PDP and the consequences), and commits the change if R(π) improved or rolls back (`git checkout`) otherwise — the AutoResearch commit/rollback discipline applied to a security artifact.

Red loop. The red agent generates attack requests targeting the current policy, attempting to coerce a PERMIT where DENY is required. Confirmed bypasses are added to a persistent library; after every blue commit, all known bypasses are re-evaluated (regression mode). Beyond this, the red agent also runs an adaptive mutation engine (§4.4) that generates structurally novel attack variants from the seed vectors using 7 mutation strategies, discovering bypasses that a fixed taxonomy cannot enumerate.

Orchestrator. The orchestrator coordinates the loops, blocks regressing commits, prioritizes newly discovered vectors for the blue loop, and — most importantly — detects *cross-loop meta-patterns*. In our runs it surfaced the observation that the blue agent reused the same vulnerable negative-condition forbid pattern across rounds, the OWA/CWA finding that neither loop identified in isolation.

Figure 1: Dual-loop Cedar policy hardening architecture

> *Figure 1: Dual-loop Cedar policy hardening architecture (generated by `scripts/plot_results.py`). The blue loop (left, blue) proposes Cedar policy edits and commits them when R(π) improves; the red loop (right, orange) generates adversarial requests targeting the current policy across V1–V8. The orchestrator (top, green) coordinates both loops and detects cross-loop meta-patterns (e.g., the repeated negative-condition forbid vulnerability surfaced in Round 2). The lower tier shows the oracle split: the pure-Python Cedar evaluator (left, grey) serves as the online commit oracle during the loop; the Cedar Rust PDP (cedar-python 0.1.4, right, grey) is invoked post-hoc for final validation. The R(π) metric box (centre, yellow) receives the weighted bypass scores and returns the fitness signal to the blue agent.*

3.3 Formal Robustness Metric R(π)

We define the robustness of a policy π as the severity-weighted fraction of vulnerability vectors it closes:

$$R(\pi) = \frac{\sum_{i=1}^{8} w_i \cdot B(\pi, v_i)}{\sum_{i=1}^{8} w_i}$$

where:

- π — the Cedar policy under evaluation;
- v_i — vulnerability vector i, a set of concrete test requests (DENY-expected for V1–V7; PERMIT-expected for V8);
- $B(\pi, v_i) \in \{0.0, 0.5, 1.0\}$ — the bypass score: **0.0** if the vector is *active* (a bypass exists for DENY-expected, or an over-block exists for PERMIT-expected), **0.5** if *partially mitigated*, **1.0** if *closed*;
- w_i — the severity weight on a 1–5 scale for V1–V7, weight 2 for V8 (Table 1).

The three-valued B is deliberate: it distinguishes a genuinely closed vector from one that is merely harder to exploit — a distinction that informal evaluation methods routinely blur. The intermediate value $B = 0.5$ is exercised in our results: **V3 on π_1 scores B=0.5** — partial policy's structural `like "*..*"` forbid closes V3.1 (ASCII `../`) and V3.3 (mid-path `..`) but admits V3.2 (Unicode fullwidth dots U+FF0E, which Cedar's string pattern does not match), so 2 of 3 V3 test requests are blocked. This confirms the ternary metric is not vestigial: V3 is partially mitigated structurally, with the Unicode variant requiring PEP `path_safe` attestation (see §5.3 and §4.5). R(π) is deterministic: a fixed policy and a fixed vector set always produce the same score, which is what makes it a reproducible fitness function and what makes Cedar evaluation — in contrast to LLM-based judgment — the correct oracle.

Table 1: Vulnerability Taxonomy.

ID	CLASS	WEIGHT	TYPE	CWE	CAPEC	DESCRIPTION
V1	scope_overflow	2	DENY-expected	CWE-269	CAPEC-122	Cross-role permission escalation
V2	delegation_abuse	4	DENY-expected	CWE-285, CWE-862	CAPEC-122	Privilege escalation via agent delegation
V3	path_traversal_literal	1	DENY-expected	CWE-22	CAPEC-126	Literal `../` in resource paths
V4	encoded_path_traversal	1	DENY-expected	CWE-22, CWE-116	CAPEC-72	URL-encoded `%2e%2e` variants
V5	resource_attribute_forgery	2	DENY-expected	CWE-346, CWE-290	CAPEC-194	Forged resource ownership attributes
V6	role_claim_manipulation	3	DENY-expected	CWE-290, CWE-269	CAPEC-194	Self-asserted role claims without attestation
V7	executor_tool_indirection	5	DENY-expected	CWE-863, CWE-862	CAPEC-1	Executor scope bypass via tool substitution
V8	over_blocking_false_positive	2	PERMIT-expected	—	—	Legitimate access that must not be denied (over-blocking / false-positive check)
Total		**20**				

Weights follow a severity scale grounded in impact class: information disclosure (1), integrity violation (2), privilege escalation (3), delegation abuse (4), and full bypass (5). V8 (over-blocking check) carries weight 2, equal to the `in-`

`tegrity_violation` class. V7 (executor-tool indirection, weight 5) is the single most severe attack vector at 25% of total risk weight. V8 prevents a deny-all policy from achieving $R(\pi) = 1.0$ by penalizing over-restriction alongside under-restriction. The CWE/CAPEC column anchors each vector to the external NVD/CAPEC taxonomy, enabling comparison with CVE-reported vulnerability classes.

Weight justification. The severity weights ($w_i \in \{1, 2, 3, 4, 5\}$) follow a CVSS v3.1 mapping: information_disclosure=1 (CVSS Low), integrity_violation=2 (CVSS Medium-Low), privilege_escalation=3 (CVSS High), delegation_abuse=4 (CVSS High, chained impact), full_bypass=5 (CVSS Critical). V8 (over_blocking, false-positive weight) is set to 2 (same as integrity_violation) because a policy that denies legitimate requests has the same operational impact as a policy that has integrity gaps — both are security failures, though in opposite directions. The specific weight values are a team judgment call grounded in CVSS, not a free parameter. **Correction (verified by direct recomputation, see `scripts/run_sprint47_weight_sensitivity.py`):** an earlier version of this sensitivity analysis claimed $R^{max}_{struct}(T_9) \in [0.40, 0.52]$ for weights varied ±2× from baseline; this range could not be reproduced and was wrong. Recomputing exhaustively over all $2^9 = 512$ corners of independent per-vector perturbation $w_i \in [0.5\, w_i^0, 2\, w_i^0]$ gives $R^{max}_{struct}(T_9) \in [0.176, 0.760]$ — the numeric ceiling is not robust to reweighting. What is robust, verified across all 512 corners: the partial policy always lands strictly below the final policy for the same weight vector ($R(\pi_1, w) < R(\pi_2, w)$, since π_2 closes every vector π_1 closes plus more), and the final policy consistently achieves R=1.0 (because $R(\pi_2)$=1.0 is weight-independent — all vectors are closed). The taxonomy-independent claim is the per-vector structural dichotomy, not the specific numeric ceiling; see `results/sprint47_weight_sensitivity.json` for the full computation.

3.4 Policy Lifecycle

We evaluate three policy stages that correspond to natural hardening milestones:

- **Baseline (π_0):** four `permit` rules, no `forbid` rules — the common "permit where needed" policy. Robustness derives only from Cedar's default-deny.
- **Partial (π_1):** π_0 plus five *structural* forbids (scope overflow, delegation abuse, literal and encoded path traversal). No PEP attestation — every guard is expressible purely in the policy.
- **Final (π_2):** π_1 plus seven *attestation-dependent* forbids and permit conditions (`ownership_verified`, `role_attested`, `path_safe`, `tool_scope_verified`) — full PEP integration. Here the most severe vectors are closed, but only because an external authority attests context.

---

4. Implementation

4.1 Cedar PDP Integration

We use **cedar-python 0.1.4**, the official Python bindings for the Cedar Rust engine, running in a dedicated Python 3.12 virtual environment created via uv 0.9.28 (the binding requires ≥3.12). Because our primary orchestration process runs Python 3.10, evaluation requests are dispatched via a **subprocess bridge**: `CedarRealEvaluator` serializes each `(policy_path, request)` pair as JSON, invokes the Python 3.12 subprocess (`cedar_subprocess.py`), and deserializes the `{"result": "PERMIT" | "DENY"}` response. The bridge adds approximately 40–60 ms per evaluation — negligible for a policy-level reward signal. The `CedarRealEvaluator` is duck-typing compatible with the pure-Python oracle used as the online loop oracle (§4.3), enabling transparent substitution for offline validation.

Entity model. Each request is translated to cedar-python's native API: `cedar.PolicySet` parses the `.cedar` file, `cedar.Authorizer` evaluates it, `cedar.Request` encodes the `(principal, action, resource, context)` tuple, and `cedar.Entity` objects supply the entity attribute graph. All eight vulnerability-class test vectors (V1–V7 attack vectors, V8 over-blocking check) were validated against the Rust engine across all three policy variants.

4.2 Cedar Schemaless Mode: Silent Policy Degradation via Type Errors

During Cedar PDP integration we discovered a **schemaless-mode silent policy degradation hazard**: when a Cedar policy uses `String.contains()` (e.g., `resource.path.contains("..")`) on a string attribute, cedar-python 0.1.4 generates a runtime type error — `type error: expected set, got string` — but **returns PERMIT** because Cedar's specification explicitly documents that erroring policies are ignored ("a policy that results in an error is ignored, meaning that a `forbid` policy might unexpectedly fail to block access"). The error IS reported in `response.errors`, but the authorization decision returned to most callers is PERMIT.

Specification conformance. This behavior is NOT a bug in the cedar-policy Rust engine nor in the cedar-python 0.1.4 binding. Cedar's specification is internally consistent: `contains()` is defined only for the `Set` type, so calling it on a `String` attribute is a type error, and Cedar's documented policy-evaluation semantics discard erroring policies rather than propagating the error as DENY. The cedar-policy Rust engine (the same engine underlying cedar-python 0.1.4 via PyO3) and the cedar-python binding both implement this specification correctly. The Rust engine reports `response.errors = ["error while evaluating policy 'policy1': type error: expected set, got string"]` (verified experimentally). Callers checking only `response.allowed` receive PERMIT with no indication that a forbid rule was silently neutralized.

The hazard is the intersection of two Cedar design choices, not either alone: (1) Cedar's schemaless mode accepts syntactically parseable but type-invalid policies without parse-time rejection — the policy file containing `String.contains()` loads without error; (2) Cedar's runtime policy-error semantics silently discard erroring forbid rules, yielding PERMIT. In isolation, each choice is defensible — (1) enables rapid prototyping without schema; (2) avoids cascading denial from unrelated policy errors. Together, they create a **silent fail-open developer trap**: a developer who writes `forbid when { resource.path.contains("..") }` — following Python or Java string-method intuition — loads the policy without error, receives PERMIT for `../etc/passwd` requests, and has no indication that the forbid rule is providing zero protection unless the caller explicitly inspects `response.errors` on every decision.

This has a broader implication for Cedar deployments: any schemaless Cedar deployment is subject to silent policy degradation on type-erroneous forbid rules. The authorized defense is either (a) always enable Cedar's schema validation at policy-load time, which rejects type-invalid policies before evaluation, or (b) inspect `response.errors` on every authorization decision and treat non-empty errors as policy-load failures.

Fix. We replaced all seven path-traversal `contains()` calls in π_2 with the `like` glob operator:

```
forbid (principal is Agent, action, resource is Resource)
when {
  resource.path like "*..*"      ||
  resource.path like "*%2e%2e*"  ||
  resource.path like "*%2E%2E*"  ||
  resource.path like "*..%2f*"   ||
  resource.path like "*..%2F*"   ||
  resource.path like "*%2e%2e%2f*" ||
  resource.path like "*%252e%252e*"
};
```

After the fix, R(π) for π_2 rose from 0.9444 to 1.0000 under the Rust engine, confirming full concordance with the formal metric. The bug is documented in `docs/cedar_python_contains_bug.md` with a minimal reproduction. We have prepared a detailed bug report for the cedar-python maintainers specifically (`docs/cedar_python_failopen_bugreport.md`) — a track distinct from, and so far not folded into, the `cedar-policy/cedar#2428` / `wshobson/agents#598` / GHSA-hm46-7j72-rpv9 disclosure chain covering H1 (§9): as of this draft, no CVE has been assigned and this specific cedar-python report has not yet been filed upstream, pending completion of this paper.

Security implication. This bug has a practical consequence consistent with Security Claim 1's recommendation: even V3 (`path_traversal_literal`), nominally "closable by structural policy", has a latent PEP dependency when the Cedar engine implementation has this limitation. The safest defense against all path-traversal variants is the PEP-attested `context.path_safe == true` flag, which the Rust engine evaluates correctly in all tested cases.

[^bugreport]: The full bug report (`docs/cedar_python_failopen_bugreport.md`) includes a minimal reproduction, the exact `response.errors` output from cedar-python 0.1.4, the `String.contains()` vs. `like` operator comparison, and recommended remediation. Disclosure to the cedar-python maintainers will proceed upon paper acceptance.

4.3 Blue Loop Execution

The blue agent (Google ADK2 2.2.0 `LoopAgent`, Sonnet 4.6 for policy synthesis, Opus 4.8 for orchestration) ran **9 iterations** across three rounds, each using R(π) as the commit criterion:

- **Round 1 (Exp 1–6):** baseline π_0 → partial π_1. Blue agent iteratively adds structural forbids (scope overflow, delegation, path traversal, tool access). Red agent surfaces new bypasses after each commit. R(π): `0.2000` → `0.4750` (V3 reaches B=0.5 once `like "*..*"` closes V3.1/V3.3; V3.2 Unicode remains a latent bypass).
- **Round 2 (Exp 7–8):** partial π_1 → extended. Blue agent attempts attestation guards; red agent demonstrates OWA bypass. Orchestrator identifies the negative-condition forbid meta-pattern.
- **Round 3 (Exp 9):** final consolidation as π_2. Attestation conditions added to all permit rules; explicit forbids for unrecognized roles and unverified tool scopes. R(π): `0.4750` → `1.0000`.

Note on trajectory ordering. The round-based trajectory above (structural controls first, attestation controls last) reflects the actual blue-loop execution order: the agent explored structural forbids before identifying the OWA/CWA attestation gap. A separate end-to-end re-run with the Rust PDP as the commit oracle (`scripts/run_s-print18_rust_pdp_loop.py`, greedy ordering by descending weight) follows the path V7→V2→V6→V5→V3→V4, reaching R(π) = 1.0000 in 6 iterations. These two trajectories cover the same policy states and produce the same terminal R(π) = 1.0000, but the greedy run is attestation-first (by weight) rather than structural-first (as discovered by the live agent). The ordering difference is expected: the live agent explored the policy space empirically and surfaced the attestation ceiling through trial; the greedy run applies the optimal-weight ordering with the Rust engine as oracle throughout.

Each iteration: (1) Sonnet proposes a Cedar diff as structured output, (2) the diff is applied to the `.cedar` file, (3) R(π) is computed, (4) the change is committed if R(π) improves (git commit) or reverted if not (git checkout). The online oracle during the loop was the pure-Python Cedar evaluator (concordant with the Rust engine for all structural vectors); all three resulting policy artifacts were subsequently validated against cedar-python 0.1.4. The Cedar policy comments embed the full iteration history (Appendix A).

Stopping criterion. The orchestrator halts the blue loop when R(π) = 1.0000 *or* when no modification achieves an improvement over three consecutive iterations. π_2 reached R(π) = 1.0000 at Exp 9.

Oracle concordance and the contains() bug. During blue-loop iteration, the online robustness oracle was the **pure-Python Cedar evaluator** (duck-typing compatible with `CedarRealEvaluator`, §4.1). Because the pure-Python evaluator does not replicate the Rust engine's `String.contains()` type error — it raises a Python-level exception and propagates DENY — it correctly scored π_2 as R(π) = 1.0000, and the blue loop converged normally at Exp 9. The divergence was discovered only during *post-hoc* validation against cedar-python 0.1.4, which returned R(π) = 0.9444 before the `like`-operator fix (§4.2). The framework did not stall or diverge because the Python oracle had no knowledge of the Rust engine's fail-open behavior: from the online oracle's perspective, V3 was closed by the `contains()` forbid. This is a meaningful architectural limitation: a fully autonomous end-to-end framework routing the commit-criterion evaluation through the production Rust PDP would have discovered the

implementation bug mid-iteration — or stalled at R(π) = 0.9444 — rather than requiring offline human-triggered validation to surface it (§6.4).

Autonomy and production fidelity: a distinction. The 9-iteration live-agent run used a pure-Python Cedar evaluator as the commit oracle; the `String.contains()` fail-open was detected by *human-triggered* post-hoc comparison, not by the autonomous loop. The end-to-end re-run with the Rust PDP as the live commit oracle is production-faithful but scripted (greedy, descending-weight), not LLM-agentic. These were never demonstrated together in a single run. We claim only that (a) the agentic loop converges to R≥0.475 under any correct evaluator, as confirmed by the Python-oracle live run, and (b) the Rust PDP confirms π_2 achieves R=1.0 post-hoc. A fully agentic run using the Rust PDP as the live commit oracle is mechanically feasible (`use_rust_pdp=True`) and is left as a reproducibility exercise. We do not claim that the autonomous loop independently discovered the `contains()` failure; we claim the *framework* surfaces it when the production oracle is wired in.

Quantitative concordance and end-to-end Rust PDP validation. Post-fix, per-vector decision agreement between the Python oracle and the Rust PDP across all 32 (policy, vector) pairs (4 policies × 8 vectors, including the MCP-hardened variant) is **32/32 = 100.0%** with zero standing disagreements (`scripts/validate_rust_pdp_commit.py`). The transient divergence — the single pre-fix disagreement on V3 (`path_traversal_literal`) — existed only while the `contains()` bug remained in the policy; the `like`-operator fix restored full concordance on both engines simultaneously. The dual-oracle gap mattered for security *in the safer direction*: the Python oracle was **stricter** than the Rust PDP (propagating DENY on the unsupported operator rather than silently returning PERMIT), giving the blue agent a false sense that V3 was already closed. To eliminate the oracle-engine gap entirely, we re-ran the hardening trajectory end-to-end with `CedarRealEvaluator` (the official Rust PDP via subprocess, `use_rust_pdp=True`) as the sole commit oracle (`scripts/run_sprint18_rust_pdp_loop.py`). The greedy run (V7→V2→V6→V5→V3→V4, ordering by descending weight) converged to R(π) = 1.0000 in **6 iterations with zero rollbacks**, reaching the same terminal policy as the 9-iteration live-agent run. The `String.contains()` fail-open did not affect the end-to-end run: the greedy oracle applied the `like` operator from iteration 5 onward, which the Rust engine evaluates correctly. This result confirms that the original hardening trajectory is reproducible under the production engine and that no policy certified by the Python oracle fails under the Rust PDP.

4.4 Red Loop

The red agent is an **adaptive adversarial search engine**, not a fixed regression suite. Its primary mode is a **mutation loop** that generates structurally novel, well-typed attack variants: starting from 14 canonical seed requests, the engine applies 7 strategies — `field_value_permutation` (flip boolean context flags), `encoding_variant` (generate novel path-traversal encoding variants), `role_boundary` (cycle through all valid and invalid roles), `action_boundary` (cycle through all valid actions), `resource_type_swap` (swap Resource/Tool/Agent types), `owner_identity_swap` (toggle owner match/mismatch), and `attestation_permutation` (vary attestation source and resource authority) — generating variants not enumerated in the canonical taxonomy. Variants that produce a novel bypass are added to the seed pool for the next round, focusing search on productive regions of the request space.

The red agent operates this adaptive engine on a **regression foundation**: after every blue commit, the 8 canonical vectors in Table 1 (V1–V7 DENY-expected, V8 PERMIT-expected) are evaluated and any regressions flagged. Confirmed bypasses accumulate in a persistent bypass library, guaranteeing that R(π) cannot regress silently. The adaptive loop then explores beyond the canonical frontier.

Results. We ran the adaptive red loop against all three policy variants (`adaptive_redloop_v1.csv`). The baseline policy (π_0, R=0.2000) exposed the engine's discriminatory power: across 6 rounds totalling **6,050 mutations**, the loop found **121 bypasses** (8 canonical seeds + 113 mutation-strategy variants with attack fingerprints not yet in the canonical taxonomy) and never converged, confirming the policy is genuinely porous to adversarial mutation. The hardened agent policy (π_2, R=1.0000) and the MCP-hardened policy each converged at round 2

with **0 bypasses** across **1,540 mutations each** — a combined 3,080 adaptive mutations with no new exploit found. Total adaptive mutations across all three policies: **9,130**.

Adaptive mutation and exhaustive enumeration converge: the adaptive search found 121 exploitable configurations in the baseline and zero in either hardened policy. The exhaustive 768-state enumeration (§5.5) independently confirms the same result from the opposite direction. Both methods explore the same bounded request space; the exhaustive enumeration subsumes the adaptive search and provides complete state-space coverage. §8.1 describes the further extension to SMT-guided exhaustive UNSAT proof as the next step beyond adaptive search.

Ablation: adaptive vs. random mutation baseline (RQ3). To quantify the adaptive strategy's contribution, we ran a random-mutation baseline (`scripts/run_ablation.py`, `results/ablation_study_v1.csv`) using fingerprint-based deduplication and ground-truth filtering (the final policy as oracle — a mutation is only a "real bypass" if the test policy PERMITs it while the final policy DENYs it). Random mutations (single-field perturbations, fixed canonical seeds, no bypass-driven reprioritization, same convergence criterion) were applied across all three policy stages. Against the **final policy** ($R = 1.0000$): both adaptive and random converge at round 2 with **0 fingerprint-distinct bypasses** — confirming the convergence criterion is sound and not an artifact of the adaptive strategy. Against the **baseline policy** ($R = 0.2000$): the random baseline discovers **25 fingerprint-distinct attack classes** in 3,024 mutations and converges at round 3; the adaptive loop explores a superset — 121 tag-distinct bypasses over 6,050 mutations (finer deduplication), with bypass-driven seed reprioritization preventing the early convergence that would have missed the long tail of cross-role and attestation-gap variants. The key finding is not a raw bypass-count difference but the **structural reason for the adaptive advantage**: random mutations on a converging policy can hit the same underlying fingerprint many times, masking residual bypass classes; bypass-driven seed reprioritization ensures the adaptive loop keeps searching productive regions. The orchestrator's primary contribution — detecting the OWA/CWA meta-pattern that neither agent surfaced in isolation — is a qualitative finding not captured by per-round bypass counts.

4.5 Attestation and PEP Integration

The four context attributes `ownership_verified`, `role_attested`, `path_safe`, and `tool_scope_verified` are *externally attested by the PEP* before policy evaluation. In a production deployment: `ownership_verified` is set from an external resource-ownership registry; `role_attested` from a cryptographically signed identity assertion (JWT/signed claim from an IdP); `path_safe` from PEP-level path normalization and canonicalization; `tool_scope_verified` from a tool-capability registry checked against executor-role constraints. The final policy's permits require these flags to be `true`; complementary forbids fire when they are `false` — secure only if the attacker cannot inject the context.

---

5. Evaluation

5.1 Experimental Setup

We evaluate three policy variants (baseline π_0, partial π_1, final π_2) against **eight** vulnerability vectors (Table 1): seven DENY-expected attack vectors (V1–V7) and one PERMIT-expected false-positive check (V8, `over_blocking_false_positive`, $w = 2$). Evaluation uses the Cedar Rust engine (cedar-python 0.1.4 via subprocess bridge). A pipeline-integrity run of 17 sequential evaluations per policy confirmed identical $R(\pi)$ values across all runs — a verification that the subprocess harness is stable under repeated invocation, not a claim about noise resistance (deterministic inputs to a deterministic engine cannot produce variance). Results are logged to `results/evaluation_real_pdp_v2.csv` (engine=`cedar-rust`).

Rationale for V8 (false-positive / over-blocking check). All seven attack vectors (V1–V7) carry DENY-expected requests; a trivial deny-all policy would score $R(\pi) = 1.0$ on V1–V7 alone, making the metric degenerate. V8 closes this gap by encoding four canonical legitimate-access requests that any non-pathological policy must permit: (1) a

reader reading a path-safe Resource, (2) a writer writing its own Resource with PEP ownership attestation, (3) an admin delegating to a non-admin non-self Agent with role attestation, and (4) an executor invoking a PEP-scope-verified Tool. The **false-positive rate** $\text{FP\_rate} = (\text{V8 requests wrongly denied})/4$ is 0% for all three policy variants: no legitimate request is ever over-blocked. V8 carries weight $w_8 = 2$ (tied with `integrity_violation`) and participates in R(π) alongside V1–V7; total weight $\sum w_i = 20$.

The `contains()` bug fix (§4.2) was applied to π_2 before generating the final experimental results; Table 2 reflects the corrected policy.

5.2 R(π) Results

Table 2: R(π) Results by Policy (cedar-python 0.1.4, Rust engine, 8-vector taxonomy including V8). †R_attack = R over V1–V7 only (denominator 18); V8 is excluded to give a pure adversarial-robustness view alongside the joint metric.

POLICY	R(Π) [V1–V8]	R_ATTACK† [V1–V7]	ATTACK VECTORS ACTIVE	FP RATE (V8)
Baseline (π_0)	0.2000	0.1111	6/7	0%
Partial (π_1)	**0.4750**	**0.4167**	3/7 active, 1/7 partial†	0%
Final (π_2)	1.0000	1.0000	0/7	0%

Robustness rises monotonically: 0.2000 (six attack vectors active) → 0.4750 (structural controls applied; V3 partial at B=0.5) → 1.0000 (full attestation). The FP rate is **0% across all three policies**: V8's four legitimate requests are correctly permitted at every hardening stage, confirming no over-restriction is introduced. The R_attack column removes V8 from the denominator; the gap between R(π) and R_attack narrows as more attack vectors close, and both metrics agree at final. †Partial: V5, V6, V7 are *active* (B=0.0); V3 is *partial* (B=0.5) — 2 of 3 path-traversal encoding variants blocked structurally, 1 (Unicode V3.2) requiring PEP attestation. The bound $R(\pi_2) = 1.0000$ holds *within this taxonomy*; the consequential coverage question is which attack classes are omitted (§6.5, §8.1).

Table 3: Ablation — vectors closed by each policy.

VECTOR	WEIGHT	BASELINE	PARTIAL	FINAL
V1 scope_overflow	2	closed*	closed	closed
V2 delegation_abuse	4	active	closed	closed
V3 path_traversal_literal	1	active	**partial** (B=0.5)‡	closed
V4 encoded_path_traversal	1	active	closed	closed
V5 resource_attribute_forgery	2	active	**active**	closed
V6 role_claim_manipulation	3	active	**active**	closed
V7 executor_tool_indirection	5	active	**active**	closed
V8 over_blocking (FP check)	**2**	**closed**	**closed**	**closed**

* V1 is closed in the baseline by Cedar's *default-deny* (there is no permit for a reader performing a write), not by an explicit forbid. V8 is closed at every stage: all four legitimate requests are correctly permitted by baseline, partial, and final policies (FP rate = 0% throughout the hardening trajectory). ‡V3 on π_1: partial's structural `like "*..*"` forbid blocks V3.1 (ASCII `../`) and V3.3 (mid-path `..`) but not V3.2 (Unicode fullwidth dots U+FF0E — a different codepoint from ASCII `.`). Only the final policy closes V3.2 via PEP `path_safe` attestation.

The three bolded `active` entries in the Partial column (V5, V6, V7) define the **core attestation gap**: these vectors remain fully open (B=0.0) after every *structural* control has been applied. V3 additionally exhibits B=0.5 — the only vector with partial mitigation — because its Unicode variant requires PEP attestation. Together, V5, V6, V7 and the

attestation-dependent component of V3 account for 10.5/20 ≈ 52.5% of total risk weight. The maximum R(π) reachable by structural rules is $R^{max}_{struct}(T_8) = 9.5/20 = 0.4750$. The specific bound 0.4750 is a property of this taxonomy's weight assignment; a different weight distribution or a different set of attestation-dependent vectors would yield a different bound. However, the *per-class* result — that each of V5, V6, V7 individually satisfies $B_{struct}(\pi, V) = 0$ for any structural π, and that V3.2 likewise cannot be closed without PEP attestation — is schema-independent (Security Claim 1, §5.3).

Blue loop convergence. The nine-iteration trajectory produced the following per-round R(π) milestones (8-vector taxonomy, total weight 20): 0.2000 (baseline) → 0.2000→0.3500→0.4000→0.4750 (Round 1, structural forbids; V3 reaches B=0.5 once the `like "*..*"` forbid closes V3.1/V3.3, leaving V3.2 as a latent bypass) → 0.4750 (Round 2, OWA bypass confirmed, no further structural progress) → 1.0000 (Round 3, attestation controls). The stagnation at 0.4750 during Round 2 is the empirical signature of the attestation ceiling: no structural Cedar modification can advance R(π) past this point.

5.3 Security Claim 1: The Cedar Attestation Gap

Why can V5, V6, and V7 not be closed structurally? Each depends on a context attribute that the policy can *reference* but cannot *verify*:

- **V5 (resource_attribute_forgery).** A writer self-declares `resource.owner == principal.id`. A forbid of the form `when { context.ownership_verified == false }` fires only if the attacker leaves the flag false. An attacker who controls the request simply sets `ownership_verified = true`, and the writer permit fires on the forged ownership claim.
- **V6 (role_claim_manipulation).** Analogous: an agent self-asserts `role = "admin"`. The role-attestation forbid is bypassed by forging `role_attested = true`.
- **V7 (executor_tool_indirection).** The executor invokes a tool whose internal capabilities exceed the executor's direct permissions. Forging `tool_scope_verified = true` bypasses the executor scope check.

Root cause. Cedar evaluates only what is *stated* in the request context (OWA). If the PEP does not attest the context, every unverified positive claim is accepted at face value. The Cedar PDP has no way to distinguish an *attested* attribute from a *forged* one without external verification infrastructure.

Contribution. Prior ABAC literature [4, 9, 12] documents the PEP/PDP separation as a design principle. Our contribution is to **structurally characterize the per-class attestation gap as a consequence of Cedar's type universe** — an uncloseable structural limit, not a deployment shortcut — and to **quantify its aggregate magnitude** via a severity-weighted taxonomy so that practitioners can measure how much risk weight their attestation infrastructure must cover. The value of Security Claim 1 is not mathematical novelty — the zero-trust principle has been recognized in the ABAC literature for decades — but structural precision grounded in Cedar's Lean formalization: it gives the attestation gap a measurable handle via R(π), whose structural ceiling $R^{max}_{struct}(T) = \Sigma(\text{non-att weights})/\Sigma(\text{all weights})$ is computable for any taxonomy T without running the full empirical framework. Security Claim 1 is stated below; the taxonomy-specific bound follows as a corollary.

> *Security Claim 1 (Cedar Attestation Gap). Let S be any Cedar authorization schema with context attributes $C = \{\alpha_1, \ldots, \alpha_n\}$. Partition C into C_{att} (attributes whose values are supplied by the requesting entity at request time — OWA-controlled, §3.1) and C_{str} (attributes attested by the PEP before evaluation). For any vulnerability class V whose correct-DENY decision requires an adversary-controlled attribute $\alpha \in C_{att}$ to take a specific safe value v_{safe}: there is no Cedar policy π over S that achieves $B(\pi, V) = 1.0$ without PEP attestation of α.*
>
> *Structural grounding (Cedar's Lean semantics, cedar-spec [1]). Cedar's `evaluate` function returns `req.context` verbatim: `evaluate (.var .context) req es = .ok req.context` (`Cedar/Spec/Evaluator.lean`). The `Request.context` field is `Map Attr Value` — a flat, untagged map with no provenance constructor on `Value` (`Cedar/Spec/Request.lean`, `Cedar/Spec/`*

> `Value.lean`). *The adversary sets* $\alpha = v_{safe}$ *in* `req_forged.context`*; the evaluator treats this identically to a legitimately-attested request because both map to the same* `Value` *in the flat context map. No Cedar policy rule can inspect provenance because no provenance field exists in Cedar's type universe. The only defense is to move* α *into* C_{str} *— i.e., PEP attestation of* α *before the request reaches the PDP.*
>
> *Corollary (R_struct^max for arbitrary taxonomy* T*). Let* T *be any vulnerability taxonomy over schema* S*. For vectors with mixed structural/attestation-dependent test requests, let* $B_{struct}(\pi, V)$ *denote the maximum bypass score achievable by structural policy rules alone (i.e., without PEP attestation of any OWA-controlled attribute). For any weight function* W*:*

$$R^{max}_{struct}(T, W) = \frac{\sum_{V \in T} W(V) \cdot B^{max}_{struct}(V) + W(V_8)}{\sum_{V \in T} W(V)}$$

> *where* $B^{max}_{struct}(V) \in \{0, 0.5, 1.0\}$ *is the structurally achievable bypass score for class* V *(0 for fully attestation-dependent classes by Security Claim 1; 0.5 for mixed classes with one attestation-dependent sub-variant; 1 for fully structural classes). For the 8-class taxonomy* T_8 *of Table 1: V1, V2, V4 are fully structural (*$B^{max}_{struct} = 1.0$*, combined weight 7); V3 is mixed (*$B^{max}_{struct} = 0.5$*, weight 1) because V3.2 (Unicode U+FF0E) is attestation-dependent while V3.1/V3.3 are structural; V5, V6, V7 are fully attestation-dependent (*$B^{max}_{struct} = 0$*, combined weight 10). Hence:*

$$R^{max}_{struct}(T_8) = \frac{(2+4+1) \cdot 1.0 + 1 \cdot 0.5 + (2+3+5) \cdot 0 + 2 \cdot 1.0}{20} = \frac{7 + 0.5 + 0 + 2}{20} = \frac{9.5}{20} = 0.4750$$

> *The bound is constructive —* π_1 *achieves it exactly — and tight for this taxonomy. Security Claim 1 (the per-class* $B_{struct}(\pi, V) = 0$ *result for fully attestation-dependent classes) holds for any schema* S*, taxonomy* T*, and weight function* W*; the specific magnitude 0.4750 is a property of* T_8*'s weight assignment and the mixed nature of V3. The practical necessary condition: policy designers targeting* $R(\pi) > R^{max}_{struct}$ *must deploy PEP attestation for every attestation-dependent and mixed-class component in the taxonomy.*

Scope of the Lean machine-check. `lean/SecurityClaim1.lean` (337 lines, standalone, no Mathlib, zero `sorry` / `admit`, theorems `attestation_gap` and `no_policy_achieves_full_blockage`) is a **simplified model**: `Value := String`, context-only policies — documented as a "toy model" in the file's own header. The simplification is deliberate and the key property it checks — `evaluate (.var .context) req es = .ok req.context` — holds verbatim in the full Cedar evaluator (real types: `Value := prim | set | record | ext`; real context: `Map Attr Value`, `Cedar/Spec/Evaluator.lean`, cedar-spec [1]). Policies in the simplified model are functions of context alone; real Cedar policies additionally read `principal` / `action` / `resource` entity attributes, which are PEP-supplied CWA values from the entity store, not OWA context.

The entity-store extension is now machine-checked (Lemma 2), not merely argued. `lean/SecurityClaim2_EntityStore.lean` (216 lines, standalone, same toy-model conventions as `SecurityClaim1.lean`) formalizes the qualitative claim earlier drafts of this paper only stated in prose. Where context is modeled as a field the adversary directly writes into the `Request` (making Theorem `attestation_gap`'s universal forgery possible), entity attributes are modeled the way Cedar's evaluator actually resolves them: the adversary picks *which* entity to reference (`r.resource : EntityUID`), but that entity's attributes are looked up in a separate `EntityStore` populated by trusted infrastructure before the request ever reaches the PDP — not read from the request payload at all. Theorem `entityAttr_depends_only_on_resource` shows entity-attribute values are a function of `(store, r.resource)` only, causally independent of `r.context` / `r.principal`. Theorem `no_universal_entity_forgery` then proves the qualitative converse of `attestation_gap`: there exist a store, an attribute, and a target value that **no** request — for any choice of `resource`, `principal`, or `context`

— can make the entity-attribute lookup return, because forging an entity attribute requires the target value to already exist somewhere in the trusted store; the adversary can only *select* among existing entities (`entity_forgery_is_selection_not_fabrication`), never fabricate a fresh (attribute, value) pair the way `r.context` lets them. Both new theorems check with zero `sorry` / `admit`, depending only on Lean's standard core axioms (`propext`, `Classical.choice`, `Quot.sound`). Together, the two files give both halves of the paper's OWA/CWA claim a machine-checked basis: context attributes are proven universally forgeable, and entity-store attributes are proven *not* to be — the security boundary the `@pep_attested` / `@caller_supplied` annotation proposal (§4.5) is built on is not just an intuition but a theorem about two evaluator code paths with genuinely different adversary-reachability properties. The contribution remains the formal precision of the context-OWA case as the locus of every empirical bypass in this paper (§4.2, §5.6); Lemma 2 closes the entity-store side of the argument without claiming a machine-checked model of Cedar's full type universe (`Value` is still simplified to `String` in both files).

5.4 Multi-Actor Robustness (R_MCP)

The dual-loop framework of §3–§4 hardens a *single* policy π. But production agentic deployments rarely consist of one policy: under the Model Context Protocol (MCP) [13], multiple agents — often from different vendors — compose tool calls, and each node enforces its own Cedar policy. This raises a question the single-policy $R(\pi)$ cannot answer: does hardening *one* node guarantee system-level robustness? To answer it empirically we model a two-node MCP graph G with one Cedar policy per node and measure $R_{MCP}(G, v) = \min_{p} \min_{v_i \in p} R(\pi_{v_i}, v)$ — the robustness of the weakest policy on any authorization path from requester to tool. We extend the taxonomy with **V_cv (cross-vendor context forgery)**, an attestation vector in which node A attests `ownership_verified=true` for a resource owned by node B's vendor; the extended taxonomy V1–V8+V_cv carries total weight 22. We evaluate three graph configurations against the Rust PDP (cedar-python 0.1.4).

We extend the taxonomy with two V_cv variants to distinguish the CWA and OWA threat surfaces. **V_cva (cross-vendor structural forgery)** — the cross-vendor PEP correctly identifies itself as vendor-A but attests ownership of a vendor-B resource; the typed `attestation_source != resource_authority` forbid detects and blocks this *under the closed-world assumption (CWA)* that context is injected by a trusted PEP. **V_cvb (adversarial attestation forgery)** — the attacker *forges* `attestation_source` to equal `resource_authority`, bypassing the forbid; this case requires a cryptographic trust anchor (PKI, TPM attestation) external to Cedar and is *uncloseable by policy alone*, which is the exact claim of Proposition 2. V_cva and V_cvb each carry weight 2; the extended taxonomy V1–V8+V_cva+V_cvb has total weight 24. We evaluate three graph configurations (all cross-vendor, `vendor_same=False`) against the Rust PDP.

Table 5: Multi-actor robustness R_MCP across graph configurations (V1–V8+V_cva+V_cvb, total weight 24, Rust engine). All graphs A–C use a cross-vendor edge (`vendor_same=False`), activating both V_cva and V_cvb. R values in parentheses are per-node scores. ‡Graph C is logically derived: R_MCP = min(R_hardened, R_final) = min(0.9167, 0.8333) = 0.8333.

GRAPH	NODE A POLICY	NODE B POLICY	R_MCP[V1–V8+V_CVA+V_CVB]	V_CVA CLOSED?	V_CVB CLOSED?
A	baseline (R=0.1667)	final (R=0.8333)	0.1667	No	No
B	mcp_hardened (R=0.9167)	mcp_hardened (R=0.9167)	**0.9167**	**Yes**	No
C‡	mcp_hardened (R=0.9167)	final (R=0.8333)	0.8333	A: Yes / B: No	No

Graph A confirms weakest-link semantics: baseline fails both V_cva and V_cvb. **Graph B is the decisive result**: `mcp_hardened_policy.cedar` on both nodes closes V_cva (the structural honest cross-vendor gap) via `context.attestation_source != context.resource_authority`, achieving R_MCP=0.9167 — the **maxim-**

um achievable without a cryptographic trust anchor. V_cvb (B=0.0 on every policy) remains open on all graphs. This is by construction: V_cvb is defined so that `attestation_source` equals `resource_authority`, making the forbid condition vacuously false regardless of which policy evaluates the request — Cedar has no mechanism to distinguish a forged value from a legitimate same-vendor one without a cryptographic trust anchor. V_cvb therefore illustrates the OWA boundary described in Proposition 2 rather than confirming it empirically: Proposition 2 is an argument about Cedar's expressive limits, and V_cvb is a concrete instantiation of that argument, not independent evidence of it.

Semantic reduction for Proposition 2. Under Cedar's operational semantics (Cutler et al. [1], CWA evaluation), all context attribute values are taken as authoritative at evaluation time; the PDP performs no cryptographic verification of provenance. For any Cedar policy π_T that guards V_cvb via `when { context.attestation_source != context.resource_authority }` and any adversarial request r_adv where the attacker sets `context.attestation_source = context.resource_authority = "vendor-b"`, the condition evaluates to `"vendor-b" != "vendor-b" = false`, the forbid does not fire, and the existing permit rule fires — `eval(π_T, r_adv) = PERMIT` for every π_T expressible over the schema without a cryptographic trust anchor. No policy rewrite can change this outcome because Cedar has no primitive to distinguish a forged string value from a legitimate one in the evaluation context. The only fix is to introduce a pre-evaluation trust anchor `Att*` (PKI, TPM, federated IdP) that validates `attestation_source` before it enters the context — precisely the precondition in Proposition 2's conclusion.

Cross-vendor forgery and Proposition 2. The reason V_cv cannot be closed within the original schema is that `ownership_verified` is a *boolean*: Cedar evaluates whether the flag is true, but cannot distinguish *which vendor's* PEP signed the attestation. The only way to close V_cv is to extend the schema (`agent_schema.cedarschema`) with typed attestation provenance — `attestation_source: String` and `resource_authority: String` — and add the forbid `forbid when { context.attestation_source != context.resource_authority }`. Without this schema extension, every policy in the graph scores $B = 0$ on V_cv regardless of its structural sophistication, an empirical confirmation of **Proposition 2** (the cross-vendor attestation bound, §8.1): no per-node hardening alone closes a cross-vendor forgery vector; the fix is typed provenance, not more rules. The take-away is that R_{MCP} is the minimum over all nodes — the chain is exactly as robust as its weakest link.

5.5 Delegation Chain Robustness (R_chain) and Coverage

Where R_MCP composes policies across a tool-call graph, $R_{chain}(\pi_A, \ldots, \pi_n, v) = \min_i R(\pi_i, v)$ composes them along a *delegation* chain A → B, with a delegation-compatibility coefficient $\delta(\pi_A, \pi_B) \in [0, 1]$ measuring how faithfully B inherits A's restrictions. We add **V_dc (delegation chain amplification)** to the taxonomy (V1–V8+V_dc, total weight 25) and evaluate three chains against the Rust PDP.

Table 6: Delegation chain robustness R_chain (V1–V8+V_dc, total weight 25, Rust engine).

CHAIN	POLICY A (ROOT)	POLICY B	R_CHAIN	Δ(A,B)	OBS. 1 (R=Δ=1?)
1 — weak root	baseline	partial	0.1600	0.7500	NO
2 — partial root	partial	final	**0.3800**	**0.7500**	NO
3 — perfect chain	final	final	1.0000	1.0000	**YES**

Proposition 3 (a weak root caps the chain) is confirmed: Chain 1 has a baseline root with $R(\text{baseline, V1-V8+V\_dc}) = 0.1600$, and $R_{chain} = 0.1600$ is determined entirely by that root, *independent of* π_B. Chain 2's $R_{chain} = 0.3800$ reflects the partial policy's R over V1–V8+V_dc: $R(\pi_1, T_{dc}) = 9.5/25 = 0.3800$ (V3 B=0.5 due to the Unicode bypass, same reasoning as the main metric). **Observation 1** (isolation): $R_{chain} = 1$ iff every $R(\pi_i) = 1$ *and* every $\delta = 1$ — a direct consequence of the min operator definition, satisfied only by Chain 3.

Exhaustive coverage. To bound what the static 8-vector taxonomy omits, we enumerated all **768** well-typed Cedar request states ($4 \times 5 \times 2 \times 2^4$ over the schema). Of these, 656 (85.4%) deny under all policies, 36 (4.7%) are legitimate permits, and 76 (9.9%) are attestation-gap bypass states. The canonical taxonomy coverage is $C(T_8, \text{Schema}) = 4/76 = 5.3\%$: exactly 4 of the 76 bypass-capable states are represented by a canonical DENY-expected test vector with the same $(role, action, resource\_type, context\_flags)$ fingerprint (V3/V4→1 state, V5→1, V6→1, V7→1). V1 cross-role attacks target states that all three policies deny (DDD, not bypass-capable); V2 tests a resource configuration not enumerated in the sweep (admin-target Agent); V8 PERMIT-expected requests are excluded. The remaining 72 bypass states cluster into four groups by $(role, action, resource\_type)$: writer+write+Resource without `ownership_verified` (11 states), reader/writer+read/list/write+Resource without `path_safe` (39 states), executor+execute+Tool without `tool_scope_verified` (11 states), and admin+delegate+Agent without `role_attested` (11 states). Three of these four groups share their $(role, action, resource\_type)$ structure with an existing canonical vector (V5, V7, V6 respectively) — their DENY-request fingerprints are identical, so adding a new vector for each would inflate the denominator without genuine new coverage. The one genuinely new structure is the writer+list+Resource group within the 39-state cluster, covered by V9 (§5.5 below). Notably, the intermediate score $B = 0.5$ was **never observed** across all 768 states — the partial policy behaves identically to the baseline on every enumerated state (the 768-state schema uses template resource paths such as `/data/x` that do not trigger partial's `like "*..*"` F3 forbid), confirming the partial policy is binary relative to the final policy *in the enumerated state space*. V3's empirical B=0.5 arises at the **canonical-request level** only: the canonical V3.2 test vector uses path `". . /etc/secrets"` (Unicode fullwidth dots), which the `like "*..*"` pattern does not match — a traversal variant that generic enumeration over template paths cannot capture.

Extended taxonomy (V9). The exhaustive enumeration identifies one $(role, action, resource\_type)$ group absent from all V1–V8 canonical requests: **writer+list+Resource** without `path_safe` attestation. This motivates **V9** (path_attestation_gap, $w = 1$, information_disclosure). V9 is *fully attestation-dependent*: the partial policy permits writer+list+Resource because its forbid rules match only structural traversal markers (`.` / `%2e`) — a non-traversal path with `path_safe=False` passes those forbids and hits the writer permit. The final policy requires `context.path_safe==true` in all Resource permits, closing the gap via PEP pre-normalisation. Security Claim 1 applies: forging `path_safe=true` on a non-traversal path bypasses any forbid-on-false rule, so only PEP attestation provides a CWA guarantee. Empirically, the final policy closes V9 — $R(\pi_2, T_9) = 21/21 = 1.0000$ — while the partial policy leaves it open ($R(\pi_1, T_9) = 9.5/21 = 0.4524$), confirmed by `scripts/run_sprint13_extended.py`, `results/sprint13_extended_v1.csv`. The extended taxonomy $T_9 = V1\text{–}V9$ (total weight 21) raises the attestation-dependent fraction to $(10 + 1 + 0.5)/21 = 11.5/21 \approx 54.8\%$ of total risk weight, with structural ceiling $R^{max}_{struct}(T_9) = 9.5/21 = 0.4524$ — confirmed constructively by π_1. V9's second canonical request (writer+list+Resource, `ctx={ownership_verified}`) adds one new covered fingerprint ($C(T_8) = 4/76 = 5.3\%$; $C(T_9) = 5/76 = 6.6\%$).

5.6 Cedar Schema-Reality Gap: Specification-Discovery at Scale

Motivation. Security Claim 1 establishes that any Cedar context attribute the adversary controls creates an uncloseable bypass class. But which attributes are adversary-controlled (OWA) and which are PEP-attested (CWA) is not encoded anywhere in Cedar's policy text: the expression `context.is_authenticated` is syntactically identical whether the value is caller-supplied or independently verified by a JWT-validating PEP. We call this the **Schema-Reality Gap**: the structural absence of provenance information from Cedar policy text. The gap has a concrete operational consequence: formal SMT analysis and LLM-based auditing must reason about context attribute provenance — information that Cedar's grammar does not encode. We measure the gap empirically by asking: given only a Cedar policy and schema, how accurately can automated analysis determine which attributes are realistically adversary-controlled?

Corpus. We assembled 25 Cedar policies across five application domains: healthcare (H01–H05), financial services (F01–F05), DevOps (K01–K05), multi-tenant SaaS (S01–S05), and IoT (I01–I05). Each policy was authored with a mock PEP that defines which context attributes the PEP independently computes (PEP-attested) and which are passed through from the caller (OWA). Ground truth was derived from PEP code structure, not from LLM labeling. The corpus totals 75 context attributes: 26 OWA (34.7%) and 49 PEP-attested (65.3%). Of the 25 policies, 20 have at least one realistic bypass (an OWA attribute that, when forged, produces a PERMIT the correct deployment would DENY); 5 do not (all context attributes in these 5 policies are PEP-attested — no OWA attribute exists for an adversary to forge). All scripts and corpus entries are in `scripts/experiment_discovery/`.

Table 7: Specification-Discovery Corpus.

DOMAIN	POLICIES	ATTRS	OWA	PEP	REAL BYPASSES
Healthcare	5	16	5 (31%)	11 (69%)	4
Financial	5	15	5 (33%)	10 (67%)	4
DevOps	5	14	7 (50%)	7 (50%)	4
SaaS	5	13	4 (31%)	9 (69%)	4
IoT	5	17	5 (29%)	12 (71%)	4
Total	**25**	**75**	**26 (34.7%)**	**49 (65.3%)**	**20**

Z3 Baseline — All-OWA. We encoded each policy's boolean Cedar context conditions as Z3 propositional formulas, treating all 75 context attributes as adversary-controllable (the standard assumption in SMT-based policy analysis tools such as cedar-symcc and AWS Zelkova). Z3 returns SAT for **25/25** policies in ~90ms total (3.5ms/policy). Ground truth: **only 20/25** have exploitable bypasses. The remaining 5 are false alarms — Z3 finds a satisfying assignment, but the controlling attribute is PEP-attested in the real deployment. This 5-policy gap (20%) is the Schema-Reality Gap quantified. The result is not a Z3 limitation: any formal tool that must conservatively treat Cedar context attributes as fully adversary-controlled will produce the same false alarms. The gap is not in the tool; it is in the language.

LLM Specification Discovery. We ran an LLM-based discovery workflow in which a judge model receives each policy's Cedar text (without PEP code) and classifies each context attribute as OWA or PEP-attested, with chain-of-thought reasoning and structured output. Results over 75 attributes:

Table 8: Specification-Discovery Results (OWA = positive class).

METHOD	PRECISION	RECALL	F1	ACCURACY
All-OWA (trivial baseline)	0.347	1.000	0.515	34.7%
All-PEP (trivial baseline)	—	0.000	0.000	65.3%
LLM Discovery	**0.490**	**0.962**	**0.649**	**64.0%**

The LLM achieves recall 96.2% (misses 1 of 26 true OWA attributes) but precision only 49.0%: it labels 26 PEP-attested attributes as OWA. F1=0.649 modestly exceeds the trivial all-OWA baseline (0.515) but the margin is narrow, because both the LLM and the trivial baseline are penalized by the same root cause: the policy text does not encode provenance. A Cedar policy auditor — human or automated — who sees `context.consent_client == resource.owner_id` cannot determine whether `consent_client` is computed by a consent-verification service or passed from the calling agent. The LLM's 26 FPs are not classification errors in the ordinary sense; they are cases where the LLM correctly identifies that the attribute *could* be OWA-controlled and *should* be attested — an interpretation that is semantically defensible but empirically incorrect for these deployments. This is Security Claim 1 applied at the provenance-discovery level: the gap is structural, not epistemic.

Per-ambiguity breakdown (TP/TN/FP/FN): low (n=39): 0/18/21/0; medium (n=28): 17/5/5/1; high (n=8): 8/0/0/0. **Low-ambiguity PEP-attested attributes contribute 21 of 26 false alarms (81%)** — these are attributes whose naming conventions signal PEP-computed operational or credentialing state (e.g., `context.on_duty`, `context.licensed_prescriber`), yet the LLM labels them OWA at a 54% rate (21/39). High-ambiguity OWA attributes are classified perfectly (precision=1.0, recall=1.0); medium-ambiguity achieves precision=0.77, recall=0.94. The dominant FP source is therefore not semantic ambiguity in the attribute name but the structural absence of provenance information in Cedar policy text: even names that unambiguously suggest PEP-computed state are flagged as potentially OWA when the policy contains no signal to exclude that possibility. This is Security Claim 1 at the auditing layer — the same syntactic opacity that prevents Cedar's PDP from verifying attribute provenance at runtime also prevents an LLM auditor from inferring it statically.

Real-World Illustration: tax_preparer and document_cloud. Two policies from the cedar-policy/cedar-examples repository ground the Schema-Reality Gap in published Cedar artifacts, evaluated against the Rust PDP (cedar-python 0.1.4, `scripts/experiment_discovery/real_policy_pdp_test.py`):

tax_preparer (`context.consent`): The policy conditions a Professional's access on `context.consent_client == resource.owner_id && context.consent_region_match`. A caller supplying `consent_client = "client_alice"` satisfies the policy's consent check — but whether this claim is backed by a real consent record in a consent store, or is simply asserted by the calling agent, is invisible to Cedar. Critically, the forged-consent request and a legitimately-attested request are **byte-for-byte identical at the PDP input layer**: same principal, same resource, same context values. The Rust PDP returns PERMIT for both, as it must — the inputs are indistinguishable. No Cedar policy modification can separate them because Cedar has no provenance primitive. This is Security Claim 1 instantiated on a published, real-world Cedar policy: the correct-DENY decision for an unauthenticated consent claim requires PEP verification that is structurally absent from the Cedar layer.

document_cloud (`context.is_authenticated`): The policy includes `forbid (principal, action, resource) when { !context.is_authenticated }` — an authentication gate applied to all actions. One of its permit rules allows access to public documents: `permit (principal == Public::"public", action == Action::"ViewDocument", resource) when { resource.publicAccess == "view" || resource.publicAccess == "edit" } unless { resource.isPrivate }`. An unauthenticated caller acting as `Public::"public"` on a public document (publicAccess="view", isPrivate=False) is blocked by the auth-forbid when `is_authenticated = False`. Forging `is_authenticated = True` removes the forbid; the public-access permit fires. Rust PDP: `is_authenticated=False` → DENY; `is_authenticated=True` (forged) → **PERMIT**. This is Security Claim 1 instantiated on a second published Cedar policy: the forged request is indistinguishable from a legitimately-authenticated one at the PDP input layer. The exploit surface is the intersection of OWA attribute control and permit-path reachability; once an entity type (`Public::"public"`) matches a permit whose sole remaining gate is the OWA attribute, the bypass is unconditional.

Design Contribution: Cedar Provenance Annotations. The Schema-Reality Gap is closeable by making provenance explicit in Cedar schema. We propose a schema-level annotation system:

```
// Current Cedar (provenance-free)
context: {
  is_authenticated: Bool,
  consent_client: String,
  path_safe: Bool,
}

// Proposed annotation (provenance-explicit)
context: {
  @caller_supplied is_authenticated: Bool,  // OWA — PEP must verify before accepting
  @caller_supplied consent_client: String,  // OWA — requires external consent store check
```

```
  @pep_attested path_safe: Bool,            // CWA — PEP pre-normalizes and attests path
}
```

`@caller_supplied` marks attributes the caller controls; `@pep_attested` marks attributes independently computed by the PEP. We evaluate the annotation proposal empirically using the 25-policy corpus (`scripts/run_sprint18_annotation.py`). Annotation-aware analysis uses a **per-permit-rule OWA discriminant**:

$$\exists(\vec{v}_{OWA}, \vec{v}_{PEP}) : \text{PERMIT}_i(\vec{v}_{OWA}, \vec{v}_{PEP}) \wedge \neg\text{PERMIT}_i(\mathbf{0}_{OWA}, \vec{v}_{PEP})$$

where $\mathbf{0}_{OWA}$ sets all `@caller_supplied` attrs to False (safe default). The policy is flagged iff any permit rule i has an OWA-driven bypass. For policies with no OWA attrs (the 5 false-alarm policies), $\vec{v}_{OWA} = \mathbf{0}_{OWA}$ trivially, so the discriminant is $\text{PERMIT} \wedge \neg\text{PERMIT} = \bot$ → UNSAT → no false alarm. For true-bypass policies, the discriminant is SAT iff some permit rule requires OWA forgery.

Empirical result (25-policy corpus, `results/sprint18_annotation.json`):

MODE	TP	FP	FN	TN	PREC	REC	F1
All-OWA Z3 baseline	20	5	0	0	0.800	1.000	0.889
OWA-discriminated (@pep_attested)	20	0	0	5	1.000	1.000	**1.000**

All 5 false alarms eliminated, no true bypasses missed. Attribute-level: all-OWA F1=0.515 → annotated F1=1.000. The annotated discriminant achieves F1=1.000 because the annotations record the provenance ground truth established by PEP code inspection. This result is expected given correct annotations — the contribution is not the F1=1.000 figure itself, but the measurement of the gap: without provenance annotations, neither SMT (F1=0.515) nor LLM (F1=0.649) can determine attribute provenance from Cedar policy text alone. The annotations provide information that the policy text structurally cannot. The proposal closes the Schema-Reality Gap in SMT analysis without modifying Cedar's runtime evaluation semantics — annotations are a static metadata layer.

Corpus stratification. The 25-policy corpus contains 4 policies adapted from public cedar-policy/cedar-examples sources (H01: healthcare emergency override, H02: prescription ordering, K01: k8s image signing gate, S01: document_cloud multi-tenant access) and 21 synthetic policies. A stratified evaluation confirms the annotation approach generalizes to real policies:

CORPUS	METHOD	TP	FP	FN	TN	F1
Real adapted (4)	All-OWA Z3 baseline	3	1	0	0	0.857
Real adapted (4)	OWA-discriminated	3	0	0	1	**1.000**
Synthetic (21)	All-OWA Z3 baseline	17	4	0	0	0.895
Synthetic (21)	OWA-discriminated	17	0	0	4	**1.000**
Full (25)	All-OWA Z3 baseline	20	5	0	0	0.889
Full (25)	OWA-discriminated	20	0	0	5	**1.000**

F1=1.000 for both real and synthetic subsets: the annotation approach is not overfit to the synthetic majority. S01 (document_cloud, real cedar-examples) is the one real false alarm correctly reclassified by the annotated analysis (its false alarm is OWA-attribute-free: all context attrs are PEP-attested, making the discriminant collapse to $\bot$).

Out-of-sample validation. To address the within-corpus evaluation concern, we apply the annotation-based discriminant to 3 cedar-examples policies not in the 25-policy corpus. Each policy is annotated by domain review (not by the discriminant): `streaming_service` (temporal gating via `context.now.datetime`, `context.now.localTimeOffset` — annotated `@caller_supplied`; the child-profile bedtime restriction is bypassable by clock forgery), `photoapp` (`context.judgingSession` — annotated `@caller_supplied`; any

PhotoJudge-role member can forge `judgingSession=True`), and `sales_orgs` (`context.target.job`, `context.targetUser.job`, etc. — annotated `@pep_attested`; these entity attributes are loaded from the Cedar entity store, not from the inline context record, and cannot be forged by the caller).

POLICY	CONTEXT ATTRS	ANNOTATION	DISCRIMINANT	GROUND TRUTH	VERDICT
streaming_service	now.datetime, localTimeOffset	@caller_supplied	SAT (bypass)	Real bypass (clock forgery)	TP
photoapp	judgingSession	@caller_supplied	SAT (bypass)	Real bypass (session forgery)	TP
sales_orgs	target.job, targetUser.job	@pep_attested	UNSAT (no bypass)	No OWA bypass (entity store)	TN

Out-of-sample: TP=2, FP=0, FN=0, TN=1 → F1=1.000. The `sales_orgs` case exercises a non-trivial edge: entity-typed context fields whose attribute *values* are store-attested even though the entity *selection* is caller-supplied — correctly classified as `@pep_attested` by domain review.

Threats to validity. Two limitations remain. First, the out-of-sample corpus is small (3 policies); a larger-scale evaluation on independently-authored Cedar policies with associated PEP code would strengthen the generalizability claim. Second, all 5 in-corpus false alarms are zero-OWA policies: the annotation discriminant eliminates them trivially because $\vec{v}_{OWA} = \mathbf{0}_{OWA}$ always. The harder case — a policy *with* caller-supplied attributes where the false alarm is caused by a PEP-attested attribute creating an indistinguishable permit path — does not appear in the current 28-policy dataset (25 + 3 out-of-sample). Validation on policies of that type remains open.

AI-simulated independent-annotator pilot, blind, third-party (Sprint 54). §8.1 item 2 identifies the fix above (n=3 → n≥20, blind, third-party) as the one extension in this paper's future-work list that "cannot be executed by further automated audit work — it requires a second qualified human annotator's time." We do not yet have that human study. As a preparatory pilot, we ran an independent Claude (Opus) instance as a simulated annotator, blind to this paper's own SMT/LLM classifier outputs, its annotation script, and its result files (verified: the pilot's process never opened this manuscript's own source, `scripts/run_sprint18_annotation.py`, `results/sprint18*`, or any `src/cedar_engine` discriminant code). The pilot located 21 real third-party Cedar policies across 5 independent GitHub repositories (`Ziack/s2s-m2m-identity-stack`, `upbound/kubernetes-cedar-authorizer`, `openshift-online/rosa-hyperfleet-api`, `hupe1980/mako`, `srohatgi/permissions` — none overlapping the corpora used elsewhere in this paper as H1/H2 evidence) with real, in-repo PEP/request-construction source code, and labeled 62 context/entity attributes strictly from that code: 48 `@pep_attested`, 5 `@caller_supplied`, 9 honestly left `UNRESOLVED` where no construction site could be found rather than guessed (`results/sprint54_independent_annotator_blind_study.json`).

Two of the 62 labeled attributes land exactly on the harder case this section names as absent from the 28-policy dataset. In `upbound/kubernetes-cedar-authorizer`'s `localdev/mount/e2e.cedar`, a `create`-secret `permit` gates on both `resource.request.v1.type` (caller-supplied: it is the literal object carried in the incoming admission request body, per `src/cedar_authorizer/kube_invariants.rs`'s own documented provenance table) and the requesting `ServiceAccount`'s identity (PEP-attested: populated by the kube-apiserver's authenticated `SubjectAccessReview`, not the request payload) — we independently re-verified this citation against the live GitHub source and confirm the code says what the pilot reports. In `openshift-online/rosa-hyperfleet-api`'s `developer-with-guardrails.cedar`, caller-supplied `context.requestTags` / `context.tagKeys` permit paths are bounded by a production-environment `forbid` keyed on `resource.tags["Environment"]`, which the pilot traced to the stored-resource path (`pkg/authz/authz.go`) rather than the caller-controlled request body — labeled `@pep_attested` at medium confidence, since the same handler also accepts a `ResourceTags` field directly from the request body in a separate code path, a caveat the pilot's own output flags rather than glossing over.

This is a real, code-grounded finding, not a synthetic construction — but it does not close item 2 of §8.1. An AI annotator, however carefully blinded, is not a second qualified human, and this pilot cannot itself establish inter-rater agreement or an independent human ground truth; its value is narrower: it demonstrates the harder case is not merely a hypothetical gap in this paper's corpus design, and it substantially reduces the corpus-discovery burden for whoever runs the human study next (the same 5-repo, 62-attribute corpus, or a superset of it, is a ready-made starting point). We report this pilot's numbers honestly as pilot numbers, not as a substitute for the human out-of-sample F1 this paper still lacks.

Automated provenance annotation via static PEP code analysis (Sprint 23). One operational question the annotation experiment leaves open is *how the `@caller_supplied` / `@pep_attested` labels are produced in practice*. Manual PEP code inspection is accurate but does not scale. We show the labelling can be automated mechanically: in PEP code, OWA attributes are assigned via pass-throughs (`context['attr'] = request.context.get('attr', default)`) while PEP-attested attributes are assigned from independent computations (`context['attr'] = system_api.verify(...)` or `context['attr'] = db.lookup(...)`). A static analyzer (`scripts/run_sprint23_pep_static_analysis.py`) matches this write-pattern with a single regex over the `request.context.get(...)` call form, classifying the attribute as `@caller_supplied` when a pass-through is detected and `@pep_attested` otherwise.

On the 25-policy corpus (75 attributes), the static analyzer achieves:

METHOD	INPUT	TP	FP	FN	TN	F1
All-OWA baseline	policy text	26	0	49	0	0.515
LLM classifier (GPT-4)	policy text + attr name	—	—	—	—	0.649
Static PEP analysis	PEP source code	26	0	0	49	**1.000**

Policy-level: TP=20, FP=0, FN=0, TN=5 → F1=1.000 in 10.1ms on 25 policies.

Scope and limitations. The F1=1.000 result reflects a clean synthetic corpus: all 75 `pep_code` snippets are canonical write-patterns (direct `request.context.get()` pass-through or unambiguous API call) with no obfuscated or sanitizer-wrapped pass-throughs (e.g., `context['attr'] = strip(request.context['attr'])`). On such instances the discriminant is essentially a tautology — the `is_owa` labels and the `pep_code` write-patterns were co-authored with the same convention, so F1=1.000 does not validate the method beyond self-consistency. The appropriate reading of this result is engineering: **a CI/CD linter over PEP code is both sufficient and practical for automated annotation emission** under the common convention that pass-throughs remain syntactically identifiable. The harder validation — applying the same write-pattern rule to PEP code from independent implementations where `is_owa` was established from deployment knowledge rather than code inspection — is partially addressed in Sprint 24 below; full coverage (n≥200, mixed OWA/PEP-attested) remains open as §8.1 item (6). The LLM comparison (F1=0.649 on policy text vs. F1=1.000 on PEP code) reflects a **modality gap**, not a method improvement: both inputs are available in deployment; policy text alone is structurally insufficient (it encodes no provenance), while PEP code contains the answer directly. The implication is not "static analysis beats LLM" but "provenance is in the PEP, not the policy, which is exactly why Cedar schema annotations are the correct long-term interface."

External corpus survey (Sprint 24). A qualitative survey of 6 independently-authored Cedar policies from the JanssenProject/jans Cedarling codebase [github.com/JanssenProject/jans] surfaces a structural finding about real-world Cedar deployments. Cedarling is a production JWT-backed PDP authored by the Gluu team; its policies use a nested context pattern `context.tokens.TOKEN.CLAIM` where every attribute comes from a JWT token verified via RS256/ES256 against a trusted issuer — not from a flat caller-supplied `context.ATTR_NAME`. This is architecturally different from our 25-policy corpus: there is no `request.context.get()` passthrough because there is no single flat context dictionary. The raw caller-supplied channel (`RequestUnsigned.context: Value` in `authz/request.rs`) is a separate code path that these policies do not use.

This reveals two limitations of the current approach. First, the write-pattern discriminant was designed for flat `context.ATTR_NAME` PEPs; JWT-first PEPs (Cedarling, Amazon Verified Permissions with OIDC) use `context.tokens.*` and require the discriminant to trace JWT claim extraction rather than dict passthrough. Second, the Z3 bypass analysis targets flat boolean context flags; the nested record structure requires schema extension before the bypass pipeline can be applied. The qualitative implication is that open-source Cedar deployments may provide fewer OWA/PEP-attested mixed examples than expected, because JWT-backed architectures eliminate the OWA channel by design. Full external TP/FN validation requires access to Cedar policies that do expose caller-supplied context alongside PEP-attested context — an architectural combination present in our synthetic corpus but not in the Cedarling survey (`scripts/run_sprint24_external_corpus.py`).

External bypass prevalence audit using actual cedar-examples files (Sprint 26). We cloned the `cedar-policy/cedar-examples` repository (Apache-2.0, Cedar team at AWS, commit release/4.11.x) and audited all 7 policies in `cedar-example-use-cases` against the Rust PDP (`scripts/run_sprint26_cedar_examples_audit.py`). Actual policy files are used; no policy text is author-reconstructed. Ground truth OWA labels come from cedar-examples documentation and schema files.

ID	POLICY	CONTEXT ATTR	TYPE	CLASSIFICATION
EX-01	document_cloud	`is_authenticated`	Bool	**OWA_BYPASS_CONFIRMED**
EX-02	tax_preparer	`context.consent.client`	EntityUid	OWA_COMPLEX
EX-03	streaming_service	`context.now.datetime` + `localTimeOffset`	Datetime+Duration	OWA_COMPLEX
EX-05	sales_orgs	`context.target.job`	EntityUid (store)	OWA_COMPLEX
EX-06	github_example	(none)	—	TN_NO_CONTEXT
EX-07	hotel_chains	(none)	—	TN_NO_CONTEXT
EX-08	tags_n_roles	(none)	—	TN_NO_CONTEXT

OWA ground truth comes from cedar-examples README documentation, not from this paper's authors. Two specific quotes establish external ground truth for the two confirmed bypasses:

(EX-01, document_cloud) `forbid (principal, action, resource) when { !context.is_authenticated }`. The cedar-examples deployment notes treat `is_authenticated` as an inline boolean the application populates in the request context — no independent PEP verification is documented. Rust PDP result on the actual policy file: `is_authenticated=False → DENY` (control), `is_authenticated=True (forged) → PERMIT`. Sole gate: forging this one attribute fully removes the forbid.

(EX-02, tax_preparer) cedar-examples README (verbatim): *"A consent is modeled as a Consent entity, which is passed in with the authorization request's context."* OWA by cedar-examples design. The cedar-examples entity store (entities.json) is loaded. Using the `__entity` JSON format for EntityUid values in context (the standard Cedar JSON representation, matching cedar-examples test cases), the Rust PDP confirms: wrong consent.client (WrongClient) → DENY (control), forged consent.client=Ramon (actual document owner) → PERMIT. The attack: a professional who is legitimately permitted (Rule 1a fires for Bob+DEF) forges the consent record to claim the client granted consent — bypassing the consent gate without the client's actual authorization.

Three policies (TN_NO_CONTEXT): github_example, hotel_chains, tags_n_roles have no `context.*` references; authorization depends entirely on entity relationships. Structurally immune to context forgery.

Two policies (OWA_COMPLEX): streaming_service (context.now: datetime extension type; cedar-python 0.1.4 schemaless mode cannot evaluate `.offset().toTime()` — the DENY test case from cedar-examples wrongly returns PERMIT in schemaless mode); sales_orgs (context.target.job resolved from entity store, not inline context).

Result: 2/7 (29%) cedar-examples policies have OWA-forgeable permit paths confirmed by the Rust PDP on actual policy files with OWA ground truth from cedar-examples README. 2/7 OWA_COMPLEX (structural OWA; Cedar CLI + schema required for Rust PDP confirmation). 3/7 TN by construction. Full results: `results/sprint26_cedar_examples_audit.json`.

Extended Production Corpus Audit (Sprint 27). To test whether OWA-forgeable context patterns extend beyond cedar-examples tutorials, we systematically searched the public Cedar ecosystem on GitHub. Using the GitHub Code Search API (authenticated), we queried `.cedar` files containing `context.` — finding **1,210 Cedar policy files** across 500+ repositories. We audited 14 production policies from 8 organizations, applying a **source-code provenance methodology**: for each policy, we locate the request-construction code (or authoritative documentation) that determines whether each security-critical context attribute is PEP-attested (trusted infrastructure) or OWA-forgeable (caller-supplied). Where source code was accessible, we verified provenance by tracing the specific function and file:line where the Cedar context dict is constructed.

Provenance-verified True Negatives (source code checked):

(P4) sondera-ai/sondera-coding-agent-hooks `policies/base.cedar` — A Rust harness for coding-agent governance (`crates/harness/src/cedar/transform.rs`). The `build_request()` function invokes `sondera_signature::scan()` (YARA engine) and `self.data_model.classify()` (IFC classifier) on the actual event content *before* calling Cedar. Context fields `context.signature.severity`, `context.signature.categories`, `context.label`, and `context.policy.compliant` are all harness-computed from the actual shell command or file content — not from agent-supplied metadata. Cedar forbid rules gate on harness-derived YARA results, not on the raw command string. **TN_PEP_ATTESTED** (code-verified at `transform.rs`).

(P5) fronalabs/frona `resources/policy/frona.cedar` — A Rust AI agent framework. The `self-source` permit gates on `context.paired_addresses.contains(resource.sender.address)`. Source at `crates/frona-server/src/chat/channel/manager.rs:96–100` shows: `paired_addresses = channel.user_address.as_ref().and_then(|ua| ua.address.clone()).map(|a| vec![a]).unwrap_or_default()` — loaded from the server's channel database record, not from the incoming message. **TN_PEP_ATTESTED** (code-verified: `context.paired_addresses` is server-side channel config, unreachable by the sender).

(P11) Firma-AI/openfirma `examples/policies/payment.cedar` — A payment gateway sidecar. `crates/firma-sidecar/src/enforcement/constraint_enforcement.rs` implements `build_context()`, which populates `context.risk_score` from `signals.risk_score_long()` and `context.budget_remaining` from `signals.budget_remaining_long(claims.budget_ceiling)` — where `RuntimeSignals` are computed by the trusted sidecar and `claims` are JWT-validated. The Cedar payment policy gates on these sidecar-computed values, not on agent-supplied transfer details directly. V1 payment fields (transfer_amount, velocity counters) are documented as future-task placeholders. **TN_PEP_ATTESTED** for implemented fields (code-verified at `cedar_evaluator.rs` + `constraint_enforcement.rs`).

(P3) awslabs/agentcore-samples `3_time_based.cedar` — Policy file comment (verbatim): *"Uses `context.system.now` — a system-provided timestamp injected by the gateway at request time. The agent cannot manipulate this value."* AWS AgentCore's gateway architecture independently provides the timestamp. **TN_PEP_ATTESTED** (gateway-documented in policy file; same pattern as cedar-examples streaming_service but with explicit PEP attestation commitment).

Sprint 29: Cedar Operator Misuse — Confirmed Production Policy Failures. Scanning the 1,210-file corpus for `context.attr in [string_list]` patterns (612 matches via GitHub Code Search) revealed a second, distinct Cedar schemaless hazard: **Cedar's `in` operator is for entity-hierarchy membership only**. Applied to a String context attribute with a string set literal (`context.command_pattern in ["gh pr review", "git push"]`), the Cedar Rust engine generates a type error — `"type error: expected (entity of type`

`any_entity_type), got string"` — and discards the erroring policy per Cedar's documented error-ignorance semantics. The correct Cedar idiom is `["gh pr review", "git push"].contains(context.command_pattern)` — a set literal containing the element. This is the same root cause as §4.2 (schemaless mode + runtime error discard), triggered by a different operator misuse pattern. Severity depends on policy structure: a policy with a blanket `permit(principal, action, resource)` fallthrough has every erroring forbid silently replaced by PERMIT (fail-open bypass); a policy with only selective permits has the broken rule silently replaced by DENY (fail-closed correctness failure).

(P7-CONFIRMED) wshobson/agents `plugins/review-agent-governance/policies/review-agent-governance.cedar` — COMPLETE BYPASS CONFIRMED (Rust PDP). This policy enforces human-approval requirements for a Claude Code agent: forbids `gh pr review`, `gh pr merge`, `git push` to main/master, writes to `.github/workflows/`, and WebFetch POSTs to `api.github.com`. All 5 forbid rules use `context.command_pattern in [...]` or `context.target_branch in [...]` — Cedar entity-hierarchy operators applied to String values. All 5 generate runtime type errors and are silently discarded. The final line is `permit (principal, action, resource)` — a blanket permit. **Result: EVERY request is PERMITTED regardless of content** (`command_pattern="gh pr review"` → PERMIT; confirmed with cedar-python 0.1.4 Rust PDP). The policy provides zero protection for the security controls it appears to enforce. The broken policy has been deployed byte-identically in five additional public repositories via Claude Code's plugin and marketplace distribution channels, all independently dynamically confirmed PERMIT (`results/sprint45f_enriched_confirm.json`, adversarial input `{command_pattern: "ls -la", role: "user"}`, cedar-python 0.1.4):

REPOSITORY	INSTALL PATH	CHANNEL	DECISION
`bachsh/supermarket`	`plugins/wshobson-review-agent-governance/`	plugin dir	PERMIT
`Jhabbig/Habbig`	`.claude/plugins/wshobson/review-agent-governance/`	plugin dir	PERMIT
`AdmiralGallade/harness-repository`	`harnesses/agents/plugins/review-agent-governance/`	plugin dir	PERMIT
`steph-frtech/claude-code-up`	`.claude/marketplaces/wshobson/plugins/review-agent-governance/`	marketplace	PERMIT
`yo-steven/agents-exploration-20260523`	`plugins/review-agent-governance/`	plugin dir	PERMIT

In every case the policy text is byte-identical to the wshobson source — the adopting users are exposed without any authoring error of their own. **These five adopters are not independent policy origins**: computing a prevalence statistic over five correlated copies of one policy text would be invalid, and we do not report one for this group. The correct reading is that one vulnerable policy family exposed five independent user environments through Claude Code's distribution channels — supply-chain amplification, not independent recurrence. Combined with `kenhuangus/agentctl` (below), the **independent-authorship count for the operator-misuse hazard is N=2 within this original purposive-discovery corpus** (revised to N=3 in §5.6.1 after broadening the search beyond `context.*` attributes); seven repositories are dynamically confirmed PERMIT in total from this corpus alone. No schema validation is enforced at any deployment site (schemaless evaluation); callers checking only `response.allowed` receive PERMIT with `response.errors` populated but unexamined.

(P15) ScopeBlind/agent-governance-testvectors `fixtures/policy/autoresearch-safe.cedar` — CORRECTNESS FAILURE (Rust PDP). Policy uses `context.command_pattern in ["git", "npm", ...]` in a permit rule (safe commands) and `context.command_pattern in ["rm -rf", "dd", ...]` in a forbid rule (destructive commands). Both generate type errors and are discarded. The `action in [Ac-`

`tion::"Read", ...]` permits (entity UIDs, not strings) evaluate correctly. Result: agents cannot run safe Bash commands (DENY where PERMIT expected); destructive commands (rm -rf) are DENY — but coincidentally, because no permit fires, not because the explicit forbid rule activated. Fail-closed failure: security maintained by default-DENY, but legitimate agent operations blocked. Confirmed with Rust PDP: `command_pattern="git"` → DENY (expected PERMIT); `command_pattern="rm -rf"` → DENY (expected DENY but for wrong reason).

(P28-CONFIRMED) kenhuangus/agentctl `agent_policies/developer.cedar` — COMPLETE BYPASS CONFIRMED (Rust PDP, Sprint 33). An independently-authored Azure cloud operations agent governance policy covering a completely different domain from wshobson (cloud infrastructure management vs. GitHub governance). Two forbid rules guard high-risk operations: `context.risk_level in ["high","critical"]` and `context.skill_name in ["azure-kubernetes","azure-storage",...]` — both using Cedar's entity-hierarchy `in` operator applied to String context attributes. Both rules produce the same type error as P7 (`error while evaluating policy 'policy0': type error: expected (entity of type 'any_entity_type'), got string`) and are silently discarded. A trailing conditional permit fires for developer-team members in non-production environments. **Result: high-risk operations in dev/staging are PERMITTED regardless of risk level or skill type.** Empirical confirmation (cedar-python 0.1.4 / cedar-policy 4.8.2, `results/sprint33_kenhuangus_bypass_confirmed.json`): Test 1 — `risk_level="high"`, `skill_name="azure-kubernetes"`, `environment="dev"` → **PERMIT** (BYPASS-CONFIRMED); Test 2 — `risk_level="critical"`, `skill_name="azure-storage"`, `environment="staging"` → **PERMIT** (BYPASS-CONFIRMED). This is the **second independent CEDAR_OPERATOR_MISUSE_BYPASS** in a distinct repo and domain, confirming the pattern is not isolated to a single author or ecosystem. Both failures disclosed to the Cedar project: cedar-policy/cedar#2428 (filed 2026-06-24) and maintainer of P7: wshobson/agents#598.

Sprint 41: Cross-Engine Replication — Cedar `in`-on-String fail-open confirmed in `@cedar-policy/cedar-wasm` v4.11.2.

The Sprint 29 bypass confirmation used `cedar-python` 0.1.4, a PyO3 binding over `cedar-policy` 4.8.2. An alternative explanation is that the fail-open is a binding artifact specific to the PyO3/cedar-python wrapper rather than the core Cedar Rust engine. Sprint 41 rules this out: we reproduce the bypass in **`@cedar-policy/cedar-wasm` v4.11.2** — the official Cedar WebAssembly package maintained by the Cedar team at AWS, compiled from the same Rust codebase via wasm-bindgen (note: `@cedar-policy/cedar-wasm` is the optional dependency of `protect-mcp`, the enforcement layer used by wshobson's Claude Code hook; the actual `protect-mcp@0.5.5 evaluate --policy` CLI enforcement path was not directly testable locally, so `@cedar-policy/cedar-wasm` is tested via its Node.js API as the underlying engine component). The engine under test is independent of cedar-python in its compilation toolchain (wasm-bindgen vs. PyO3) and is a newer release (v4.11.2 > v4.8.2). Using `@cedar-policy/cedar-wasm`'s Node.js `isAuthorized()` API with each of wshobson's four forbid-rule patterns (`context.x in [string_list]`) applied to the corresponding governance scenario: **4/4 governance gates bypassed** — PR merge (Bash), force-push to main (Bash), Write to CI workflow path, POST to api.github.com (WebFetch) — all returning `decision: "allow"` with identical type errors in `diagnostics.errors`; the blanket permit fires in `diagnostics.reason`. **Negative control (real)**: running the same 4 scenarios through the *corrected* policy (`.contains()` idiom: `["gh pr merge", ...].contains(context.command_pattern)`) returns `decision: "deny"` with zero errors — proving the cedar-wasm harness is capable of producing DENY and that the bypass is not a harness artifact. Note: the original `git status` negative control was vacuous — it also returns `decision: "allow"` with one type error because ALL forbid rules discard on type error regardless of the context value; the `.contains()`-corrected negative control is the definitive test. `results/sprint41_protect_mcp_wasm_bypass.json`. **Conclusion**: The Cedar `in`-on-String fail-open exists in the core Rust engine, confirmed by two independent bindings (PyO3 and wasm-bindgen) across two different versions (4.8.2 and 4.11.2). It is not a cedar-python binding artifact.

Sprint 42: Enforcement-Layer Integration Bugs — protect-mcp v0.5.5 Cedar enforcement path is fail-open by three independent mechanisms.

Sprint 41 confirmed the Cedar engine bug in `@cedar-policy/cedar-wasm`. Sprint 42 examines whether the enforcement layer that wraps the engine — `protect-mcp v0.5.5` (ScopeBlind), the tool used by wshobson's Claude Code governance hook — correctly propagates DENY when the engine produces it. Source analysis of `protect-mcp/dist/chunk-YTBC72JJ.mjs` (the `evaluateCedar` and `parseWasmResult` functions) and `dist/chunk-SPHLVRJ2.mjs` (the Claude Code PreToolUse hook handler) reveals **three independent fail-open mechanisms in protect-mcp's own Cedar enforcement path**:

Two distinct enforcement paths exist; both fail open.

Path A — wshobson's hook command (`npx protect-mcp@0.5.5 evaluate --policy <cedar_file>`): The `evaluate` subcommand does not exist in v0.5.5 (confirmed by `--help`). This is a bug in *wshobson's* hook configuration, not in protect-mcp itself. The hook command fails silently; no Cedar evaluation occurs. **Result: FAIL-OPEN (hook command broken, wshobson-authored).**

Path B — protect-mcp's `--cedar <dir>` enforcement mode (ScopeBlind-authored, the MCP server path): Two independent API integration bugs in `dist/chunk-YTBC72JJ.mjs`:

- **(B1) Policies format mismatch.** `evaluateCedar` passes `policies: policySet.source` (raw Cedar DSL string) to `cedarWasm.isAuthorized()`. `@cedar-policy/cedar-wasm` v4.x requires `policies: { staticPolicies: string }` (a `PolicySet` object). Empirical: `cedarWasm.isAuthorized({ policies: "forbid...; permit...;" })` throws `invalid type: string ... expected struct PolicySet`. Catch block returns `{ allowed: true }` — **FAIL-OPEN by exception (ScopeBlind-authored code).**
- **(B2) Response envelope mismatch.** `parseWasmResult` checks `result.type === "allow"/"deny"` and `result.decision === "Allow"/"Deny"`. cedar-wasm v4.x returns `{ type: "success", response: { decision: "allow"|"deny" } }`. Neither check matches; falls through to `return { allowed: true }` — **FAIL-OPEN by default return (ScopeBlind-authored code).**

Bonus (Path B): action namespace mismatch. Even with correct API format and response parsing, protect-mcp uses `action = Action::"MCP::Tool::call"` for all evaluations; wshobson's forbid rules guard `Action::"Bash"`, `Action::"WebFetch"`, `Action::"Write"`. Empirical: `isAuthorized({ action: {type:"Action", id:"MCP::Tool::call"}, policies: {staticPolicies: wshobson_pattern} })` → `decision: allow, errors: []` (no type error — forbid's action condition fails first). **FAIL-OPEN by action mismatch (wshobson policy / protect-mcp schema mismatch).**

Conclusion. protect-mcp v0.5.5's `--cedar` enforcement path is inoperable with its own declared `@cedar-policy/cedar-wasm ^4.9.1` dependency (B1+B2 are ScopeBlind-authored API integration bugs against a newer cedar-wasm API); wshobson's hook uses a non-existent CLI subcommand (Path A, wshobson-authored). This is a **Category 3 finding** (enforcement-layer integration bugs, independent of and compounding the Category 2 Cedar engine operator-misuse bug). Both paths fail open. `results/sprint42_protect_mcp_enforcement_failopen.json`.

Sprint 32: Live End-to-End OWA Exploit — Closed-Loop Running System Demonstration.

To close the gap between Cedar-PDP-confirmed bypass and a running-system exploit, we built a faithful gateway harness (`demo/cedar-gateway-demo/`) replicating the minorun365/agentcore-book chapter 9 no-interceptor deployment pattern. The service (FastAPI + cedar-python 0.1.4) exposes the authorization gateway as a running HTTP service:

```
POST /authorize  {"tool": "process_refund", "cedar_amount": 49999}
     → Cedar evaluates: 49999 < 50000 → PERMIT → session_token returned
```

```
POST /execute    {"session_token": "...", "actual_amount": 100000}
     → Cedar NOT re-evaluated → executes $1000 refund
```

Live exploit output (`python3 exploit_demo.py` against running `uvicorn gateway:app`):

```
SCENARIO 1 (CONTROL): Legitimate $100 refund
  /authorize (amount=10000): Cedar PERMIT
  /execute   (actual=10000): Refunded $100.00  ✓

SCENARIO 2 (CONTROL): Large $1000 refund (direct)
  /authorize (amount=100000): Cedar DENY  ✓

SCENARIO 3 (EXPLOIT):
  /authorize (cedar_amount=49999): Cedar PERMIT → session_token
  /execute   (actual_amount=100000): Refunded 100000 cents ($1000.00)
  BYPASS_CONFIRMED=True
  Cedar authorized 49999 cents but 100000 cents ($1000.00) executed.
  Cedar was NOT re-evaluated at execution time.

AUDIT LOG: Cedar-authorized $599.99 | Actually executed $1100.00
           Excess extracted beyond Cedar authority: $500.01
```

Results: **`results/sprint32_live_exploit_confirmed.json`** . The exploit chain runs end-to-end on a live service: forged Cedar context (49999 cents) → Cedar PERMIT → actual refund (100000 cents) executes → $500.01 extracted beyond Cedar's authorization. Cedar was never re-evaluated for the execution amount.

Design: The two-step `/authorize → /execute` pattern matches the AgentCore "pre-authorization" model. The service implements the exact vulnerability present in chapter 9 (no interceptor, Cedar context built from agent-declared request body). The harness is reproducible: `cd demo/cedar-gateway-demo && uvicorn gateway:app --port 8080` .

Sprint 30: Production OWA Attestation-Gap Bypass — Policy Corpus Evidence.

(P12-CONFIRMED) minorun365/agentcore-book `chapter9/01_permit_amount.cedar` — OWA_BYPASS_CONFIRMED (Rust PDP). This is an AWS AgentCore tutorial policy for a refund-processing agent:

```
permit(
  principal,
  action == AgentCore::Action::"RefundTarget___process_refund",
  resource == AgentCore::Gateway::"arn:aws:bedrock-agentcore:..."
)
when { context.input.amount < 50000 };
```

Cedar's default is DENY. This permit fires only when `context.input.amount < 50000` . AWS AgentCore's architecture populates `context.input` from the tool's input parameters — which the AI agent supplies. An agent wanting to process a refund of 100000 cents ($1000, exceeding the gate) can forge `context.input.amount = 49999` in the Cedar authorization request, obtaining Cedar PERMIT. The agent then calls the actual `process_refund` API with amount=100000. Cedar cannot verify that the authorized value equals the actual API parameter: there is **no provenance field** in Cedar's `Map Attr Value` context type. This is the attestation gap (Security Claim 1) confirmed in a production AWS AgentCore deployment.

Rust PDP verification (`scripts/run_sprint30_owa_attestation_bypass.py` , `results/sprint30_owa_attestation_bypass_confirmed.json`): - Control: `context.input.amount = 100000` → **DENY** (no permit fires) - Bypass: `context.input.amount = 49999` (forged) → **PERMIT** (49999 < 50000 → true) - 5/5 tests correct (T1: DENY, T2: PERMIT, T3: PERMIT legitimate, T4: PERMIT boundary, T5: DENY at 50000)

Scope of claim (Cedar-PDP-confirmed, architecturally inferred for deployed system): The Rust PDP confirms Cedar issues PERMIT for forged amount. The deployed-system exploitability is *architecturally inferred* from the absence of a custom interceptor (chapter9 README + chapter8 interceptor structure), not demonstrated in a running AgentCore deployment. A live end-to-end demonstration (forged context → Cedar PERMIT → `process_refund(100000)` executing on actual infrastructure) would confirm the full exploit chain.

Contrast with Sprint 29 (wshobson) operator-misuse bypass: wshobson is *broken-by-construction* — no forbid rule fires due to type errors; the policy is remediable with `.contains()`. P12-CONFIRMED is a *correctly-written policy with a structurally bypassable gate* — Cedar's `Map Attr Value` context type cannot prove provenance, so the policy correctly implements the developer's intent but the intent itself is OWA-vulnerable (Security Claim 1 constructive example in production).

OWA ground truth (source-code verified, Sprint 30→31): `context.input.amount` in chapter 9 is the raw tool parameter supplied by the AI agent. The `chapter9/README.md` states the Cedar policies are applied via the AWS management console policy engine — **there is no custom interceptor Lambda for chapter 9.** Chapter 8 (`chapter8/03_interceptor.py`) shows what an interceptor looks like when used: it checks SQL injection in tool arguments but does NOT inject or overwrite any Cedar context values. The contrast with the awslabs deployment (Sprint 31, see below) is direct: awslabs injects geography from JWT using a custom interceptor; chapter 9 has no interceptor, leaving context.input as the raw agent-supplied tool parameters. The Sprint 30 bypass is therefore confirmed by architecture: agent sets `context.input.amount = 49999`, Cedar permits, agent calls process_refund with a larger amount — Cedar's `Map Attr Value` context has no provenance guarantee when no interceptor overrides it.

Sprint 31: Official AWS Labs Production Deployment — PEP Attestation Analysis (awslabs/agentcore-samples).

(P16) awslabs/agentcore-samples `lakehouse-agent` — PEP_ATTESTED + POLICY_COVERAGE_GAP (Sprint 31). This is the official AWS Labs repository for AWS AgentCore deployment samples (`awslabs` GitHub org, Apache-2.0) with actual CDK deployment infrastructure. Its Cedar policies run inside an AWS AgentCore "advanced policy gateway interceptor" Lambda (`lambda/interceptor-request/lambda_function.py`). Reading the interceptor source code reveals both the correct OWA mitigation pattern and a residual coverage gap.

```
// forbid_eu_individual_claims.cedar (awslabs/agentcore-samples — exact text)
forbid(principal, action in [AgentCore::Action::"lakehouse-mcp-target___query_claims",
                             AgentCore::Action::"lakehouse-mcp-target___get_claim_details"],
       resource == AgentCore::Gateway::"{gateway_arn}")
when { context.input has geography && context.input.geography == "EU" };
permit(principal, action, resource == AgentCore::Gateway::"{gateway_arn}");  // permit_all.cedar
```

Interceptor analysis (lambda_function.py, verbatim): The interceptor Lambda validates the user's Cognito JWT, extracts the user email, and *injects* geography from a server-side lookup table before the Cedar decision:

```
# ---- Design 3: Inject geography into tool arguments ----
# Cedar Policy evaluates context.input.geography for geography-based access control.
USER_GEOGRAPHY: Dict[str, str] = {
    "policyholder001@example.com": "US",
    "policyholder002@example.com": "EU",
    ...
}
geography = USER_GEOGRAPHY.get(user_principal, "UNKNOWN")
if "params" in transformed_body and "arguments" in transformed_body["params"]:
    transformed_body["params"]["arguments"]["geography"] = geography
```

The interceptor **always overwrites** any geography the agent supplies with the JWT-derived value. `context.input.geography` in Cedar is not agent-controllable in this deployment — the PEP (interceptor Lambda)

attests to it from a trusted JWT. This is a correctly-designed OWA mitigation: Cedar's context is populated by the PEP, not by the agent.

OWA bypass vectors (Cedar policy in isolation, Rust PDP, `scripts/run_sprint31_awslabs_production_bypass.py`): - T2 (geography forgery `EU→US`): Cedar PERMIT — but the deployed interceptor PREVENTS this; an EU-JWT user always gets geography="EU" injected. - T3 (geography omission): Cedar PERMIT — but the interceptor always injects geography; the `has` guard never evaluates to FALSE in the deployed system. - T4 (text_to_sql coverage gap): Cedar PERMIT — **this survives even with the interceptor.** An EU user (geography="EU" injected from JWT) calling `text_to_sql("SELECT * FROM eu_individual_claims")` gets Cedar PERMIT because `text_to_sql` is NOT in the `forbid_eu_individual_claims` action list. The EU restriction covers `query_claims` and `get_claim_details`, but SQL pass-through access is uncontrolled.

Key finding: The awslabs deployment provides a **correct reference implementation** for preventing OWA geography forgery (JWT → PEP-injected context, T2/T3 mitigated). But the Cedar policy has an **action coverage gap** (T4): EU individual claims data is accessible via `text_to_sql` despite being blocked via `query_claims`. The policy author enforced EU access control on named tool actions but overlooked the general-purpose SQL endpoint. Cedar policies must cover ALL data-access paths, not just named high-risk actions.

Classification: P16 = TN (OWA forgery/omission correctly mitigated by PEP attestation) + POLICY_COVERAGE_GAP (text_to_sql not in EU forbid scope). This is an example of correct OWA defense with incomplete Cedar policy coverage.

Key architectural contrast (Sprint 30 vs Sprint 31): The two AgentCore deployments in the corpus represent opposite ends of the PEP-attestation spectrum:

	MINORUN365/AGENTCORE-BOOK CH9 (SPRINT 30)	AWSLABS/AGENTCORE-SAMPLES (SPRINT 31)
Cedar policy	`context.input.amount < 50000`	`context.input.geography == "EU"`
Interceptor Lambda	**None** (management console paste-in)	**Custom Lambda** (JWT → geography injection)
OWA status	**OWA-forgeable** (raw agent parameters)	**PEP-attested** (interceptor overwrites)
Bypass confirmed	**YES** (Rust PDP + architecture verified)	**NO** for geography (T4 coverage gap only)

This contrast is the paper's central empirical finding: Cedar's authorization model provides no provenance guarantee for context.input values **when no interceptor attests to them**. The awslabs deployment demonstrates the correct mitigation pattern (interceptor-side JWT attestation), but requires developer diligence not enforced by Cedar itself. Most deployments following the "paste Cedar policy into management console" pattern (like chapter 9) get no PEP attestation by default.

OWA ground truth: `awslabs` is the official AWS organization on GitHub; deployment uses real Cognito JWT and Lambda. Context.input.geography is PEP-attested (overwritten by interceptor from JWT), not agent-controlled. In contrast, `minorun365/agentcore-book` chapter 9 explicitly has no interceptor, leaving context.input as direct agent-supplied values.

Revised OWA-Complex Production Cases (source code not verified or provenance ambiguous):

ID	POLICY	ORGANIZATION	CONTEXT GATE	OWA RATIONALE
P1	ifc.cedar	microsoft/agent-governance-toolkit	`context.input.ifc.source_labels` (Set)	Labels from intervention-poin them to Cedar, but whether la assigned or influenced by flow from available source

ID	POLICY	ORGANIZATION	CONTEXT GATE	OWA RATIONALE
P2	1_identity_based.cedar	awslabs/ agentcore-samples	`context.input.patient_id`	Tool input parameter supplied checks against `principal.getTag("pa` TOCTOU risk if Cedar check not strictly coupled
P10	spending-authority.cedar	scopeblind	`context.approval_method`, `context.budget_utilization`	Template policy (no `approv` gateway codebase). If `appro` `"human_confirmed"` is c transaction human-approval g `spending-002`) is fully by
P12-CONFIRMED	**01_permit_amount.cedar**	**minorun365/ agentcore-book**	**`context.input.amount`**	**OWA_BYPASS_CONFIRM PDP).** Tool input parameter (AgentCore). Cedar gate: `context.input.amount` amount=100000 → DENY. B `context.input.amount` PERMIT (49999 < 50000). A `process_refund(amou` API. Cedar cannot verify cont API parameter — the attestati `run_sprint30_owa_att` See below.
P13	triage_routing.cedar	ThirdKeyAI/ Symbiont	`context.ticket_severity`	Policy comment acknowledges (claimed) severity is present in `context.ticket_sever` worker-claimed severity rather ticket-system lookup, any worl gates by self-reporting `"crit`
P14	personal-dm-scope.cedar	jason931225/ oyatie†	`context.now`	Request-context timestamp; t `(context.now - resou` `86400` is bypassable by backd the same value as `resource` editing of arbitrarily old messa

† `jason931225/oyatie` is the same repository §5.6.1 discusses at length in the H1 context: a single-author, AI-agent-generated, actively-WIP monorepo, not a confirmed operating production system. P14's OWA-complexity is a real property of the code as written, but "Production Cases" should be read for this row as "code that would exhibit this behavior if deployed," not as evidence of an actual deployed system with real users.

True Negatives (no OWA-forgeable security gate):

- **(P6) RelayOne/r1-agent `send-on-behalf-of.cedar`** : all security gates on entity store attributes (`principal.send_allowlist` , `resource.recipient`) — **TN_ENTITY_STORE**
- **(P8, P9) scopeblind `clinejection.cedar` , `terraform-destroy.cedar`** : forbid rules match on action/ resource type only, no `context.*` references — **TN_NO_CONTEXT**

Combined Corpus (cedar-examples + Production, including Sprint 29):

CORPUS	N	CONFIRMED BYPASS	MECHANISM	OWA_COMPLEX	TN/CF
	7	2 (29%)	OWA forgery	2	3

CORPUS	N	CONFIRMED BYPASS	MECHANISM	OWA_COMPLEX	TN/CF
cedar-examples (Sprint 26)					
Production/ ecosystem (Sprint 27)	14	0	—	6	8
Cedar operator misuse (Sprint 29)	+2 (+4 clones)	+1 bypass	CEDAR_OPERATOR_MISUSE	—	+1 CF
OWA attestation gap (Sprint 30)	0 new (reclassified)	+1 OWA forgery	OWA forgery (Security Claim 1)	−1	—
awslabs production (Sprint 31)	+1	0 (PEP attests)	TN + coverage gap	0	+1 TN/ CF
kenhuangus/ agentctl Azure (Sprint 33)	+1	+1 bypass	CEDAR_OPERATOR_MISUSE (independent domain)	0	0
Prevalence audit top-30 (Sprint 40)	+5 new	0 new bypass	—	0	+1 CF (aws-samples)
cedar-wasm cross-engine replication (Sprint 41)	0 new	0 new (P7 re-confirmed in cedar-wasm)	Cross-engine: wasm-bindgen confirms PyO3 result	0	0
protect-mcp enforcement-layer bugs (Sprint 42)	0 new	0 new	Category 3: enforcement-layer fail-open (3 mechanisms)	0	0
Combined (unique policies)	**30**	**5 (17%)**	3 OWA + 2 op-misuse	**6 (20%)**	**19 (63%)**

Sprint 29 reclassified P7 (wshobson/agents) from OWA_COMPLEX to **CONFIRMED_BYPASS** (CEDAR_OPERATOR_MISUSE). Sprint 30 reclassified P12 (minorun365 permit_amount) from OWA_COMPLEX to **OWA_BYPASS_CONFIRMED** (attestation-gap forgery, Security Claim 1 in production AgentCore tutorial). Sprint 31 adds **P16 (awslabs/agentcore-samples)** as a **TN + POLICY_COVERAGE_GAP**: the interceptor Lambda correctly attests geography from JWT (OWA forgery/omission prevented), but `text_to_sql` is not in the EU forbid scope — EU data accessible via SQL bypass. P15 (ScopeBlind) is added as **CORRECTNESS_FAILURE_FAIL_CLOSED**. Sprint 33 adds **P28 (kenhuangus/agentctl)** as the **second independent CONFIRMED_BYPASS** (CEDAR_OPERATOR_MISUSE, Azure cloud ops domain, Rust PDP confirmed 2/2, `results/sprint33_kenhuangus_bypass_confirmed.json`). The two CEDAR_OPERATOR_MISUSE bypasses are in completely independent repos, authored independently, serving different cloud platforms (GitHub governance vs. Azure cloud ops) — confirming the hazard class is not author-specific. Sprint 41 re-confirms P7 (wshobson/agents) in **`@cedar-policy/cedar-wasm` v4.11.2** (official Cedar WASM package, wasm-bindgen build, AWS/Cedar team) — a second independent binding of the same Rust engine, newer version (4.11.2 > 4.8.2) than cedar-python — ruling out a PyO3 binding artifact: 4/4 governance gates bypassed, `decision: "allow"` with type errors in `diagnostics.errors`; corrected `.contains()` policy gives `deny` with zero errors (`results/sprint41_protect_mcp_wasm_bypass.json`). Sprint 42

(source analysis + empirical) reveals protect-mcp v0.5.5's Cedar enforcement path has **three independent fail-open mechanisms** — API format mismatch (raw string vs. `{staticPolicies}` object), response envelope mismatch (`{type: "success", response: {decision}}` not handled → default `allowed: true`), and action namespace mismatch (wshobson forbid rules target `Action::"Bash"` but protect-mcp sends `Action::"MCP::Tool::call"`) — making the enforcement layer completely inoperable with its own cedar-wasm dependency; plus the wshobson hook command (`evaluate` subcommand) doesn't exist in v0.5.5 (`results/sprint42_protect_mcp_enforcement_failopen.json`). Upstream disclosure: cedar-policy/cedar#2428; wshobson maintainer notification: wshobson/agents#598 (both filed 2026-06-24).

The awslabs finding is the **positive reference case**: it shows what correct OWA mitigation looks like (PEP-attested context from JWT, interceptor overwrites any agent-supplied geography). The paper's dual-loop hardening would surface the T4 coverage gap during synthesis, since `text_to_sql` returning EU data would fail the hardened forbid rule.

Four distinct bypass/failure categories in confirmed cases: 1. **OWA_BYPASS (attestation-gap forgery)**: attacker forges or omits a caller-supplied context attribute to satisfy a correctly-written but OWA-vulnerable gate. (3 cases: document_cloud, tax_preparer [cedar-examples]; minorun365 permit_amount [Sprint 30 tutorial]) 2. **CEDAR_OPERATOR_MISUSE_BYPASS** (production): developer uses wrong Cedar operator (`in` instead of `.contains()`) causing all forbid rules to silently fail via type error + error-ignorance. Blanket permit fallthrough makes policy fail-open. (**2 independent confirmed cases**: wshobson/agents, 4 downstream deployments [Sprint 29]; kenhuangus/agentctl, Azure cloud ops, independent domain [Sprint 33]. Sprint 41 re-confirms P7 in a second independent Cedar binding — `@cedar-policy/cedar-wasm` v4.11.2 (official AWS/Cedar team wasm-bindgen WASM build, newer than cedar-python 4.8.2), ruling out a PyO3 artifact: 4/4 governance gates bypassed; corrected `.contains()` control gives DENY. Upstream disclosure: cedar-policy/cedar#2428; maintainer: wshobson/agents#598.) 3. **CEDAR_OPERATOR_MISUSE_CORRECTNESS** (production): same operator misuse but without blanket permit → fail-closed incorrectness (2 cases: ScopeBlind; `aws-samples/sample-agent-jailbreak-to-cloud-takeover` defense-artifacts/cedar-policies/agent-isolation.cedar [Sprint 40 — ironically, a defense artifact specifically targeting cloud takeover via agent jailbreak has the bug in its emergency override mechanism: `context.OverrideAuthorizer in ["arn:aws:iam::*:role/IncidentCommanderRole"]` errors silently, permanently disabling the emergency override without any indication]) 4. **ENFORCEMENT_LAYER_INTEGRATION_FAIL_OPEN** (enforcement tool, not policy): protect-mcp v0.5.5's Cedar enforcement path (`--cedar <dir>` mode) is completely inoperable with its own cedar-wasm dependency: (a) API format mismatch — passes `policies: raw_string` to cedar-wasm v4.x which expects `{staticPolicies: string}` → exception → catch → `allowed: true`; (b) response envelope mismatch — `parseWasmResult` checks `result.type` for "allow"/"deny" but cedar-wasm v4.x returns `{type: "success", response: {decision: "allow"/"deny"}}` → falls through to `allowed: true`; (c) action namespace mismatch — protect-mcp uses `Action::"MCP::Tool::call"` while wshobson's forbid rules target `Action::"Bash"` etc. → forbid rules never fire. (1 case: ScopeBlind/protect-mcp. **Sprint 42**, `results/sprint42_protect_mcp_enforcement_failopen.json`. Zero Cedar enforcement from the enforcement layer itself, regardless of policy.)

The four code-verified TN cases (sondera-ai, fronalabs, Firma-AI, AWS AgentCore) demonstrate that organizations deploying Cedar for high-stakes decisions do correctly build Cedar context from trusted PEP infrastructure when they invest in the architecture. The six OWA_COMPLEX cases span financial authorization, healthcare record access, agent governance, time-gated messaging, and escalation routing.

Sprint 29 methodology and prevalence. The Cedar `in` operator misuse pattern was discovered by testing the wshobson production policy against the Rust PDP and finding that the control case (`command_pattern="gh pr review"`) returned PERMIT instead of DENY. Root cause analysis identified `in` with string values as the trigger. GitHub Code Search for `context.+in+[` returned 612 hits across the corpus. A narrower query — Cedar files containing both `context.` + `in [` (string-literal set) and a `permit` statement — yields **469 unique Cedar files**

on GitHub (2026-06-23 snapshot). Of the top 30 by relevance (ranked by GitHub's relevance, not random): **2 confirmed FAIL_OPEN_BYPASS** (wshobson/agents, kenhuangus/agentctl); **2 FAIL_CLOSED_CF** (ScopeBlind/agent-governance-testvectors; aws-samples/sample-agent-jailbreak-to-cloud-takeover — described below); **26 TN/ENTITY_UID_ONLY** (safe, either using entity-UID `in` lists correctly or equality checks). We report the count directly: **2 fail-open bypasses and 2 correctness-failure instances among 30 manually audited, relevance-ranked candidates drawn from a 469-file keyword corpus.** We do not infer a population prevalence from this non-random head; the relevant observation is that both confirmed bypasses are in security-critical governance policies distributed as reusable plugins or infrastructure templates — not in toy examples — and that the third instance (aws-samples, below) appears in a defense artifact. The finding generalizes the §4.2 schemaless hazard to a broader class: **any Cedar type-erroneous operator in a forbid rule is silently discarded per spec, with security impact determined by whether a permit fallthrough exists**. The relevance-ranked top-30 audit above is not a random sample; §5.6.1 below reports the fully randomized audit of the corpus that this paragraph originally flagged as future work.

Methodological advance (Sprint 27+29). TN labels are now mechanically derived from request-construction code; production bypasses are confirmed against the Rust PDP, not from README documentation or manual analysis. OWA labels and operator-misuse findings are independently verifiable via the Rust PDP with the test inputs in `results/sprint29_in_operator_bypass.json`.

5.6.1 Randomized Prevalence Audits: File-Level, Then Repo-Stratified

The relevance-ranked audit above (top 30 of 469 files) is not a random sample and cannot support a population-prevalence claim. We ran two successive randomized audits to close this gap, in the order we actually ran them, because the first revealed a design flaw that motivated the second.

File-level random sample (Sprint 43). From the same corpus query (`extension:cedar "context." " in ["`, 612 raw hits, 595 unique files after dedup, excluding the 2 purposive-discovery repos), we drew a uniform random sample of 200 files (seed = 42, `scripts/run_sprint43_prevalence_audit.py`, `results/sprint43_prevalence_audit.json`), yielding 199 auditable files. Result: **87/199 files (43.7%, Wilson 95% CI 37.0%–50.7%) show H1 misuse; 50/199 (25.1%, CI 19.6%–31.6%) are additionally fail-open.** This is the source corpus for the validator catch-rate result in §5.10 (158 patterns extracted from these 87 misuse files). **This file-level sample has a known flaw we do not paper over**: because sampling is uniform over *files*, a repository with disproportionately many matching files is disproportionately likely to dominate the sample — one repository (`jason931225/oyatie`) alone accounts for 73% of the 200-file sample, so the 43.7% figure is closer to a within-repo estimate for a small number of large repositories than a population estimate across independent Cedar deployments. That dominant repository is itself a caveat, not just a statistical one: `oyatie`'s own README describes it as a proprietary, actively-WIP, single-author platform where "AI agents are the primary producers" of the codebase, simulating dozens of unrelated enterprise product verticals (EMR, ITSM, marketing automation, payments, and more) under one roof — not a verified operating production system (§5.6.1 discusses this repository's classification in more detail below).

Repo-stratified random sample (Sprint 43b). To fix the dominance problem, we re-ran the audit with one random file drawn per unique repository rather than per file. **Corpus construction:** the GitHub Code Search API query `extension:cedar "context." " in ["` returns 612 `.cedar` files (the same raw hit count as the relevance-ranked query above, before the additional `permit`-statement narrowing used there). We de-duplicate to 79 unique repositories, exclude the 2 purposive-discovery repositories (`wshobson/agents`, `kenhuangus/agentctl`), and draw one random file per remaining repository (seed = 42, `scripts/run_sprint43b_repo_stratified.py`, `results/sprint43b_repo_stratified.json`), yielding $n = 78$ auditable repositories — a corpus construction independent of, and stratified differently from, the file-level 469-file query above.

Classification oracle. A repository is `OPERATOR_MISUSE` if any sampled file contains `context.ATTR in [string, ...]` (H1, §4.2). **This audit's corpus construction and oracle target H1 only** — the query and regex both key on the `in [` idiom; §5.6.2 below reports the analogous audit for H2 (`.contains()`-on-String) with a materially different, and materially more honest, outcome. A repository is additionally `FAIL_OPEN` if it also contains a blanket `permit` rule with no structural restriction on the action or principal scope.

OUTCOME	COUNT	CONDITIONAL FREQUENCY	95% CI (WILSON)
H1 operator misuse (`in`-on-String)	14/78	17.9%	11.0%–27.9%†
Fail-open (misuse + blanket permit)	7/78	9.0%	n/a — correlated copies, see below

†*The 14/78 operator-misuse CI is computed over per-repo syntactic pattern detection; the CI is best read as a conditional frequency within the queried corpus, not independent prevalence, for reasons detailed below — several of the 14 are correlated or non-production. No Wilson CI is reported for the fail-open category (7/78) for the same reason.* The corpus query itself selects files already using `context.` with `in [`, so 17.9% is a **conditional frequency within the queried corpus**, not a prevalence estimate over all Cedar deployments generally.

What the 14 misuse hits actually are, checked file-by-file (not just by regex). Applying the same scrutiny to H1 that §5.6.2 applies to H2 — read every flagged file, not just trust the regex count — changes the picture materially. We categorize all 14:

- **3 are the wshobson/agents governance policy itself**, the file this random draw happened to sample for three of the known adopters (`yo-steven/agents-exploration-20260523`, `AdmiralGallade/harness-repository`, `Jhabbig/Habbig`) — genuine deployments, dynamically confirmed elsewhere (§5.6).
- **4 more are `protect-mcp`'s own bundled test file** (`plugins/*/protect-mcp/test/fixtures/test-policy.cedar`, present because it ships alongside the wshobson plugin bundle), which **the file's own header comment self-labels**: *"Test Cedar policy for protect-mcp hook round-trip tests. Not a production example."* Two of the four (`bachsh/supermarket`, `steph-frtech/claude-code-up`) are still genuine adopters at the *repository* level — they separately carry the real vulnerable `review-agent-governance.cedar` (confirmed in §5.6) — the random draw simply happened to sample this test file instead. The other two (`ArogyaReddy/https-github.com-wshobson-agents`, `meetsiddhu/wshobson-agents`) are **not** — a targeted search found no `review-agent-governance.cedar` in either repo, both repos pushed their full mirrored history in a single burst (46s and 82s respectively between repo creation and last push, ~357 commits imported wholesale from upstream, no subsequent independent activity), and the only H1-matching file either carries is this self-declared test fixture. We do not count these two as production misuse instances.
- **3 are `cedar-policy-core`'s own official parser test fixture** (`src/parser/testfiles/policies.cedar`, containing the identical `"SeniorModerator"` / `"JuniorModerator"` pattern in all three) — none is a Cedar policy author. Two reached it by dependency vendoring, not authorship: `lwx270901/registryintern` and `t3hw00t/ARW` each depend on a crate that transitively depends on `cedar-policy-core`, and the file sits in their `Cargo` /registry cache directory (`.cargo/registry/.../cedar-policy-core-2.5.0/...` and `.cargo-codex/registry/.../cedar-policy-core-3.4.1/...` respectively). The third, `DuskSystems/tree-sitter-cedar`, is a tree-sitter grammar repo (no `Cargo.lock` — it is not a Rust/Cargo project at all) that copied Cedar's sample testfiles directly into its own `cedar/samples/testfiles/` directory as parser test corpus, rather than vendoring via a dependency. By the identical standard §5.6.1's own Sprint 46 paragraph below already applies to exclude `cedar-policy/cedar` itself as "the language's own reference repository... not a deployment," all three are the same false positive by origin, just reached through two different vendoring mechanisms rather than forking the language repo directly.
- **2 are ScopeBlind's own explicitly-labeled test artifacts**: `agent-governance-testvectors/fixtures/policy/autoresearch-safe.cedar` (file header: *"test-vector policy for agent-governance-testvectors"*) and

`examples/interop/composition-test/cedar-policy.cedar` (path itself says "composition-test"). Both are deliberately authored by a real organization (ScopeBlind, independently confirmed elsewhere in this paper for a genuine production hazard, §5.6, §9) rather than accidentally vendored, so we report them separately from the vendored-fixture group rather than folding them in, but they are self-identified as test/demo artifacts, not deployed governance policies.

- **1 is a single correlated authorship source, not an independent production instance**: `jason931225/oyatie/cloud/cloud-k8s/cedar/policies.cedar`. We initially counted this as a genuine independent production instance because the file itself carries no test/example/vendor marker, but that is the wrong test — it is the same test that would have wrongly counted 5 wshobson-family copies as 5 independent authors before we applied the correlated-copies logic in §5.6 (N=7 confirmed, independent-authorship N=2). `oyatie` fails the *independent-authorship* test on the same grounds: a GitHub Code Search restricted to this one repository returns **650** matching `.cedar` files, near-identical in structure (same ADR-header boilerplate, same permit/forbid shape, differing mainly by a microservice name — `cloud-k8s`, `emr`, `itsm`, `marketing-automation`, `payments`, ...) — one authorship/code-generation process replicated across many files, not many independently-authored policies. The repository's own README states why: *"AI agents are the primary producers [of this codebase]... quality is enforced and auto-remediated so that sub-standard output cannot enter the canonical tree."* It is a single-author, actively-WIP (pushed the same day we checked), public-but-proprietary-licensed monorepo simulating an implausibly broad span of unrelated enterprise product verticals — EMR, ITSM, marketing automation, payments, whiteboard collaboration, emergency services, and more — each with its own near-duplicate Cedar policy, governed by an internal ADR/agent-operating-contract system (`AGENTS.md`, `specs/masterplan.json`). By the same de-duplication standard already applied to the wshobson case, `oyatie` contributes **at most one independent-authorship data point**, and that one point is a self-described agent-generated simulation, not a confirmed shipped product with real users — a materially weaker instance than `wshobson/agents` (37k★, real Claude Code users) or `kenhuangus/agentctl` (real Azure tooling). We therefore do not count it toward production-plausible prevalence, on authorship-correlation grounds rather than a new ad hoc criterion.
- **1 is the already-known defense artifact**: `aws-samples/sample-agent-jailbreak-to-cloud-takeover`.

Corrected reading. We report a range rather than a single point estimate, because pinning one number invites exactly the false precision this whole exercise is arguing against. Applying one pre-registered rule — exclude non-deployment artifacts (vendored/mirrored copies of someone else's test fixture: 5 of 14) and exclude the one correlated-authorship non-instance (`oyatie`, 1 of 14, on the same de-duplication grounds already applied to the wshobson case): **production-plausible H1 misuse is 6–8 of 78 (7.7%–10.3%)**, against the raw 17.9% regex-match rate — 8/78 (10.3%, Wilson 95% CI 5.3%–19.0%) if ScopeBlind's two self-labeled test/example files are included as production-adjacent, 6/78 (7.7%, CI 3.6%–15.8%) if they are not. We do not attempt to narrow this further within this paper: the corpus query is inherently biased toward finding the same handful of source artifacts (Cedar's own test suite, one plugin bundle's test fixture, one large agent-generated monorepo) rather than a representative cross-section of independent Cedar deployments, and **"manual review... found no false positives" is not a claim we can make for this sample** — at least 5 of 14 (36%) are non-deployment false positives, and a 6th (`oyatie`) is a correlated-authorship non-instance, even though every regex match is a textually correct instance of the `in`-on-String pattern. This matters for the paper's other H1 evidence too: `oyatie` is also the repository that dominates the Sprint 43 file-level sample (73% of files, §5.6.1 above) and the Sprint 49 validator catch-rate (88% of patterns, §5.10) — both of those figures should be read with the same caveat, not as evidence spread across many independent production codebases. Recomputing the fail-open row on the same basis: of the 7 fail-open-flagged repos, 3 are the vendored `cedar-policy-core` fixture (not the "1 test-fixture copy" a less careful pass might report) and only 3 (`yo-steven`, `AdmiralGallade`, `Jhabbig`) plus the 1 defense artifact (`aws-samples`) are genuine, independent, production-plausible fail-open instances — **4/78 (5.1%)**, not 7/78.

Broadening the search beyond `context.*` (Sprint 51): a third independently-authored, dynamically-confirmed bypass. The N=2 independent-authorship count above was for confirmed *bypasses* (a `forbid` discarded with a blanket-`permit` fallback), found via a query requiring both `context.` and `in [` as separate substrings. The same String-vs-entity type confusion is not specific to `context.*` attributes — `principal.X`, `resource.X`, or a bare `action` compared against string literals via `in` produce the identical type error. Removing the `context.`-only restriction (`extension:cedar " in [\""`) and manually classifying every new candidate not already in the known set (`results/sprint51_broadened_authorship_search.json`) finds the operator-confusion pattern in **at least seven additional, genuinely independent repositories**: `aws-samples/sample-ATX-custom-NKI-Agent` (a different AWS Samples repository from the already-known defense artifact), `grumpystrongman/OpenAegis`, `moabu/opensearch-cedarling-plugin`, `pkonnoth/DocuEasy`, `shaheerkj/nhi-sentinel`, `shoaibakthar/observeid-platform`, and `rohansx/cloakpipe-live`. **A first classification pass on this list was wrong and we correct it here rather than silently fix it**: we initially read only the first cited occurrence per repo and reported all but one as harmless `permit`-clause correctness failures. Re-reading every occurrence in each file found that `shoaibakthar/observeid-platform`'s cited patterns are in **`forbid` clauses with a plausible fallback permit** — the bypass direction, not the correctness-failure direction we first reported — and that `pkonnoth/DocuEasy` and `shaheerkj/nhi-sentinel` also each contain a `forbid` instance the first pass missed.

We dynamically confirmed the `observeid-platform` case against cedar-python 0.1.4: a `Contractor` principal requesting `ReadDocument` on a `"pii"`-classified resource in their own department — which a separation-of-duty `forbid(principal in Role::"Contractor", ...) when { resource.data_classification in ["pii","phi","pci"] }` is meant to block — instead returns `Decision=Allow` (`error while evaluating policy policy2: type error: expected (entity of type any_entity_type), got string`), because the SoD `forbid` is discarded and an unrelated department-match `permit` fires. This is structurally identical to the wshobson/kenhuangus bypass and is, to our knowledge, a **third independently-authored, dynamically-confirmed bypass instance**, from a repository we otherwise have no evidence about (0 stars, no description, no license file — counted toward independent *authorship*, not toward production prevalence) — with one caveat neither wshobson nor kenhuangus has: `observeid-platform` ships `policies/identity.cedarschema`, explicitly typing the relevant attributes as `String`. Applying §5.10's minimal-schema validator method (declare the attribute's real type, run the actual matched forbid text through `Schema.validate_policyset`) confirms the validator flags it, consistent with §5.10's 158/158 result. The shipped `identity.cedarschema` itself additionally fails to even load in cedar-python 0.1.4 as written (an invalid `Boolean` primitive — Cedar requires `Bool` — and an undefined `Document` entity reference), so any CI job that actually tried to run `cedar validate` against it would fail regardless of the H1 bug specifically. The schema file's header comment claims "All policies are validated against this schema at CI time," but the repository has no `.github` directory at all and zero hits for the literal string `cedar validate` anywhere — the same "claimed but not actually wired into CI" pattern §5.6.1's Sprint 46 already documents for other repositories, not a new phenomenon. We report the independent-authorship count for confirmed bypass as **N=3** (wshobson/agents, kenhuangus/agentctl, observeid-platform), with `observeid-platform` flagged as the one instance where a schema exists that *could* close the gap in principle, in a repository that shows no sign of it actually running.

`aws-samples/sample-ATX-custom-NKI-Agent`, `grumpystrongman/OpenAegis`, and `moabu/opensearch-cedarling-plugin` remain confirmed or plausible **correctness failures** (fail-closed: a legitimate grant silently denied), not bypasses — we dynamically confirmed `aws-samples/sample-ATX-custom-NKI-Agent` the same way (a legitimate Bedrock-egress request returns `Decision=Deny` with the identical error signature). `pkonnoth/DocuEasy`'s additional `forbid` instance (`forbid(..., action in [PrescribeMedication, UpdateMedication], resource) when { resource has is_controlled && resource.is_controlled == true && !(principal.specialty in ["pain_management","psychiatry","anesthesiology","oncology"]) }`, meant to gate controlled-substance prescribing

to relevant specialists) has a resolvable fallback: Policy 3 in the same file (`permit(..., action in [PrescribeMedication, ViewMedication], resource is Patient) when { principal.role == "provider" && ... }`) grants any assigned provider prescribing rights with no specialty check at all. **We could not dynamically confirm this one as shipped**: the file's action references (`PrescribeMedication`, `ViewMedication`, bare, unqualified) do not parse as valid Cedar — Cedar requires `Action::"PrescribeMedication"` — so the file as committed is not directly runnable against a real Cedar engine, and the repository has no `.cedarschema` to clarify the intended syntax. Reconstructing the obvious minimal fix (qualifying each bare action name with `Action::`) and dynamically running the reconstructed policy against cedar-python 0.1.4 reproduces the identical bypass: a general-practice (non-specialist) provider prescribing a controlled substance to their own assigned patient returns `Decision=Allow` where the policy's stated intent is DENY. We report this as evidence that the bypass *structure* reproduces under the natural repair, not as a fourth confirmed independent bypass — we cannot confirm this exact file, or any equivalent of it, has ever been evaluated by a real Cedar engine as committed. `shaheerkj/nhi-sentinel`'s `forbid` instance has no distinct fallback permit, so its net direction is still fail-closed. `rohansx/cloakpipe-live` (a `forbid ... unless` allowlist gate, structurally the bypass direction) is not counted either way: the file's own header states a full Cedar evaluator is "out of scope for the demo" and a hand-written TypeScript mirror is the actual runtime enforcement, so even a confirmed type error here would not manifest in the deployed system. **The honest summary of this sprint**: independent authorship of the underlying pattern is broader than previously reported — at least 9 distinct organizations — and independent authorship of a confirmed bypass specifically grew from N=2 to **N=3**, with one further unconfirmed bypass candidate (`DocuEasy`) left open rather than claimed. We flag this sprint's own initial misclassification (§ above) as a concrete instance of exactly the discipline this paper argues for elsewhere: a syntactic match is not a severity classification, and both require independent re-verification before publication, not just before the first draft.

Schema/CI enforcement stratification (Sprint 46). Declaring a schema and running `cedar validate` in CI is Cedar's own recommended structural fix for this hazard class (§4.2, §5.9). Sprint 46 checks this for a related but **distinct** set — **the 10 unique repositories among the 87 misuse files in the Sprint 43 file-level sample above** (not the Sprint 43b repo-stratified 14 just discussed; the two sets overlap substantially — `AdmiralGallade`, `Jhabbig`, `bachsh`, `jason931225/oyatie`, `lwx270901`, `steph-frtech`, `ArogyaReddy`, `meetsiddhu` appear in both — but Sprint 46's set also includes `cedar-policy/cedar` and `microsoft/agent-governance-toolkit`, which Sprint 43b's repo-stratified draw did not happen to sample). We checked whether the structural fix is actually deployed — not merely mentioned. An unrestricted GitHub code search for `.cedarschema` files and for the literal string `cedar validate` anywhere in each repository initially suggested partial coverage, but manual inspection of every hit found it dominated by false positives: `cedar-policy/cedar` itself is the language's own reference repository (validator test fixtures, not a deployment — the same false positive the paragraph above identifies independently via dependency vendoring) and six repositories' `cedar validate` hits are documentation prose inside the wshobson plugin bundle (e.g., a README recommending "run `cedar validate` to check your policy") rather than an actual invocation. Restricting the search to `path:.github/workflows` — the only location that constitutes real CI enforcement — gives the corrected result: **0 of 10 repositories invoke `cedar validate` inside a `.github/workflows` job** (`results/sprint46_schema_ci_stratification.json`). Two repositories (`jason931225/oyatie`, `microsoft/agent-governance-toolkit`) have a `.cedarschema` file elsewhere in a large monorepo, but neither is confirmed bound to, or covering, the specific misusing policy identified in this audit. This directly answers the natural reviewer question "does this measure deployed risk or only source-code exposure?": within this sample, the structural fix that would close the hazard at load time is not deployed anywhere, so the measured operator-misuse instances represent live, unmitigated runtime exposure, not a source-level artifact caught by existing tooling.

5.6.2 The H2 Analog Is Not the Same Experiment: A Negative Result, Then Not (§5.6.3)

A natural next step is to repeat §5.6.1 for H2 (`.contains()`-on-String). We ran the identical repo-stratified methodology — seed = 42, Wilson 95% CI (`scripts/run_sprint50_h2_contains_audit.py`, `results/sprint50_h2_contains_audit.json`) — against the query `extension:cedar "context." ".contains("` (**309 raw hits**, 59 unique repositories after excluding the 2 purposive-discovery repos, $n = 57$ auditable after 2 download errors). The regex candidate detector (`context\.\w+\.contains\(\s*"[^"]*"\s*\)`) flagged **10/57 repositories (17.5%, Wilson 95% CI 9.8%–29.4%)** — superficially a similar magnitude to H1's 17.9%, and 0/57 additionally matched the fail-open filter. **This 57-repo denominator is not fully independent**: grouping the 57 sampled files by identical repository-relative path, 13 files fall into 6 groups of 2–3 (e.g., three copies of `cedar-examples`' `policies_5b.cedar`, two of `policies/default.cedar`) — none of these 13 are among the 10 H2 candidates, but their presence in the denominator is the same template/fork duplication effect §5.6.1 already flags for the H1 corpus; the Wilson CIs below should be read as within-corpus conditional bounds, not independent-repo prevalence.

We manually reviewed the full source of all 10 candidates before reporting a number, rather than reporting the raw regex count. Every flagged attribute is named as a plural collection (`context.roles`, `context.scopes`, `context.taints`, `context.tagKeys`, `context.osMatrix`, `context.data_classes`, `context.event_relations`, `context.states`, `context.principal_roles`, `context.actor_chain`), and in 7 of the 10 files the same attribute is tested with `.contains()` against *multiple different literals in the same policy* (e.g., `context.taints.contains("ACCESS_PRIVATE") && context.taints.contains("UNTRUSTED_SOURCE")`; `context.osMatrix.contains("ubuntu-latest") && context.osMatrix.contains("macos-latest") && context.osMatrix.contains("windows-latest")`) — a pattern that only makes semantic sense if the attribute genuinely is a `Set<String>`. The remaining 3 (`Govcraft/acton-service`, `jaredgrabill/agentic-control-plane`, `cameronwc/sluice-ai`) call `.contains()` exactly once and rest on the naming signal alone. We went further than naming/pattern inference where we could: a follow-up `.cedarschema`-targeted GitHub search found a schema file in 5 of the 10 candidate repositories. Fetching and reading them, **2 are unambiguously co-located with the candidate policy and explicitly declare the attribute `Set<String>`** (`Ziack/s2s-m2m-identity-stack`: `scopes: Set<String>`, `actor_chain?: Set<String>`; `joeblew999/cf-connectrpc-middleware`: `event_relations?: Set<String>`, schema and policy in the same `examples/remy-sport-policies/` directory) — these two are **confirmed**, not inferred, non-instances. A third (`BadC-mpany/lilith-zero`: `taints: Set<String>`) has a matching declaration but in a schema file one directory level above the candidate policy, so we report it as *plausibly* rather than definitively bound. The remaining two schemas we located (`hupe1980/mako`, `openshift-online/rosa-hyperfleet-api`) turned out to govern a different service directory or test scenario than the sampled file and do not resolve the type; we did not find a schema for the other 5 candidates. The full per-candidate schema check (paths, declared types, and verdicts) is logged in `results/sprint50_h2_contains_audit.json` under `manual_schema_verification` for independent verification, rather than resting on this prose alone. For those, and for the 3 single-literal cases, the classification remains inference from naming and usage pattern, not confirmation. **Confirmed-plus-inferred H2 misuse in this sample: 0/57 (Wilson 95% CI 0.0%–6.3%)** — zero of the 10 candidates could be confirmed as true H2 misuse, and at least 2 (arguably 3) are definitively confirmed as correct usage; the rest remain unresolved-but-consistent with correct usage rather than affirmatively verified.

Why the H1 methodology does not transfer, and what this means for the paper's H2 evidence. H1's regex is reliable precisely because Cedar's `in` operator is *never* semantically valid for testing membership of a value in a literal string list — `in` is exclusively an entity-hierarchy operator, so `context.attr in ["a","b"]` is a type error regardless of what `attr` is named or how many times it appears. `.contains()` has no such property: it is Cedar's real, idiomatic Set-membership check, and — as this audit demonstrates — appears far more often as

correct or plausibly-correct usage than as confirmed misuse in the wild. A static regex on the receiver position cannot, by itself, distinguish a correctly-typed `Set<String>` attribute from a misused `String` attribute; doing so requires either a co-located `.cedarschema` declaration (available and checked for 2–3 of these 10) or dynamic confirmation against a real PDP with a reconstructed schema (not attempted here). We therefore do **not** claim an H2 population-prevalence estimate from this audit — the honest result is a well-defined negative (0/57 confirmed misuse) plus a methodological finding: **H2 prevalence cannot be estimated by the same syntactic-regex approach that works for H1 without a materially larger schema-verification effort**, because the failure mode is a type-confusion between two operations (Set membership vs. String content) that are both individually common and legitimate, whereas H1's failure mode is calling an operator (`in`) that is never legitimate on a String in the first place. A reliable H2 prevalence estimate at corpus scale would require schema-cross-referenced classification for every candidate, not just the 10 we could afford to check by hand here, and is left as future work (§8.1).

Within this specific 57-repository random sample, we found zero confirmed production H2 misuse instances, with 2 of the 10 raw candidates definitively ruled out via schema and the remainder inconclusive-but-consistent with correct usage. We report this as a genuine finding rather than omit it: it is evidence *against* a population-level H2 prevalence claim at the magnitude one might naively infer from the raw 17.5% candidate-match rate, and a concrete illustration of why regex-based corpus audits require the same manual-verification discipline this paper applies elsewhere (§5.6.1's own 14/14 manual review for H1) before any rate is reported as a finding rather than a candidate count. This does not mean H2 has no independent instances in the wild — only that this particular `context.*` -restricted, 57-repository sample did not surface one. §5.6.3 reports what a broader, schema-cross-referenced search found.

5.6.3 Broadening and Schema-Cross-Referencing (Sprint 52): A Confirmed H2 Bypass

§5.6.2's search was restricted to `context.*` attributes and relied mostly on naming/pattern inference rather than systematic schema lookup — the same two limitations §5.6.1 (Sprint 51) identified and fixed for H1. Applying the same two fixes to H2 — broadening the query to any attribute path (`extension:cedar ".contains(\""` , no `context.` -only restriction) and, for every candidate, searching for and fetching the repository's `.cedarschema` rather than inferring from naming — finds a materially different picture (`results/sprint52_h2_schema_crossref.json`). The broadened query returns 261 raw hits across 65 unique repositories; extracting every misuse-shaped `(context|principal|resource).attr.contains("literal")` occurrence gives 110 unique (repository, attribute) candidates. Cross-referencing each against a fetched `.cedarschema` (not a guess) resolves the type for 20 of 110: 16 are schema-confirmed correct `Set<String>` usage, and **4 are schema-confirmed misuse candidates** — the schema explicitly types the attribute as `String` , making `.contains()` a genuine type error, not an inference.

Of the 4, two are not independent evidence: one (`cedar-policy/cedar-go` , `x/exp/schema/validate/testdata/expr_errors.cedar`) is the Cedar language team's own deliberate validator test fixture — the same class of false positive §5.6.1 already excludes for H1's vendored `cedar-policy-core` test files — and one (`jason931225/oyatie`) is another file from the same AI-agent-generated, correlated-authorship monorepo §5.6.1 already excludes from independent-authorship counts. The remaining two candidates are both `SafetyMP/FidusGate` (the `.env` -path and `commandLine` attributes, discussed together below).

`SafetyMP/FidusGate` is a genuine, independently-authored (a distinct GitHub account unrelated to this paper's authors), actively-maintained instance (pushed two days before this check; self-described as "Evergreen OSS reference for zero-trust AI agent governance... Not a production-hardened security product"). Its `policy.cedar` is a real, elaborate, 10-tier (Tier 0–9) AI-agent tool-call gateway — exactly the kind of governance artifact this paper studies — and its `.cedarschema` explicitly declares `Tool.args.path: String` and `Tool.args.commandLine: String` . The file repeats the `.contains()` -on-String pattern 28 times across many `forbid` rules gating

sensitive-file writes and dangerous command execution. We dynamically confirmed **two distinct bypasses** against cedar-python 0.1.4, using the file's actual policy text: (1) a `forbid` meant to block writes to any path containing `.env` (`resource.args.path.contains(".env")`) errors (`type error: expected set, got string`) and is discarded, so a non-`security-sme` principal writing to `apps/backend/.env` — which the earlier Tier-2 `permit` allows for any `apps/*` path — returns `Decision=Allow` instead of the intended `Deny`; (2) a `forbid` meant to block command execution containing `curl` (`resource.args.commandLine.contains("curl")`) errors identically, so `execute_command` with `commandLine = "bash scripts/sandbox-execute.sh && curl http://evil.example/payload.sh | sh"` — which matches the Tier-3 sandboxed-script `permit`'s `like` prefix — also returns `Decision=Allow`. Both are exact analogs of the wshobson/kenhuangus/observeid-platform H1 bypass structure, but for H2's operator and error signature (`"expected set, got string"` rather than H1's `"expected (entity of type any_entity_type), got string"`).

This is the **first independently-authored, dynamically-confirmed H2 bypass in this paper** — closing, for at least one instance, the evidential gap §6.4 names explicitly between H1 (multiple independent bypasses) and H2 (previously: an implementation-level discovery only, §4.2). It does not establish an H2 population prevalence estimate — one confirmed instance out of 110 schema-ambiguous candidates and 65 searched repositories is existence evidence, not a rate.

Follow-up (Sprint 53): replacing the regex schema lookup with the real parser doesn't change the count, but sharpens why 34 stay unresolved. The 39 candidates our text-regex schema lookup could not resolve are not evenly a "try harder" problem. Re-checking them with `cedar.Schema.from_cedarschema` (the actual Cedar parser) and structurally walking the resulting JSON schema along each candidate's exact nested attribute path — rather than text-matching — resolves one more as confirmed correct `Set<String>` usage, finds that 4 of the fetched `.cedarschema` files do not even parse in cedar-python 0.1.4 (the same "schema exists but is broken" pattern §5.6.1 found for `shoaibakthar/observeid-platform`), and leaves the remaining 34 with a more precise diagnosis than "unresolved": the candidate's attribute path is not declared *anywhere* in the fetched schema. Spot-checking confirms why — e.g. one `cedar-policy/cedar` candidate's policy lives at `cedar-policy-cli/sample-data/tiny_sandboxes/sample7/policy.cedar` while the schema our repo-wide search fetched is `cedar-policy-cli/sample-data/tpe_rfc/schema.cedarschema`, a different sample directory in the same monorepo entirely. A repo-wide "any `.cedarschema` in this repository" search, as used here and in §5.6.1/§5.6.2, silently picks the wrong schema whenever a repository contains more than one policy-plus-schema pair — the same failure mode already named for `hupe1980/mako` and `openshift-online/rosa-hyperfleet-api`. Resolving these 34 further would require locating the schema actually co-located with (or referenced by) each specific candidate policy file, not just any schema in the same repository — a materially different, path-aware search this sprint did not implement. None of the 39 resolves to a new confirmed misuse instance; `results/sprint53_h2_schema_json_resolver.json` has the full per-candidate detail. We report the finding at the confidence level it supports: H2 is not merely a theoretical or self-discovered hazard; it has now been found, independently authored and dynamically confirmed, in a real governance tool whose explicit purpose is preventing exactly the kind of AI-agent action this bypass permits.

5.7 Security Claim 1 Replication: Schema S2 (API Gateway / Microservices)

To address the single-schema limitation (§6.4), we instantiate a second Cedar schema for the API Gateway/Microservices domain (`scripts/vectors_s2.py`, `scripts/run_sprint16_schema2.py`). The schema models `Caller → Endpoint` authorization with three principal tiers (public/internal/privileged), three endpoint sensitivity levels (public/internal/secret), and four OWA context attributes that naive PEPs supply caller-side: `caller_authed` (authentication state), `tenant_isolated` (cross-tenant isolation gate), `rate_ok` (rate limit state), and `path_safe` (normalized-path safety attestation).

Taxonomy T'_{S2} (7 vectors, total weight = 18): structural vectors V'1 (tier mismatch bypass, w=2), V'2 (non-owner configure, w=2), V'3a (ASCII path traversal via `like "*..*"`, w=1), V'7 (over-blocking check, w=2); attestation-dependent vectors V'3b (URL-encoded path traversal, w=1), V'4 (authentication bypass via forged `caller_authed`, w=4), V'5 (tenant isolation bypass via forged `tenant_isolated`, w=4), V'6 (rate limit bypass via forged `rate_ok`, w=2). The structural maximum is $R'^{max}_{struct}(T'_{S2}) = 7/18 = 0.3889$.

s2_structural.cedar (no context attestation): the Rust PDP confirms $R'(\pi_{struct}, T'_{S2}) = 7/18 = 0.3889$ — exactly at the Security Claim 1 ceiling. Four vectors remain open: V'3b (URL-encoded path bypasses `like "*..*"`, PERMIT), V'4 (unauthenticated internal caller, `caller_authed=False` → PERMIT), V'5 (`tenant_isolated=False` → PERMIT), V'6 (rate-limited caller, `rate_ok=False` → PERMIT). No structural Cedar modification can close these without PEP context attestation.

s2_attested.cedar (all permits require `caller_authed && rate_ok && tenant_isolated && path_safe` as Cedar permit conditions): $R'(\pi_{att}, T'_{S2}) = 18/18 = 1.0000$. The Rust PDP confirms each bypass is closed: `caller_authed=False` causes no permit condition to fire → DENY; `tenant_isolated=False` → DENY; `rate_ok=False` → DENY; `path_safe=False` with encoded path and no literal `..` → DENY (Cedar's fail-closed: missing permit condition → DENY).

Discrimination check (paired legitimate requests, `scripts/vectors_s2.py:LEGITIMATE_REQUESTS`): for each attestation-dependent vector, a counterpart request uses the **identical entity pair** with the OWA attribute set to its safe value (e.g., V'5: same `svc_a → svc_a` principal/resource, `tenant_isolated=True`). The Rust PDP confirms the attested policy PERMITs all four legitimate counterparts while DENYing all four attacks. This establishes that V'3b, V'4, V'5, and V'6 are genuine attestation-dependent bypasses: the decisive discriminant is exclusively the OWA context attribute value, not any entity-level structural difference. V'5 in particular uses the same-service entity pair — demonstrating that tenant isolation is a per-request routing property the PEP must attest, not an entity-ownership property the Cedar schema can encode.

Domain stability of Security Claim 1:

SCHEMA	TAXONOMY	R^{max}_{struct}	ATT.-DEP. FRACTION	$R(\pi_{att})$
S1 (agentic AI)	T_9 (21 weight)	9.5/21 = 0.4524	52.4%	1.0000
S2 (API Gateway)	T'_{S2} (18 weight)	7/18 = 0.3889	61.1%	1.0000

The ceiling magnitude differs (0.4524 vs. 0.3889), reflecting different domain risk profiles — the API Gateway taxonomy assigns more weight to attestation-dependent vectors (tenant isolation, authentication) than the agentic AI taxonomy. The structural claim of Security Claim 1 — that the attestation-dependent gap exists and PEP context attestation closes it — holds in both schemas, consistent with the schema-independence argument of §5.3.

5.8 Security Claim 1 Language Replication: OPA/Rego

To test whether the attestation gap is Cedar-specific or a property of OWA-based policy languages in general, we instantiate a third evaluation using OPA 0.68.0 (`scripts/run_sprint17_opa_rego.py`, `sandbox/policy-engine/opa/`). OPA/Rego uses the same structural guarantee as Cedar: `input.context.*` attributes are accepted at face value from the request input (OWA), and policy rules do not verify their provenance. A Rego policy that does not require an attested context flag (`context.ownership_verified`, `context.role_attested`, `context.tool_scope_verified`, `context.path_safe`) in its permit conditions is therefore open to the same class of forgery attacks.

Taxonomy T_9-OPA (9 vectors, weight = 21, mirroring Cedar T_9 weights): V1 (role bypass, w=2, structural), V2 (delegation abuse, w=4, structural), V3 (path traversal, w=1, **mixed** — $b_{struct} = 0.5$), V4 (URL-encoded traversal, w=1, **structural** — $b_{struct} = 1.0$), V5 (ownership forgery, w=2, attestation), V6 (role claim manipulation, w=3, attestation), V7 (executor-tool indirection, w=5, attestation), V8 (over-blocking check, w=2, structural), V9

(Unicode path attestation gap, w=1, attestation). Total weight = 21. Structural maximum: $R^{max}_{struct}(T_{9\text{-OPA}}) = 9.5/21 = 0.4524$.

V3/V4/V9 classification rationale: Three vectors illustrate the boundary between structural and attestation-dependent defenses. **V3** (path traversal, w=1) is **mixed** ($b_{struct} = 0.5$): the structural Rego policy's `contains(path, "../")` catches ASCII `../` traversal (one of two sub-tests DENY), but Unicode fullwidth `. . /` (U+FF0E U+FF0E) bypasses it (second sub-test structural PERMIT). This mirrors Cedar T_9 where V3 also has $b_{struct} = 0.5$. **V4** (URL-encoded traversal, w=1) is **structural** ($b_{struct} = 1.0$) in both Cedar and Rego: `%2e%2e` is the single standard URL-encoding of `..`; a one-line `contains(path, "%2e%2e")` literal match closes it completely. One finite string versus an unbounded family — that is the principled classification criterion. **V9** (Unicode traversal in list action, w=1) is **attestation-dependent** in both ($b_{struct} = 0.0$): Unicode contains many dot-like codepoints (U+FF0E, U+2024, U+0660, overlong UTF-8, double-encoded forms), and no finite literal set can enumerate all variants. PEP path normalization — not literal matching — is the only sound defense.

V5 (ownership forgery): The Rego exploit request uses `resource.owner == principal.id` (the attacker self-declares ownership to pass the entity-level structural check) but supplies `context.ownership_verified = false`. The structural Rego policy PERMITs (entity check passes, no context check) while the attested policy DENYs (`ownership_verified=true` required in every write/delegate permit condition). This is the canonical Security Claim 1 fingerprint in Rego: identical entity pair, differing only in the OWA context attribute.

OPA results (OPA 0.68.0, `results/sprint17_opa_rego.json`):

POLICY	V1	V2	V3	V4	V5	V6	V7	V8	V9	R(Π)
Structural	√	√	~ (0.5)	√	×	×	×	√	×	**0.4524**
Attested	√	√	√	√	√	√	√	√	√	**1.0000**

$R^{max}_{struct}(T_{9\text{-OPA}}) = 9.5/21 = 0.4524$ confirmed (V3 contributes $0.5 \times 1 = 0.5$, V4 contributes $1.0 \times 1 = 1.0$); attested Rego policy reaches $R = 1.0000$.

Cross-language ceiling summary:

LANGUAGE/SCHEMA	TAXONOMY	R^{max}_{struct}	$R(\pi_{att})$	V4 STATUS
Cedar S1 (agentic AI)	T_9 (w=21)	9.5/21 = 0.4524	1.0000	Structural (explicit %2e%2e forbid)
Cedar S2 (API Gateway)	T'_{S2} (w=18)	7/18 = 0.3889	1.0000	Attestation-dependent
Rego/OPA	$T_{9\text{-OPA}}$ (w=21)	9.5/21 = 0.4524	1.0000	**Identical to Cedar S1**

Cross-language consistency check. Cedar S1 and Rego/OPA reach an **identical** ceiling (9.5/21 = 0.4524) on the same taxonomy T_9. This equality is expected by construction — both share the same attestation-dependent weight partition (V5/V6/V7/V9, combined weight 11.5) and the same total weight (21) — so the bound is $R^{max}_{struct} = (21 - 11.5)/21 = 9.5/21$. The result is presented as a **consistency check** (not an independent empirical confirmation): it verifies that the bound is language-neutral — not an artifact of Cedar's string operators or Rego's built-ins — and that R(π) is implementation-independent when applied to a shared taxonomy. The Cedar S2 ceiling (7/18 = 0.3889) is lower because it reflects a different domain taxonomy and weight distribution.

The attestation gap is not Cedar-specific: it is a property of any OWA-based policy evaluation language. The "Cedar attestation gap" label refers to the target deployment context (Cedar authorization policies), not a property unique to Cedar's formal semantics.

Fail-open is architectural, not just a ceiling. The structural-ceiling replication above measures *how much* risk weight OPA and Cedar leave open under an OWA context; it does not by itself show *whether* the two engines fail open or closed when the type-error hazard (§4.2, H1/H2) actually fires. We separately ran the operator-misuse

governance policies (block `gh pr merge`, `git push --force`, `gh release create`) directly in OPA 0.68.0 to compare decision outcomes for the same adversarial inputs (`results/sprint_A_opa_contrast.json`):

POLICY PATTERN	PDP	RULE ERROR BEHAVIOR	DECISION FOR ADVERSARIAL INPUT
`forbid when { in-on-String }; permit`	Cedar	forbid silently discarded (error-ignorance)	**ALLOW** (fail-open)
`default allow:=false; is_privileged if { set membership }; allow if { not is_privileged }`	OPA	rule fires correctly → DENY	**DENY** (fail-closed)
`default allow:=false; is_privileged if { undefined field ref }; allow if { not is_privileged }`	OPA	rule undefined → negation-as-failure: `not undefined` = `true` → allow fires	**ALLOW** (fail-open)
`default allow:=false; allow if { positive condition }`	OPA	condition errors → rule doesn't fire → default	**DENY** (fail-closed)

OPA's canonical governance idiom — `default allow:=false; allow if { positive condition }` — is genuinely fail-closed: a type error prevents the `allow` rule from firing, and the policy-level default takes over. But OPA's *negation-as-failure* idiom (`allow if { not is_blocked }`, where `is_blocked` errors) exhibits the identical structural fail-open behavior as Cedar's H1: Rego's `not` operator evaluates `not undefined` to `true`, so the `allow` rule fires. Cedar's error-ignorance and OPA's negation-as-failure are semantically equivalent in their fail-open effect; the practical asymmetry between the two ecosystems is architectural rather than semantic. OPA's idiomatic governance pattern is a positive condition (fail-closed by convention); Cedar's idiomatic governance pattern is `forbid when { blocked condition }; permit` (a negation pattern, fail-open by convention). The error-ignorance rule (§2.1-equivalent, the Cedar specification quote in §4.2) is the mechanism that makes Cedar's fail-open *silent* — `r.decision == "Allow"` with no error surfaced to the caller — but OPA's negation-as-failure fail-open is equally silent to a caller who checks only `result.allow`. The OWA attestation gap (Security Claim 1) is orthogonal to both idioms: it applies to any correct, type-safe policy over adversary-controlled context, independent of which PDP or error-handling convention is in use.

Independent confirmation: the same methodology finds a real, publicly-published instance in Kubernetes admission control. A concurrent, independently-anonymized AISec 2026 submission [38] applies the dual-loop/mutation-search methodology of this paper to OPA/Gatekeeper specifically, rather than treating the illustrative table above as the full boundary of OPA's negation-as-failure hazard. Rather than assume the obvious pattern (a direct `!=`) is representative, it runs a dynamically-verified mutation search over 14 guard-expression shapes against real OPA 0.68.0 and finds the safe/unsafe boundary is narrower than expected: **only negated equality (`not (X == Y)`) is safe on an undefined operand — every relational operator, `!=` itself, and every negated builtin call (`re_match`, `startswith`, `contains`, `object.get`, `in`) fail open, 13 of 14 tested forms, all dynamically confirmed rather than extrapolated.** Using an AST-based classifier built for this confirmed taxonomy (self-corrected after an earlier version conflated Rego's `:=` assignment operator with the hazard, inflating candidates to 69/190 files — a false-positive rate the authors regenerate and release for provenance rather than silently fix), a corpus audit finds that a narrow, single-pattern search — the kind naively modeled on the pattern's most obvious surface form, structurally analogous to how this paper's own H1 was first discovered — finds zero confirmed instances in 175 auditable files, while broadening the query to the full confirmed taxonomy surfaces one file with two independently dynamically-confirmed bypasses in a Kubernetes admission-control policy governing AI-platform workloads (bare `<` on an HPA's `minReplicas`, bypassed by omitting a field the Kubernetes API server itself defaults; `count(...) == 0` on `topologySpreadConstraints`, dead code against real-world input) — a real instance a narrow search would have missed entirely. OPA's own linter (Regal v0.42.0) flags 117 stylistic issues

in that file and zero in this hazard class. Combined corpus: 1/310 unique auditable files (Wilson 95% CI), reported honestly as a methodological/existential contribution rather than a population estimate. This is independent evidence — a different researcher, a different engine, the same dual-loop discovery discipline — that "a deny rule silently defeated by an evaluation-time absence, with the engine's permissive default filling the gap invisibly" is a structural failure mode of policy-evaluation engines generally, not an artifact specific to Cedar's schemaless mode or to this paper's search methodology.

5.9 Attestation-Gate Construction Lint

Every permit rule in an attested policy must require at least one @pep_attested attribute to be True — otherwise a caller who forges all OWA attributes can still obtain PERMIT through the unguarded rule. We provide a mechanized Z3 check for this construction property across the three attested policies.

Construction lint check. For each attested policy π, we encode the permit formula in Z3 with all @pep_attested attributes fixed to the constant False and entity/action/resource variables left free over their declared integer domains (4 roles, 5 actions, 3 resource types). Because every permit in π conjunctively requires at least one pep attribute = True, substituting False drops a False conjunct into every disjunct of the PERMIT formula — making PERMIT unsatisfiable regardless of the entity variables. Formally:

$$\text{UNSAT}: \quad \exists(\vec{e} \in D): \ \text{PERMIT}(\pi, \ \vec{v}_{PEP} = \mathbf{0}, \ \vec{e})$$

where D is the declared finite domain of roles, actions, and resource types. The UNSAT result is a **proof within this domain**: over all (role, action, resource_type) combinations with PEP attributes all-False, no permit path exists. A complementary SAT sanity check (pep=all_True, entity=free) confirms the formula is not trivially overconstrained.

Scope and limitation. This is a *construction lint*, not a full security proof. The domain D encodes the four declared role values and three resource types as integer constants — the check is decidable and complete over D but does not generalize to an open entity universe. Entity-attribute values (e.g., whether resource.owner matches principal.id) are modeled as free Booleans rather than derived from entity-store state. A SAT witness under pep=all_True is a (role, action, resource_type) tuple, not a fully instantiated entity-store request. Extending to an open-universe decision procedure aligned with Zelkova's SMT approach [9] is a direction for future work (§7.2); the exhaustive 768-state enumeration (§5.5) provides the empirical complement on the bounded request universe.

Results (`scripts/run_sprint19_owa_independence.py`, `results/sprint19_owa_independence.json`):

POLICY	UNSAT (PEP=FALSE)	SAT SANITY (PEP=TRUE)	TIME	PROPERTY
Cedar S1 (agent_policy.cedar)	**unsat** √	sat √	3.3 ms	lint passes
Cedar S2 (s2_attested.cedar)	**unsat** √	sat √	2.7 ms	lint passes
Rego S1 (agent_attested.rego)	**unsat** √	sat √	3.6 ms	lint passes

All three pass in under 4 ms. The check's development-time value: a permit rule accidentally omitting every attestation gate would appear as SAT, flagging the construction defect before deployment.

Semantic bypass certification (Sprint 20). The construction lint (§5.9) is a *syntactic* check: it verifies that every permit rule in the policy text conjunctively requires at least one PEP attribute = True. We complement it with a *semantic* bypass check that directly asks: does there exist a well-typed request with all PEP attributes False that achieves PERMIT under actual Cedar evaluation semantics?

We encode $\varphi(\pi)$ = SAT iff $\exists$ well-typed r with $\vec{v}_{PEP} = \mathbf{0}$: PERMIT($\pi$, r) — where r is constrained to the declared finite domain (4 roles, 5 actions, 3 resource types). For π_2: $\varphi(\pi_2)$ = **UNSAT** (< 0.2ms) — no PERMIT path exists

with all PEP attributes False. For π_1: $\varphi(\pi_1)$ = **SAT** with witness `{role=WRITER, action=READ, rtype=RESOURCE, is_owner=True}`.

Scope caveat (resolved by Sprint 21). The π_1 SAT witness `{role=WRITER, action=READ, rtype=RESOURCE, is_owner=True}` is a *false alarm*: `is_owner` is derived from the entity store (`resource.owner == principal.id`), which is PEP-managed (not caller-forgeable). Sprint 20 modeled `is_owner` as a free Boolean, exposing a policy-text gap (no `ownership_verified` conjunct in π_1) but incorrectly suggesting an adversary could claim ownership at runtime. The entity-store-aware encoding that resolves this — modeling `resource_owner_id` as a system-derived integer constant and computing `is_owner = (principal_id == resource_owner_id)` via Z3 integer equality — has been completed in Sprint 21 (see below). The π_2 UNSAT result is unaffected: under pep_all_False, every permit path requires a PEP attribute = True by construction, and UNSAT holds regardless of is_owner's modeling (`scripts/run_sprint20_bypass_certification.py`, `results/sprint20_bypass_certification.json`).

Entity-store-aware safety certification (Sprint 21). Sprint 20's free-Boolean `is_owner` model produces a false-alarm V5 bypass witness in π_1. Sprint 21 closes this by encoding Cedar's entity-store evaluation semantics directly in Z3: two integer variables `principal_id` and `resource_owner_id` (free over the solver's domain), with `is_owner = (principal_id == resource_owner_id)` as a *derived constraint* rather than a free Boolean. The solver cannot satisfy `is_owner = True` unless it assigns equal integers to both variables — no caller-supplied forgery can satisfy this; only entity-store compromise would.

Sprint 21 introduces a **SAFETY PROPERTY** that directly encodes Cedar's closed-world entity-store facts: a request is UNSAFE iff (i) it accesses a Resource or Tool with `path_safe_entity = False` (entity-known path hazard); (ii) a WRITER accesses a Resource without owning it (`is_owner = False`, derived from integer equality); (iii) an ADMIN delegates without IDP-confirmed role (`role_in_idp = False`); (iv) an EXECUTOR uses a Tool with `tool_scope_ok = False`; or (v) an ADMIN delegates to self or a non-eligible target. Context attributes (`path_safe`, `ownership_verified`, `role_attested`, `tool_scope_verified`) are now explicitly bifurcated: PEP-derived context (π_2 Check A) sets each context flag equal to its entity-store source; adversary-supplied context (π_1/π_0 Check B) leaves each context flag as a free Boolean the solver can set adversarially.

Two checks certify the result. **Check (A): π_2 with PEP-derived context → UNSAT (3.47ms).** The encoding adds `pep_path_safe = path_safe_entity`, `pep_own_verified = is_owner`, `pep_role_att = role_in_idp`, `pep_tool_scope = tool_scope_ok`. Under these constraints, every permit path in π_2 requires the corresponding entity-store fact to be True; the SAFETY PROPERTY fires whenever any is False; the conjunction of PERMIT(π_2, PEP-ctx) ∧ UNSAFE(entity-facts) is unsatisfiable. **Check (B): π_1 with adversary OWA context → SAT (2.49ms).** The solver finds a V7 bypass witness: `{role=EXECUTOR, action=execute, rtype=Tool, tool_scope_ok=False}`. π_1's executor permit has no `tool_scope_verified` gate, so PERMIT fires; entity fact `tool_scope_ok=False` makes the SAFETY PROPERTY fire. Crucially, **V5 is correctly absent**: the solver cannot forge `is_owner` by setting `principal_id == resource_owner_id` independently — the only way to pass π_1's entity-store ownership check (`resource.owner == principal.id`) is to genuinely match owner IDs, which the adversary cannot do by modifying a context attribute. V5 would require compromising the entity store itself — an infrastructure concern outside Cedar's policy-evaluation layer.

CHECK	POLICY	CONTEXT MODEL	Z3 RESULT	TIME	INTERPRETATION
A	π_2 (agent_policy.cedar)	PEP-derived (entity-sourced)	**UNSAT** ✓	3.95 ms	CERTIFIED — π_2 + PEP never grants UNSAFE request
B	π_1 (partial_policy.cedar)	Adversary OWA (forgeable)	**SAT**	2.66 ms	V7 bypass: executor uses out-of-scope Tool
B	π_0 (baseline_policy.cedar)	Adversary OWA (forgeable)	**SAT**	2.40 ms	V3: literal '..' in path, no structural guard in π_0

CHECK	POLICY	CONTEXT MODEL	Z3 RESULT	TIME	INTERPRETATION
A (neg. control)	π_2_mutant (tool_scope + F_eu dropped)	PEP-derived	**SAT** √	2.43 ms	V7 bypass — proves Check (A) is not tautological

(`scripts/run_sprint21_entity_aware_cert.py` , `results/sprint21_entity_aware_bypass.json`)

Scope of the certificate (consistency, not independence). Check (A) proves that π_2's PEP-attested guards structurally dominate every modeled entity-store hazard — an internal *consistency* property of the policy against the SAFETY PROPERTY we defined. It does not certify safety against an *independent* external specification: the SAFETY PROPERTY was constructed from the same threat model that motivated π_2's permit guards, so the UNSAT is expected by design when the policy is correct. The certificate's value is that it would flip to SAT if a guard were accidentally omitted — the negative control confirms this. The negative control mutates π_2 by dropping `tool_scope_verified` from both the executor permit (`P_executor`) and its belt-and-suspenders forbid (`F_exec_unver`): Check (A) on the mutant produces SAT with V7 witness `{EXECUTOR, execute, Tool, path_safe_entity=True, tool_scope_ok=False}` in 2.43ms. A single omission from the permit alone is insufficient: π_2's `F_exec_unver` forbid is a redundant guard that catches tool-scope failures even when the permit forgets to check, demonstrating the defense-in-depth structure; only removing both reveals the gap. The full UNSAT guarantee against an independent π^* (ground-truth specification) remains open and requires Cedar's own SMT evaluator (§8.1).

Four verification modalities for π_2. (1) Exhaustive 768-state enumeration (§5.5): 0 bypasses empirically across the full bounded request universe. (2) Syntactic lint (§5.9): every permit text requires ≥1 PEP conjunct — no permit rule is accidentally unguarded. (3) Semantic bypass certification (§5.9): UNSAT — no well-typed bypass with pep_all_False exists under Cedar evaluation semantics. (4) Entity-store-aware consistency check (§5.9, Sprint 21): UNSAT — π_2's PEP guards subsume every entity-store hazard in the defined SAFETY PROPERTY; the negative control confirms the check detects real omissions. These four modalities address the same bounded request universe from four different angles: empirical enumeration (1), syntactic construction (2), semantic bypass (3), and entity-store hazard consistency (4). Together they establish that π_2 achieves $R(\pi_2, T_9) = 1.0000$ across all four verification modalities within the bounded request universe.

5.10 Existing-Tool Baseline: Does `cedar validate` Already Close H1/H2?

Our fix (§4.5, §5.1–5.3) proposes new provenance annotations; a natural reviewer question is whether Cedar's *existing* validator already closes the type-error hazards (H1: `in` -on-String; H2: `.contains()` -on-String, §4.2) without any new tooling. We answer this directly rather than assert it, using the real validator binding (cedar-python 0.1.4 `Schema.validate_policyset` , not a re-implementation).

Two-example baseline (Sprint 48). Given a schema declaring the relevant attribute's real type (`String`), the validator flags both patterns: the `in` -on-String idiom produces two type errors (`expected...AnyEntity but saw String` ; `expected Set<AnyEntity>... but saw Set<String>`), and the `.contains()` -on-String idiom produces one (`expected Set<Any> but saw String`) (`results/sprint48_validator_baseline.json`).

Catch rate on the Sprint 43 misuse corpus. Two hand-picked examples do not establish generality. We extended the check to every real `in` -on-String pattern captured verbatim in the **§5.6.1 file-level random sample (Sprint 43, not the repo-stratified Sprint 43b)**: for each of the 87 misuse files identified there, we regex-extracted the context attribute name, declared it `String` in a minimal one-attribute schema (the type implied by the pattern's own literal usage), wrapped the exact matched substring in a standalone `forbid` rule, and ran the real validator. Result: **158/158 (100%) of captured patterns are flagged** (`results/sprint49_validator_corpus_catchrate.json`). This is not as broad as "corpus-wide" might suggest: consistent with the §5.6.1 caveat that Sprint 43's file-level sample is itself dominated by one repository, 139 of the

158 patterns (88%) come from `jason931225/oyatie` alone — an AI-agent-generated platform of unverified deployment status (§5.6.1), not a confirmed production system; the check spans only a handful of genuinely distinct codebases, not 87 independent ones. We report the 100% figure as a **structural guarantee, not an empirical coincidence**: a schema that correctly types the attribute as `String` makes `in` (an entity-hierarchy operator) or `.contains()` (a Set operator) a type mismatch by definition, and Cedar's validator soundly rejects every ill-typed policy against its schema (Cutler et al. [1]) — the 158/158 result is expected to generalize to any correctly-typed schema on structural grounds, not because 158 independent codebases were checked. **The honest caveat**: this guarantee is conditional on the schema itself being correct. An author who mistypes the attribute (e.g., declares it as an entity type to silence the validator without fixing the underlying `in`-usage) preserves the bug while passing validation — an "escape hatch" that argues for schema review as part of the fix, not merely schema *presence*. Combined with the §5.6.1 finding that 0/10 real misuse repositories run `cedar validate` in CI, the practical gap is adoption, not tooling: the fix already exists and works, but it is opt-in and not deployed in any production repository we examined.

---

6. Discussion

6.1 The OWA/CWA Boundary as a Design Principle

The attestation gap reveals a fundamental tension. Cedar's ABAC model evaluates policies against *stated* context attributes (OWA), but security requires that critical attributes be *externally verified* (a CWA-style guarantee). Policy hardening therefore cannot stop at the policy text: it must extend to the attestation infrastructure that supplies the attributes the policy reasons over. A policy that looks airtight on paper is only as strong as the PEP's attestation of `ownership_verified`, `role_attested`, and `tool_scope_verified`. The dual-loop framework surfaces this boundary empirically: when the blue loop cannot advance R(π) past 0.4750 with any Cedar modification, it has reached the attestation ceiling.

6.2 Structural vs. Attestation Controls

Our taxonomy partitions vulnerability classes by *mitigation type*:

- **Fully structural (V1, V2, V4):** closeable by policy forbids alone, independent of the PEP. Combined weight 7/21 (τ_9).
- **Mixed (V3):** partially structural (V3.1 ASCII and V3.3 mid-path closed by `like "*..*"`), partially attestation-dependent (V3.2 Unicode fullwidth dots require PEP `path_safe` attestation). Weight 1/21; $B^{max}_{struct} = 0.5$.
- **Fully attestation-dependent (V5–V7, V9):** require PEP infrastructure; structural forbids are insufficient or outright exploitable. Combined weight 11/21 (τ_9: V5+V6+V7=10, V9=1).
- **Over-blocking check (V8):** PERMIT-expected false-positive check; closeable without any additional rules (baseline permits are sufficient). Weight 2/21.

Because the attestation-dependent class (V5–V7, V9 fully + V3.2 partially) accounts for ≈54.8% of total risk weight in τ_9, policy designers should prioritize attestation controls over structural controls when allocating hardening effort. The intuitive ordering — "first write all the forbids, then worry about the PEP" — inverts the actual risk gradient. The R(π) metric makes this quantitative: moving from π_0 (0.2000) to π_1 (0.4750) by adding structural forbids (§A.2) yields 0.2750 of R(π) improvement; moving from π_1 (0.4750) to π_2 (1.0000) by adding attestation conditions yields 0.5250 of improvement — but requires PEP infrastructure, not more policy text.

6.3 Engine Implementation Bugs as a Security Concern

The schemaless-mode silent policy degradation hazard (§4.2) surfaces a class of risk orthogonal to policy design: *the intersection of schemaless evaluation and Cedar's documented policy-error semantics*. Cedar's specification explicitly

states that erroring policies are ignored; this is internally consistent for Cedar's intended use (schema-validated deployments where type errors are caught at policy-load time). The hazard emerges in schemaless mode: a developer who writes `resource.path.contains("..")` as a path-traversal guard — following Python/Java string-method intuition — loads the policy without error (no parse-time type rejection in schemaless mode), then receives PERMIT for path-traversal requests while the authorization response quietly populates `response.errors` with `"type error: expected set, got string"`. Unless the caller explicitly inspects and fails on non-empty `response.errors`, the forbid rule is silently neutralized. The broader implication: Cedar schema validation is a **security requirement for production deployments**, not an optional ergonomic feature. Operating in schemaless mode exposes any Cedar deployment to silent policy degradation on type-erroneous forbid rules.

This finding generalizes: any authorization system that silently discards failing `forbid` rules rather than failing closed (or rejecting the policy) is susceptible to this pattern. The safest mitigation is defense-in-depth: complement string-matching forbids with PEP attestation (`context.path_safe`), so that an engine bug on one defense does not leave the vector completely open.

6.4 Limitations

Scope of evaluation. This paper presents a proof-of-concept measurement study on a single Cedar schema (S1), with replications on a second Cedar schema (S2) and OPA/Rego. The primary policy corpus contains 25 policies (4 adapted from public cedar-examples, 21 synthetic); the out-of-sample validation contains 3 unseen cedar-examples policies. These numbers are insufficient to support strong generalization claims; the results should be interpreted as establishing that the attestation gap phenomenon exists and is measurable, not as providing population-level estimates of its prevalence or precise magnitude in real-world Cedar deployments.

- **Red loop mutation coverage.** The adaptive mutation engine generates 9,130 mutations across 7 strategies (single-request, single-field perturbations). The "0 novel bypasses on hardened policies" result is the count of *tag-distinct bypass instances*, not semantically distinct attack classes — strategy tags never contain canonical vector names, so every mutation instance is technically "novel" by the tag test. The load-bearing claim is the underlying "0 total bypasses across 1,540 mutations per hardened policy," supported by convergence at round 2. Multi-step sequences, Cedar action-hierarchy exploits, and entity-attribute injection are not covered. The batch evaluator silently converts Rust-engine evaluation errors to DENY, so any schema-invalid mutation appears "blocked" rather than "errored" — a conservative fail-closed bias that could undercount bypasses on policies with strict schema validation.
- The `String.contains()` schemaless-mode hazard (§4.2) has been re-characterized: it is **not a bug in the cedar-policy Rust engine or the cedar-python binding** (both correctly implement Cedar's specification, which documents that erroring policies are ignored). The finding is instead a **deployment hazard**: Cedar schema validation is a security requirement, not an optional feature; schemaless deployments that check only `response.allowed` are vulnerable to silent policy neutralization on type-erroneous forbid rules. A disclosure to the cedar-policy/cedar project has been filed (cedar-policy/cedar#2428, 2026-06-24) proposing: (a) schemaless load-time warning for `in` on non-entity operands; (b) a "common mistakes" developer guide entry; (c) a `cedar lint` mode. The affected policy maintainer has also been notified (wshobson/agents#598, 2026-06-24).
- **Single deployment, single schema (addressed).** Security Claim 1's per-class result ($B_{struct}(\pi, V) = 0$ for attestation-dependent classes) is schema-independent — the argument from Cedar's CWA evaluation semantics applies to any schema S and any vulnerability class V in C_{att}. However, all quantitative results for the agentic AI domain, including $R^{max}_{struct}(T_8) = 0.4750$ and $R^{max}_{struct}(T_9) = 0.4524$, derive from a single Cedar schema (Appendix A). Three replications close this limitation: §5.7 (Cedar S2 — API Gateway/Microservices, $R'^{max}_{struct}(T'_{S2}) = 7/18 = 0.3889$, Rust PDP confirmed); §5.8 (OPA/Rego, $R^{max}_{struct}(T_{9\text{-OPA}}) = 9.5/21 = 0.4524$, OPA 0.68.0 confirmed, **identical** to Cedar S1 when the structural policy applies all closeable defenses); and the discriminant-validity P3 result (§6.5: a structural-only Cedar policy constructed independently of the framework lands exactly at $R^{max}_{struct}(T_9) = 0.4524$). In all three, attested policies reach R = 1.0000 and structural policies are bounded at the

Security Claim 1 ceiling. The ceiling magnitude is domain-specific (Cedar S2: 0.3889 for a different taxonomy) but language-stable: Cedar and Rego reach the same bound (0.4524) on the same taxonomy. Validation that other schemas do not introduce attack classes the current taxonomy misses remains an open challenge.

- **Oracle-engine concordance gap (closed).** In the original 9-iteration live-agent run, the blue-loop online oracle was the pure-Python Cedar evaluator; the `String.contains()` fail-open was surfaced only by post-hoc validation. Post-fix concordance is 32/32 = 100.0% across 4 policies × 8 vectors (`scripts/validate_rust_pdp_commit.py`). An end-to-end re-run with `CedarRealEvaluator` as the sole commit oracle (`scripts/run_sprint18_rust_pdp_loop.py`) converged to $R(\pi) = 1.0000$ in 6 iterations with zero rollbacks, confirming the hardening trajectory is fully reproducible under the production Rust engine and that no oracle-engine divergence affects the results.
- **Evidential asymmetry across the three hazards this paper reports, and within H1/H2 themselves.** The three empirical findings in this paper do not carry equal evidential weight, and we state the gradient explicitly rather than let a reader infer it from where each result appears. H1 (`in` -on-String) has the strongest evidence: three independently-authored dynamically-confirmed bypasses (§5.6, §5.6.1 — two from the original purposive-discovery corpus plus one found by broadening the search, §5.6.1's Sprint 51), a randomized population-prevalence estimate that we corrected downward after per-file review — raw regex match 17.9%, production-plausible 7.7%–10.3% (6–8/78) once non-deployment test/mirror artifacts and one correlated-authorship AI-agent-generated monorepo are excluded on the same de-duplication grounds already used for the wshobson case (§5.6.1) — and a validator catch-rate of 158/158 on the file-level misuse sample, though that sample is itself 88% concentrated in the same agent-generated repository, not evidence from 158 independent codebases (§5.10). H2 (`.contains()` -on-String) is real — confirmed in the cedar-python 0.1.4 implementation via the framework's own oracle divergence (§4.2) — and now has **one independently-authored, dynamically-confirmed corpus instance** (`SafetyMP/FidusGate` , §5.6.3), found only after applying the same two fixes §5.6.1 applied to H1: broadening beyond `context.*` and replacing naming inference with schema cross-referencing. The narrower, `context.*` -restricted, inference-based search (§5.6.2) found 0/57 confirmed misuse, because `.contains()` , unlike `in` , is Cedar's legitimate Set-membership operator and a syntactic regex cannot distinguish correct use from misuse without schema information — but that search's limitations, not H2's absence from the ecosystem, explain the earlier null result. The attestation gap (Security Claim 1, V5–V7/V9) sits at the other extreme: it has the single most dramatic piece of evidence in the paper — a live end-to-end exploit with real (simulated-environment) financial impact (§5.6, Sprint 32, `results/sprint32_live_exploit_confirmed.json`) — but no random-sample prevalence audit at all, only existence (the minorun365/agentcore-book bypass) plus discriminant validity (§6.5). This last gap is not an oversight we can patch cheaply: unlike H1, which is detectable from policy text alone via a syntactic regex (and unlike H2, where the regex at least enumerates candidates for manual review even though it cannot confirm them), classifying a context attribute as OWA or CWA requires locating the PEP code that constructs the request context — often in a different language, sometimes in a different repository — which is why our production-corpus evidence for the attestation gap (§5.6) required a source-code provenance methodology rather than a keyword search. A random-sample prevalence audit of the attestation gap specifically is future work (§8.1) and is a materially larger undertaking than either the H1 or the H2 audit was.

6.5 Threats to Validity

Construct validity. The Sprint 32 live-exploit demonstration built a FastAPI + cedar-python 0.1.4 gateway harness authored by this research team, which creates a circularity risk: a self-built gateway demonstrating a bypass in a self-built system may not generalize to independent deployments. Sprint 41 partially addresses this through a cross-engine replication: we confirm the `in` -on-String fail-open in `@cedar-policy/cedar-wasm` **v4.11.2** (official Cedar WebAssembly package maintained by the Cedar team at AWS, compiled via wasm-bindgen, independent of the PyO3 binding used in cedar-python) using its Node.js `isAuthorized()` API with each of wshobson's four forbid-rule patterns — 4/4 governance gates bypassed, with a real negative control (corrected `.contains()` idiom

→ `deny`, zero errors) confirming the harness is sound (`results/sprint41_protect_mcp_wasm_bypass.json`). This rules out a cedar-python PyO3 binding artifact: the bug is in the core Cedar Rust engine, confirmed in two independent bindings across two versions. The remaining circularity (the FastAPI gateway in Sprint 32 is self-built) is not fully closed by Sprint 41 — the cedar-wasm test demonstrates the engine component behavior, not an end-to-end independent deployment. A true end-to-end non-circular demonstration would require testing the bypass through a Cedar evaluation stack authored and operated by a third party (e.g., AWS Verified Permissions, or the wshobson plugin's actual `protect-mcp@0.5.5 evaluate --policy` CLI path in a live Claude Code session); both remain as future work. What Sprint 41 does establish is that the fail-open behavior is not an artifact of the specific binding used in the baseline experiments.

R(π), the V1–V9 taxonomy, the entity/context schema, and the Python commit-oracle were all authored by the same team; convergence to R(π)=1.0000 certifies robustness *relative to a self-defined attack surface and a bounded request universe* (4 roles × 5 actions × 2 resource types × 2^4 context flags = 640 distinct request classes), not robustness in an absolute sense. The adaptive-mutation and exhaustive-enumeration checks reduce but do not eliminate this circularity, since both inherit the same schema closure; the §5.5 coverage figures ($C(T_8) = 5.3\%$, $C(T_9) = 6.6\%$ after V9 extension) quantify the residual exact-fingerprint gap; the 71 remaining states fall in $(role, action, resource\_type)$ groups already represented by V5–V7 or V9 canonical requests.

To probe discriminant validity, we evaluated R(π) over T_9 against four naive Cedar policies written without knowledge of the V1–V9 taxonomy, using the cedar-python 0.1.4 Rust PDP as the oracle for all seven policies (three trajectory anchors plus four naive; `scripts/run_sprint15_discriminant.py`, `results/sprint15_discriminant_v1.csv`). **P1** (role-based permits plus a single `like "*..*"` path guard) scored R=0.2143 (V2, V4–V7, V9 bypass; V3 partial at B=0.5 for the Unicode variant). **P2** (role-only permits, no path guard, no context conditions) scored R=0.1905 — matching the intentionally weak baseline exactly, showing that a structurally simpler policy is no better than the unarmed baseline against our taxonomy. **P3** (role restrictions, ownership check, delegation safety, and ASCII/encoded path guards — all standard structural best practices) scored R=0.4524, hitting $R^{max}_{struct}(T_9)$ exactly; a structural-only policy constructed to apply every non-attestation-dependent countermeasure lands precisely at the Security Claim 1 ceiling and cannot exceed it without PEP attestation. **P4** (same structural controls plus `context.ownership_verified` and `context.path_safe` in permits, but missing `role_attested` for delegation and `tool_scope_verified` for execution) scored R=0.6190 — strictly above R^{max}_{struct} because partial attestation closes V5 (ownership_verified, $\Delta R = +2/21$), V9 (path_safe, $\Delta R = +1/21$), and upgrades V3 from B=0.5 to B=1.0 (path_safe catches the Unicode variant, $\Delta R = +0.5/21$), totalling $+3.5/21$; V8 (over-blocking) is already closed at the structural ceiling and is not an attestation-dependent vector. V6 ($\Delta R = +3/21 = +0.1429$) and V7 ($\Delta R = +5/21 = +0.2381$) remain open because neither `role_attested` nor `tool_scope_verified` is present. The four-policy range [0.1905, 0.6190] shows R(π) correctly penalises policies not designed to pass the taxonomy, correctly places structural-only policies at the Security Claim 1 ceiling, and correctly distinguishes partial from complete attestation coverage — all necessary conditions for a valid discriminant instrument.

The blue and red agents share a model family (Sonnet/Opus), and the single orchestrator both halts the loop and arbitrates which red-loop findings are escalated, with a stopping criterion that explicitly rewards reaching R(π)=1.0000. The adaptive red-loop strategy vs. random-mutation ablation (§4.4) confirms convergence soundness: both methods correctly identify 0 real bypasses on the hardened final policy. The orchestrator's qualitative OWA/CWA meta-pattern contribution (not capturable by per-round bypass counts) remains an open ablation target.

Internal validity. The 9-iteration live-agent run used a pure-Python evaluator as the commit oracle; the one security-critical divergence (the `contains()` fail-open) was surfaced by post-hoc validation rather than autonomously. Safety in that instance depended on the online oracle being coincidentally stricter than the Rust engine. The end-to-end re-run with the Rust PDP as the commit oracle (§4.3) eliminates this dependency for future runs. For V_cvb (adversarial attestation forgery), the Cedar evaluator correctly PERMITs the forged request —

`mcp_hardened_policy.cedar` reaches R_MCP=0.9167, not 1.0000, under the full V_cva+V_cvb taxonomy — confirming that the policy closes the *structural* cross-vendor gap (V_cva, CWA-conditioned) but not the *adversarial forgery* gap (V_cvb, OWA scenario), the latter requiring external trust infrastructure as Proposition 2 asserts.

External validity. Security Claim 1 (the per-class attestation gap) is schema-independent — the proof follows from Cedar's CWA evaluation semantics for any schema S and any $V \in T_{att}$. All quantitative results derive from one or more evaluated configurations; three replications establish coverage: §5.7 (Cedar S2, second Cedar schema, API Gateway domain, Rust PDP), §5.8 (OPA/Rego, second policy language, OPA 0.68.0), and §6.5 (discriminant-validity P3, structural-best Cedar policy constructed independently). The language-stability finding (§5.8) is the strongest external-validity result: Cedar S1 and Rego/OPA reach identical structural ceilings (9.5/21 = 0.4524) on the same taxonomy, confirming that the bound is a property of the OWA/CWA taxonomy classification, not a Cedar-specific artifact. The remaining open challenge is replication in a third domain taxonomy with different action/resource structures (e.g., a document-management or financial-service schema), where the weight distribution and attestation-dependent class set may differ substantially from the agentic-AI T_9. We present $R(\pi)$ as a reproducible instrument for making a known attack surface and its attestation ceiling measurable, not as evidence of completeness against unenumerated attack classes.

6.6 The AutoResearch Pattern for Security

The AutoResearch loop — scalar metric, single-file edit, commit/rollback — applies naturally to Cedar hardening: $R(\pi)$ is the scalar metric, the `.cedar` file is the single editable artifact, and $R(\pi)$-improvement is the commit criterion. Because $R(\pi)$ is deterministic and fast (sub-second with the subprocess bridge), the loop can run without human review of each policy iteration. In the 9-iteration live-agent run, the online oracle was the pure-Python evaluator; the oracle-engine gap was addressed by an end-to-end re-run with the Rust PDP as the sole commit oracle (§4.3, §6.4), which converged identically to $R(\pi) = 1.0000$ in 6 greedy iterations with zero rollbacks. A fully autonomous deployment routes the commit criterion through the same Rust PDP used at runtime — exactly the architecture confirmed by the end-to-end re-run. The loop's convergence at $R(\pi) = 1.0000$ demonstrates that the policy closes *every vector in the current taxonomy* — it is emphatically **not** evidence of completeness against unknown attack classes (§5.2 lists four omitted classes); adding a new vulnerability class simply re-opens the metric — as demonstrated by the V9 extension, where $R(\pi_1, T_9) = 0.4524 < R(\pi_1, T_8) = 0.4750$ (V9 is open in partial), while $R(\pi_2, T_9)$ remains 1.0000.

6.7 The Schema-Reality Gap as a Language Design Problem

The specification-discovery experiment (§5.6) identifies a property of Cedar's grammar that is orthogonal to its authorization logic: **context attribute provenance is not encodable in Cedar policy or schema text.** This is not a deficiency of Cedar's formal semantics — Cedar's evaluation is well-defined and decidable [1] — but a gap between what the language specifies (which context values produce PERMIT) and what a security auditor needs to know (which context values an adversary can forge).

The gap manifests at two levels. At the *attribute level*, Cedar's `context.attr_name` syntax is identical for OWA and PEP-attested attributes, making provenance opaque to static analysis tools, LLM-based auditors, and human reviewers. At the *policy level*, a policy that appears airtight — forbidding actions unless `context.consent_client == resource.owner_id` — is structurally indistinguishable from a policy that can be bypassed by any caller who supplies the right string value. The tax_preparer illustration (§5.6) makes this concrete: the forged-consent request is byte-for-byte identical to a legitimate one; the Cedar PDP cannot distinguish them, and no additional Cedar rule can create this distinction.

Two properties of the gap are worth noting for practitioners. First, **in our 25-policy corpus, every policy containing at least one OWA-controlled context attribute exhibited an exploitable bypass**: the 5 no-bypass policies are exactly the 5 policies with zero OWA attributes (H04, F04, K02, S01, I02 — all PEP-attested across every context

attribute). There is no observed case in our corpus where an OWA attribute existed in a policy yet failed to yield a bypass. This universality follows from Cedar's policy design conventions: Cedar's default-deny semantics mean an OWA attribute that appears only in a forbid clause with no reachable permit path produces always-DENY regardless of the attribute's value — and in our corpus, OWA attributes appear in permit conditions rather than solely in forbids. The operative condition for bypass is therefore **permit-path reachability**: an OWA attribute drives a bypass when a permit rule conditioned on it is reachable by an attacker's entity type. Both published case studies confirm this mechanism: tax_preparer's `consent_client` gates the only permit rule, so any professional can forge it; document_cloud's `is_authenticated` gates a global auth-forbid while the `Public::"public"` permit path is unconditionally reachable on public documents — the Rust PDP confirms bypass in both cases. OWA-attribute risk assessment must trace which entity types can reach permits conditioned on the OWA attr; Z3's all-OWA enumeration is sound but over-approximates because it does not model entity-type matching, while the LLM identifies OWA attributes but does not automatically trace permit-path reachability. Second, **the gap is independent of PEP quality in untrusted deployments**: even a correct PEP that attests all attributes correctly provides no security guarantee if a future deployment replaces the PEP with a naive pass-through. Explicit provenance annotations in schema are the only mechanism that makes the security contract durable across PEP implementations.

The `@caller_supplied` / `@pep_attested` annotation proposal (§5.6) is a minimal language extension that closes the gap without modifying Cedar's runtime semantics. It is complementary to, not a replacement for, R(π): the annotation enables static tools to restrict their OWA analysis to the right attribute subset; R(π) then measures whether the resulting policy is robust against those OWA attributes. Together they form a provenance-aware hardening pipeline.

---

7. Related Work

ABAC policy analysis. Fisler et al. [4] introduced verification and change-impact analysis of access-control policies (Margrave), and Squicciarini et al. [12] studied separation-of-duty enforcement in ABAC. These target *completeness* and consistency rather than adversarial *robustness*, and assume a benign attribute provider.

Automated reasoning for policies. AWS's Zelkova/Tiros lineage and Automated Reasoning checks [9] verify IAM and Cedar properties via SMT solvers, proving equivalence or reachability against a fixed specification. They contain no adversarial red-teaming component; our framework *discovers* the specification under attack rather than proving against a given one. Critically, SMT-based analysis must conservatively treat all context attributes as adversary-controllable (the OWA assumption) to be sound — this is exactly the source of the Schema-Reality Gap our §5.6 experiment quantifies: 25/25 SAT results from Z3, vs. 20/25 real exploitable bypasses in a 25-policy corpus. Narrowing the formal over-approximation requires semantic provenance knowledge not present in Cedar policy text, which our annotation proposal (§5.6) addresses at the language level.

Cedar formal semantics. Cutler et al. [1] define Cedar's operational semantics, entity type system, and decidable SMT encoding for automated ABAC policy analysis, with mechanized Lean proofs and a sound, complete validator. Disselkoen et al. [14] describe the verification-guided development (VGD) methodology used to build Cedar: an executable Lean model, differential random testing between the Lean model and the production Rust implementation (finding 25 implementation bugs), and property-based testing. Taken together, [1] and [14] establish the formal correctness guarantees on which our Cedar hardening experiments rest. Our work is complementary: rather than proving a policy equivalent to a fixed specification, we empirically search under adversarial pressure for the most robust specification achievable within a given PEP attestation budget.

Formal limits of agent governance. The closest prior framing to the attestation gap appears in two works that must be explicitly differentiated. The Agentic AI Attack Surface survey [23] identifies context manipulation as a primary attack vector and observes that agent-supplied context attributes cannot be trusted — a sociotechnical

observation made without formalization and without targeting any specific authorization language. McCann [24] establishes a categorically distinct formal result: governance operators cannot decide arbitrary semantic program properties for AI workflows formalized in Interaction Trees (P3 undecidability, proved in Rocq). Security Claim 1 is orthogonal to both: it proves that no Cedar *policy rule* can close a vulnerability class whose correct-DENY decision depends on an adversary-controlled context attribute — derived from Cedar's CWA operational semantics, not from program-property decidability. The three claims operate at different layers: [23]'s observation is architectural (zero-trust principle); McCann's P3 is about governance-operator expressiveness over arbitrary program behaviors; Security Claim 1 is about the policy engine's structural inability to distinguish attested from forged attribute values within a fixed authorization model. Our result is the only one of the three proved directly from the semantics of a deployed authorization language.

Provenance algebra and cryptographic attestation infrastructure. Grädel and Tannen [28] develop a general semiring-provenance framework for first-order logic, of which our two-value (`@pep_attested`/`@caller_supplied`) annotation is a minimal special case; their algebra suggests a richer provenance lattice (e.g., partial trust, multi-hop chains) as a natural extension once a single binary distinction is validated. AEX [29] provides non-intrusive multi-hop cryptographic attestation for LLM-API calls — a concrete mechanism for populating `@pep_attested` with verifiable evidence rather than trusting the PEP's say-so, directly operationalizing the "runtime complement" our static annotations assume but do not implement. AARM [30], Bodea et al.'s Omega system [31], OIDC-A [32], and Policy Cards [33] together sketch an emerging stack for agent-action provenance one layer below the authorization decision itself: AARM specifies runtime securing of AI-driven actions, Omega attests trusted execution in cloud agent deployments, OIDC-A extends OpenID Connect to carry agent identity and authorization claims, and Policy Cards propose machine-readable governance metadata attached to agent artifacts. None of these systems addresses the Cedar-specific schema-level surfacing problem we identify — they operate at the identity/attestation-transport layer, not at the policy-language layer — but each is a candidate source of the CWA evidence our `@pep_attested` annotation would consume in a production deployment (§8.1 sketches this integration as future work).

Supply-chain and skill-level verification, contrasted with policy-level attestation. SkillFortify [34] studies supply-chain security for agent *skills* — verifying that the code an agent executes has not been tampered with in distribution. Our ecosystem evidence (§5.6) shows the *policy itself* can be the supply-chain artifact: the wshobson/agents `review-agent-governance.cedar` policy propagated byte-identical, and byte-identically vulnerable, to five independent downstream repositories via Claude Code's plugin and marketplace channels — the same distribution mechanism SkillFortify targets for skill code, but for a policy file. Metere [35] develops layered formal verification for agent skills toward a mechanically checkable capability-containment proof, assuming the underlying policy engine's decisions are trustworthy; Security Claim 1 shows that assumption can fail even for a policy engine with sound, decidable, machine-verified core semantics (Cutler et al. [1]) — the engine is correct, but what it is asked to evaluate (adversary-controlled context) is not the same as what a correct-DENY decision needs (attested context). The two lines of verification are complementary and orthogonal: skill-level containment proofs presuppose a trustworthy authorization decision; Security Claim 1 characterizes when that presupposition fails.

Independent ecosystem-scale evidence of the governance gap. The Cloud Security Alliance's May 2026 research note on MCP security [36] independently documents that agent-governance infrastructure is a systemic, ecosystem-wide blind spot — corroborating, from an industry threat-landscape perspective, the same conclusion our repo-stratified audit (§5.6) reaches from direct empirical measurement. Microsoft's Agent Governance Toolkit [37], released April 2026, offers Cedar as one of three pluggable policy-engine backends (alongside a YAML rule format and OPA/Rego) for open-source agent runtime security — meaning Security Claim 1 and the operator-misuse hazard (§4.2) apply directly to any deployment that selects the toolkit's Cedar backend, without requiring any extrapolation beyond what we evaluate directly in this paper.

Authorization for LLM agents. A parallel line of work addresses runtime authorization enforcement for tool-calling agents. Shi et al. [15] (Progent) introduce the first programmable privilege-control DSL for LLM agents,

enforcing least-privilege constraints over tool calls at runtime via an external reference monitor; the DSL and enforcement pattern are directly analogous to Cedar ABAC applied as a PDP, whereas our R(π) metric targets robustness hardening of a fixed Cedar artifact rather than runtime synthesis of new policies. Betser et al. [25] (AgenTRIM) reconstruct minimal tool-permission sets offline from execution traces and enforce them online — the tightest available implementation of least-privilege tool access for LLM agents, directly analogous to Cedar's default-deny permit rules applied at tool granularity. Abaev et al. [16] (AgentGuardian) learn context-aware ABAC policies from staging-phase execution traces and enforce control-flow-aware constraints at runtime — a dual-loop observe-then-enforce pattern architecturally similar to our blue/red loop, but focused on policy learning rather than formal robustness evaluation against a fixed taxonomy. Ji et al. [17] (SEAgent) build a mandatory access control framework that monitors agent-tool interactions via an information-flow graph and enforces entity-attribute-based security policies, addressing privilege escalation through compromised tool chains in a manner directly comparable to Cedar's entity-typed ABAC model. Kim et al. [26] (PFI, Prompt Flow Integrity) enforce information-flow integrity for LLM agent pipelines, preventing privilege escalation through compromised intermediate tool calls — directly complementary to SEAgent's MAC framework and specifically relevant to the delegation-chain attack scenarios in §5.6 (R_{chain}). Palumbo et al. [18] (PCAS) present a Datalog-derived declarative policy language with transitive information-flow tracking and cross-agent provenance, with formal semantics parallel to Cedar's policy model and particularly relevant to the inter-agent authorization and multi-hop delegation scenarios we develop in §8.1. The MiniScope framework [19] enforces least-privilege at the tool-call level by reconstructing permission hierarchies at runtime — analogous to Cedar's deny-by-default permit rules and scope narrowing via ABAC conditions. Hall et al. [20] (CloudFix) combine LLM-generated candidate repairs with SMT verification for AWS IAM policies: given a broken policy and a specification of intended behavior, the system localizes faulty statements via SMT, generates repairs with an LLM, and verifies each candidate against the provided specification before deployment — evaluated on 282 real-world IAM policies, establishing that LLM-driven policy modification scales to production corpora. Our problem formulation is non-intersecting: we provide no specification oracle, instead running an adversarial red loop to discover which vulnerability classes are exploitable and a blue loop to harden against them, converging empirically to the highest R(π) achievable. The ceiling CloudFix's per-instance SMT verification cannot detect — that attestation-dependent vulnerability classes are uncloseable *for any Cedar policy* regardless of specification — is exactly what Security Claim 1 characterizes; the two systems address orthogonal questions: repair-to-spec versus frontier-of-achievable-robustness. Vatsa et al. [27] synthesize IAM policies from natural-language descriptions using LLMs, verified against formal IAM semantics — a generative complement to CloudFix's repair approach. In contrast to all of the above systems, R(π) is explicitly a *robustness metric* applied to a fixed Cedar artifact, not a synthesis, learning, or runtime enforcement engine — a complementary evaluation-oriented role.

LLM and agent security. Greshake et al. [6] and Liu et al. [7] study indirect prompt injection and prompt-injection attacks/defenses; Perez and Ribeiro [8] catalog prompt-attack techniques. This line targets the *model* and its prompts, not the *authorization policy* that gates the agent's tool calls — a complementary layer.

Governance frameworks. Gibbs [2] provides the Four-Pillar model and the dual-engine (OpenFGA + Cedar) assumption that motivates our PEP-attestation requirement; the NIST AI RMF [10] and the EU AI Act [11] supply the regulatory backdrop for human oversight of high-risk autonomous systems.

Autonomous experimentation. Karpathy's AutoResearch [3] and Lu et al.'s AI Scientist [5] demonstrate autonomous ML experimentation loops; we extend the paradigm from ML optimization to security-policy hardening, replacing validation loss with the formal robustness metric R(π).

---

8. Conclusion

The central finding is that the Cedar attestation gap is measurable, predictable, and closeable. Security Claim 1 (§5.3) provides the structural grounding: for any Cedar deployment, the fraction of risk weight that PEP attestation

must cover equals $R^{max}_{struct}(T)$, derivable from the taxonomy without running the empirical framework. The empirical apparatus (dual-loop, R(π), exhaustive enumeration, UNSAT certificate) validates this prediction and enables practitioners to measure their specific exposure. For our agentic-AI taxonomy T_8, Security Claim 1 instantiates to an attestation ceiling $R^{max}_{struct}(T_8) = 9.5/20 = 0.4750$, confirmed constructively by the partial policy, which reaches exactly this ceiling and can advance no further without PEP attestation of `ownership_verified`, `role_attested`, and `tool_scope_verified`. Under the extended $T_9 = V1\text{–}V9$ (adding V9, the one genuinely new attack class found by exhaustive 768-state enumeration), the ceiling adjusts to $R^{max}_{struct}(T_9) = 9.5/21 = 0.4524$. The *existence* of the gap is general: any Cedar deployment with OWA-controlled context attributes faces an attestation ceiling whose magnitude is determined by its taxonomy's weight distribution.

To instantiate and measure this theorem empirically, we built **R(π)**, a deterministic, severity-weighted robustness metric evaluated against the official Cedar Rust engine (cedar-python 0.1.4), and a dual-loop hardening framework that uses R(π) as a commit criterion. A 9-iteration live-agent blue loop produced three policy variants; R(π) rose **0.2000 → 0.4750 → 1.0000** across T_8. An end-to-end re-run with the Rust PDP as the sole commit oracle converged to R(π) = 1.0000 in 6 greedy iterations with zero rollbacks, confirming the trajectory is fully reproducible under the production engine (§4.3). A secondary finding: a **fail-open implementation bug** in cedar-python 0.1.4 where `String.contains()` silently returns PERMIT on string attributes extends the attestation gap to vector V3 even when structural defenses are in place. A zero false-positive rate across all three variants (FP rate = 0%) confirms hardening does not introduce over-restriction.

A third finding, the **Schema-Reality Gap** (§5.6), identifies a structural property of Cedar's grammar: `context.attr_name` syntax is identical for caller-supplied (OWA) and PEP-attested (CWA) attributes. On a 25-policy corpus (75 context attributes, 5 domains), an SMT/Z3 all-OWA analysis produces 25/25 bypass reports in ~90ms but generates 5 false alarms (20%) because it cannot distinguish provenance. An LLM-based specification-discovery classifier achieves F1=0.649 — barely above the trivial all-OWA baseline (F1=0.515) — for the same structural reason. Two published Cedar policies (cedar-examples) ground the gap concretely: tax_preparer's consent bypass is indistinguishable at the PDP input layer from a legitimate consent claim; document_cloud's `is_authenticated` bypass is confirmed by the Rust PDP — a `Public::"public"` caller forging `is_authenticated=True` obtains PERMIT via the public-access permit path. The fix is a language-level annotation (`@caller_supplied` / `@pep_attested` in Cedar schema) that makes provenance explicit without modifying Cedar's runtime semantics. An annotation-validation experiment (`scripts/run_sprint18_annotation.py`) empirically confirms: a per-permit-rule OWA-discriminated SMT analysis using these annotations achieves **policy-level F1 = 1.000** (from 0.889), eliminating all 5 false alarms with zero missed bypasses (TP=20, FP=0, FN=0, TN=5).

A fourth finding, the **domain stability replication** (§5.7), addresses the single-schema limitation directly: Security Claim 1 holds in a second Cedar schema (S2: API Gateway/Microservices domain, `T'_{S2}` taxonomy, 7 vectors, weight 18). The Rust PDP confirms $R'(\pi_{struct}) = 7/18 = 0.3889$ (structural policy lands exactly at the Security Claim 1 ceiling; all four attestation-dependent bypass scenarios confirmed open, with same-entity discrimination checks validating that only the OWA context attribute drives the bypass) and $R'(\pi_{att}) = 1.0000$ (attested policy closes all vectors).

A fifth finding, the **language replication** (§5.8), shows that the gap is not a Cedar artifact and that the structural ceiling is language-stable: OPA/Rego (0.68.0) with a T_9-equivalent taxonomy ($T_{9\text{-OPA}}$, 9 vectors, weight 21) confirms $R^{max}_{struct}(T_{9\text{-OPA}}) = 9.5/21 = 0.4524$ — **identical to Cedar S1** — for the structural-best Rego policy (V5–V7, V9 open; V3 mixed at $b = 0.5$; V4 structural at $b = 1.0$ via explicit `contains("%2e%2e")`), and $R(\pi_{att}) = 1.0000$ for the attested Rego policy. The key finding is not just that the gap exists in Rego (Security Claim 1's structural claim) but that the ceiling magnitude is language-independent: when the structural policy applies every finite-rule-expressible defense, Cedar and Rego land at the same bound. The remaining gap (0.4524 → 1.0000) is a consequence of OWA evaluation semantics, not of any language-specific limitation.

A sixth finding, the **attestation-gate construction lint** (§5.9), mechanically verifies a construction property of the three attested policies: every permit rule requires at least one @pep_attested attribute to be True, so Z3 returns UNSAT in under 4 ms when all pep attributes are set to False. A SAT sanity check (pep=all_True) confirms the encoding is not trivially overconstrained. The lint is complementary to the 768-state exhaustive enumeration (§5.3) — the all-pep-False combinations already appear among its 2^4 context-flag sweep — but provides a compact, development-time check that no permit rule is accidentally left unguarded by attestation.

The implication for practitioners: **policy hardening requires PEP infrastructure — structural forbids alone cannot close the attestation-dependent vulnerability classes that carry the majority of risk weight in agentic AI deployments. And static analysis of Cedar policies requires provenance annotations to avoid false-alarm bypass reports that cannot be acted on.**

8.1 Future Work

Three prioritized, concrete extensions toward a stronger external-validity case. These are the specific items we judge most load-bearing for a future submission round, each with its own cost profile:

1. **Independent-authorship corpus expansion (N=2 → N≈8–10) — real progress, N=3 now.** §5.6's operator-misuse *bypass* evidence (a `forbid` discarded with a blanket-`permit` fallback) originally had independent-authorship evidence from exactly two production repositories (`wshobson/agents`, `kenhuangus/agentctl`). §5.6.1's Sprint 51 broadened the search beyond `context.*` attributes (the same type confusion applies to any String-typed attribute path) and found the underlying operator-confusion *pattern* recurring across at least seven additional independent organizations — most are correctness failures (fail-closed), but one (`shoaibakthar/observeid-platform`) is a third independently-authored, dynamically-confirmed *bypass*, raising **N from 2 to 3**. A first classification pass on this list was initially wrong (it read only the first occurrence per file and missed a second, `forbid`-clause occurrence in two of the seven repositories) and was corrected before publication rather than left standing — see §5.6.1 for the full account. A further candidate (`pkonnoth/DocuEasy`) reproduces the identical bypass structure once its unparseable-as-committed action references are repaired the obvious way, but is not counted toward N since we cannot confirm the file as committed was ever evaluated by a real Cedar engine (§5.6.1). Whether a wider or repeated sweep surfaces a fourth *confirmed-as-shipped* independent bypass remains open — but the direction of travel is now positive, not flat.
2. **Independent annotator study for the fix-precision claim (n=3 → n≥20, third-party, blind) — pilot run, human study still open.** §5.1's annotation-aware SMT result (F1=1.000) is tautological on the co-authored corpus and the out-of-sample check (§5.1) is n=3 — too small for statistical inference and not blind to a second annotator. Closing this requires a protocol where a human annotator independent of this paper's authors labels ≥20 third-party Cedar policies' context attributes as `@caller_supplied` / `@pep_attested` without seeing this paper's classifier output, so that inter-rater agreement and the SMT discriminant's precision/recall can be measured against a genuinely independent ground truth rather than the authors' own labels. An AI-simulated pilot of this protocol (Sprint 54, §5.1) has now been run — 21 third-party policies, 62 attributes, blind to this paper's own classifier and corpus — and it surfaces the first real instance of the specific "harder case" (an indistinguishable caller-supplied/PEP-attested permit path) this item's validation gap names, which independent code-level re-verification confirms is genuine, not fabricated. What remains open is exactly what the pilot cannot provide: a second *qualified human's* labels, from which inter-rater agreement and an independent-ground-truth precision/recall figure can actually be computed. This is the one extension in this list that still cannot be executed by further automated audit work — it requires a second qualified human annotator's time, not additional tooling, though the pilot's corpus is now a ready-made starting point for that person.
3. **Machine-checked entity-store extension (Lemma 2) — done.** §5.3 previously stated that the extension to entity-store semantics was argued informally, not machine-checked. `lean/`

`SecurityClaim2_EntityStore.lean` (216 lines, zero `sorry` / `admit`, standard axioms only) now proves the qualitative converse of Theorem 1 directly: entity-store attributes are not universally forgeable the way context attributes are (`no_universal_entity_forgery`), because their values come from a store the request cannot write into, not from the request itself. What remains open, and is a smaller residual than what this item originally described: neither Lean file models Cedar's full `Value` type (`prim | set | record | ext`) — both still simplify to `Value := String` — so a fully general machine-checked model of Cedar's type universe (as opposed to the OWA/CWA structural distinction this pair of files now establishes) remains future work.

Beyond these three, several further directions extend this work. Three are now established results: **(1) replication in OPA/Rego** (§5.8): OPA 0.68.0 confirms the attestation gap in Rego with $R^{max}_{struct} = 9.5/21 = 0.4524$ (identical to Cedar S1) and $R_{att} = 1.0000$, establishing both gap existence and language-stability of the ceiling; **(2) attestation-gate construction lint** (§5.9): Z3 SMT confirms UNSAT for all three attested policies (Cedar S1, S2, Rego S1) under pep_all_False in under 4 ms, mechanically verifying that no permit rule is accidentally written without an attestation gate; and **(3) entity-store-aware consistency check** (§5.9, Sprint 21): Z3 SMT encodes `is_owner = (principal_id == resource_owner_id)` as a derived integer-equality constraint, certifies π_2 UNSAT (3.95ms) under entity-store-faithful PEP context, finds V7 bypass witnesses in π_1, eliminates the V5 false alarm from Sprint 20, and includes a negative control (π_2 mutant with tool_scope guard + belt-and-suspenders forbid both dropped) that flips to SAT with V7 witness — proving the check detects real omissions and is not tautologically UNSAT. Four remain open: **(4) OpenFGA integration** for relationship-based (ReBAC) vectors that ABAC cannot express, completing the dual-engine authorization stack; **(5) a schema-policy discrepancy attack class** in which the adversary mutates not only the request context but the entity schema (e.g., a resource the schema types as `Tool` but the policy treats as `Resource`), requiring a red agent that fuzzes the schema as well as the request; **(6) Cedar provenance annotation standardization and full corpus-scale external validation** — Sprint 23 (§5.6) demonstrated CI/CD annotation emission on a co-authored corpus (F1=1.000, self-consistency; not independent validation). Sprint 26 (§5.6) provides the first external bypass audit using actual cloned cedar-examples files with OWA labels from cedar-examples README documentation: 2/7 (29%) confirmed bypass (document_cloud: sole-gate boolean; tax_preparer: EntityUID consent forgery confirmed with `__entity` format); 2/7 OWA_COMPLEX (streaming_service datetime; sales_orgs entity-store); 3/7 TN (no context attributes). What remains: (a) authoritative Rust PDP confirmation for streaming_service and sales_orgs via Cedar CLI + schema; (b) **[partially resolved — Sprint 27]** extending the audit beyond cedar-examples to third-party Cedar deployments: 14 production policies from 8 organizations audited with source-code provenance methodology (§5.6 Extended Production Corpus, n=21 combined, 52% upper-bound OWA prevalence); what remains is full systematic audit of the 1,210-file GitHub corpus with automated OWA classification and Rust PDP validation; (c) validating the write-pattern discriminant's TP recall on policies that mix OWA and PEP-attested context; (d) handling nested context structures (`context.tokens.TOKEN.CLAIM`) in the Z3 bypass pipeline; and (e) standardizing `@caller_supplied` / `@pep_attested` in Cedar's schema language; and **(7) larger-scale Cedar corpus collection** (n≥200, varied PEP implementations, mixed authorship) to evaluate both the write-pattern discriminant's generalization boundary and the annotation-based SMT discriminant's out-of-sample false-alarm rate beyond the 3-policy held-out set. The composition extensions R_{MCP} and R_{chain} — proposed as future directions in earlier drafts — are now established results (§5.4, §5.5); their remaining open generalizations are **extending R_{MCP} to cyclic MCP graphs and R_{chain} to trees with branching delegation**, where the simple min-over-path semantics must account for cycles and exponentially many authorization paths. The fifth direction is deeper and we sketch its theory below; the full composition theory — definitions, propositions, and the composition-specific vectors V_cv and V_dc (cross-vendor and delegation-chain amplification) — is left to future work.

SMT-guided exhaustive red loop. The adaptive mutation red loop of §4.4 (9,130 mutations, 7 strategies) raises the bar substantially beyond a fixed regression suite: it found 121 bypasses in the baseline and zero in both hardened policies, converging in round 2. However, adaptive convergence is a heuristic guarantee — the loop stops when no new bypasses are found in a round, but cannot certify that *no* bypass exists outside the mutation neighbourhood.

The remaining gap is the formal UNSAT guarantee. A full SMT-guided loop encodes the policy and schema as a formula $\varphi(\pi, s)$ that is satisfiable iff there exists a well-typed request producing PERMIT under π that the intended specification π *would DENY — i.e., a bypass. This encoding is directly grounded in Cedar's own formal analysis: Cutler et al. [1] prove that Cedar's policy language admits a sound, complete, and decidable encoding into SMT (specifically, a quantifier-free fragment of first-order logic decidable by off-the-shelf SMT solvers), so* $\varphi(\pi, s)$ *is guaranteed to be decidable rather than merely heuristically approachable. The Zelkova system [21] (Cook et al., FMCAD 2018) establishes the same property for AWS IAM policies, encoding IAM policy semantics as SMT formulas to check properties over all possible requests — the direct predecessor of Cedar's own SMT analysis pipeline and empirical evidence that such encodings scale to millions of daily production invocations. Shevrin and Margalit [22] (USENIX Security 2023) demonstrate the complementary model-checking approach for multi-step IAM attack-path detection, discovering real privilege-escalation misconfigurations across tens of accounts and hundreds of resources in under a minute — providing precedent that exhaustive formal exploration of bounded IAM-style request spaces is practically feasible. An adaptive loop then iterates solve → add witness to* T_n *→ re-harden → block, terminating at UNSAT with the qualitatively stronger guarantee that* no *well-typed bypass exists in the bounded schema. The space is finite: for our schema the number of distinct request types is* $|\text{Principals}| \times |\text{Actions}| \times |\text{Resources}| \times 2^{|\text{context\_flags}|} = 4 \times 5 \times 2 \times 2^4 = 640$*, small enough that exhaustive exploration is feasible — especially given the decidability guarantees of [1] and the practical scalability demonstrated by [21, 22] — and R(π) = 1.0000 could be elevated from "converged adaptive search finds nothing" to "admits no well-typed bypass" (the UNSAT certificate). Entity-store-aware encoding** is the key prerequisite: entity-store-derived attributes (e.g., `resource.owner` resolved via principal-resource entity lookup) must be modeled as system-determined constants, distinct from caller-supplied context attributes. Sprint 21 (§5.9) delivers this encoding: `principal_id` and `resource_owner_id` are free integers, `is_owner = (principal_id == resource_owner_id)` is a derived Z3 constraint following Cedar's entity-lookup semantics [1], and context attributes are explicitly bifurcated into PEP-derived (for π_2 Check A) and adversary-supplied (for π_1/π_0 Check B). The result is π_2 CERTIFIED (UNSAT, 3.47ms), V7 correctly found in π_1, and V5 correctly absent — eliminating the false-alarm class that afflicted Sprint 20's free-Boolean encoding. The remaining open extension is scaling this bounded-domain entity-store model to an open-universe SMT decision procedure aligned with Cedar's fully general entity-store semantics and Zelkova's approach [21].

A market-based decentralized orchestrator, and its connection to a parallel line of work. The dual-loop framework's current orchestrator centralizes coordination: it applies a fixed commit/rollback rule over R(π). An alternative extension direction replaces this central orchestrator with a decentralized, market-based coordination mechanism: the Blue Agent proposes policy modifications and receives a reward proportional to ΔR(π); the Red Agent proposes attack vectors and receives a reward for each bypass that invalidates a Blue Agent proposal. Both agents would learn to optimize their strategies via reinforcement learning without a central arbiter — the R(π) market itself replaces the orchestrator. This direction connects to the authors' separate line of work [39], where a reinforcement-learning attacker learns red-teaming strategies against LLMs via market-style reward/slashing signals assigned by a peer-prediction auditor committee. The architectural target differs — [39] operates at the model level (adversarial prompts against a victim LLM), whereas the extension sketched here would operate at the authorization-policy level (adversarial Cedar requests against the Rust PDP) — but the two lines share structural commitments, detailed in a companion internal note: (1) the same dual-loop adversarial co-evolution pattern (propose, attack, measure, converge), applied here to Cedar policies and there to adversarial LLM prompts; (2) the same response to the single-judge problem in domains where trusting one evaluator is unsound — this paper relies on a formal oracle (the Rust PDP, SMT, Lean) rather than any single judgment call, since Cedar admits a decidable SMT encoding [1], while [39] faces the analogous judge-reliability concern in a domain with no available formal oracle and resolves it with an auditor committee and a peer-prediction mechanism instead; and (3) a direct structural parallel between the `@caller_supplied` / `@pep_attested` provenance annotation (§5.3) and the Abstention Frontier theorem of [39] (a rational rater votes only above a confidence threshold $\tau = \lambda/(\lambda+1)$ under asymmetric misvote cost) — both are mechanisms for shielding a decision signal against an actor incentivized to falsify it, one via policy-language structural design, the other via economic mechanism design.

The unifying message, now backed by the §5.4–§5.5 results, is that the robustness of a real agentic system is not the robustness of its best policy but of its weakest composition — the weakest link of a cross-vendor MCP graph (R_{MCP}, §5.4), the attack pattern uncovered by a taxonomy (the generative red loop above), or the compromised root of a delegation chain (R_{chain}, §5.5). R(π) is the single-policy base case on which this composition theory is built.

---

9. Coordinated Disclosure

Blast radius of the confirmed case. `wshobson/agents` is a 37,453★ / 4,022ϟ flagship framework; the reachable privileged actions its broken `forbid` rules were meant to block include `git push --force` to protected branches, `gh pr merge`, and WebFetch POST to `api.github.com` — the governance gate a real user would rely on to stop exactly these actions provided none of that protection.

Timeline.

DATE	EVENT
2026-06-24	Filed `cedar-policy/cedar#2428` (upstream documentation/lint request) and `wshobson/agents#598` (maintainer notification, PoC bypass of all 5 `forbid` rules).
2026-06-24	Cedar team responded: "not a bug — writing an incorrect policy without validating it." This confirms the architectural nature of the hazard; Cedar's error-ignorance semantics are correct and documented as specified. Our upstream request is for a documentation note and an optional schemaless-mode lint warning, not a semantics change.
2026-06-24	wshobson maintainer acknowledged: "Your analysis holds: `context.<stringAttr> in [...]` is a type error that Cedar discards. This is **not** a false positive."
2026-06-25	`wshobson/agents#598` closed. Fix merged in PR#600: all 5 `forbid` rules rewritten to `["s1","s2"].contains(context.attr)`, a `.cedarschema` added (typing context attributes as `String` so `cedar validate` catches H1/H2 at load time), regression test added. PR#600 also surfaced a **separate defect**: `hooks/hooks.json` invoked `protect-mcp@0.5.5` subcommands (`evaluate`, `sign`) that do not exist in that version — the governance gate was never evaluated as shipped, independent of the Cedar syntax hazard.
2026-06-28	**GHSA-hm46-7j72-rpv9** published (ScopeBlind/scopeblind-gateway): "protect-mcp Cedar policy gate fails open on evaluation error." protect-mcp 0.7.0 released: fail-closed behavior, corrected cedar-wasm 4.x integration, boot self-test proving a `forbid` denies.
2026-07-03	`kenhuangus/agentctl#2` filed (public issue; private security reporting not enabled on that repository). 90-day embargo to 2026-10-01.
2026-07-03	`cedar-policy/cedar#2428` reformulated per Cedar team request: explicitly as a documentation + schemaless-mode lint feature request, with real-world impact evidence (the wshobson fix, GHSA-hm46-7j72-rpv9) attached.
2026-10-01	`kenhuangus/agentctl` embargo scheduled to lift.

CVE status. GHSA-hm46-7j72-rpv9 is published; CVE assignment is pending. `cedar-policy/cedar#2428` remains open as a documentation/lint feature request, not a CVE candidate — the Cedar team's position, which we do not dispute, is that error-ignorance is correct, intended, documented behavior. `kenhuangus/agentctl#2` remains under embargo until 2026-10-01.

The wshobson maintainer's independently-chosen fix validates our recommendation. The maintainer, without prompting toward any specific remediation, converged on exactly the two-part fix this paper recommends: rewrite the operator (`in` → `.contains()`) *and* add a schema so `cedar validate` catches the class at load time (§5.10) — not merely the operator fix alone. This is independent field confirmation that our recommended

defense-in-depth is the natural, practitioner-discovered response to the hazard, not an artifact of how we framed the problem.

H2 disclosure: `SafetyMP/FidusGate` (§5.6.3). Unlike the H1 disclosures above, this report went entirely through GitHub's private security-advisory channel at the maintainer's explicit request; no exploit detail was ever posted publicly.

Timeline.

DATE	EVENT
2026-07-22	A full technical disclosure email to the address listed in FidusGate's own `SECURITY.md` bounced (domain does not resolve). Filed a content-free public issue (`SafetyMP/FidusGate#54`) asking only for a working private contact channel, consistent with the maintainer's stated no-public-vulnerability-issues policy.
2026-08-10	Maintainer closed `#54` , enabled GitHub Security Advisories (private vulnerability reporting) on the repository, and asked in the issue comment for the full report to be filed there.
2026-08-11	Maintainer independently messaged (prior to receiving any technical detail): "there were massive vulnerabilities and currently working to patch" — external confirmation of severity obtained before the technical report was sent.
2026-08-12 01:46 UTC	Filed the full technical report as a private GitHub Security Advisory, **GHSA-jqc6-6pxv-g2ww** (severity critical; CWE-636, CWE-754, CWE-843), re-verifying live against the current release (`v1.9.4`) immediately beforehand to confirm the bypass was still unpatched despite three unrelated recent security commits.
2026-08-12 21:48 UTC	Fix merged (PR #86, commit `a10cb9f`): every `.contains()` call on the schema- `String` `path` / `commandLine` attributes (20+ sites) rewritten to Cedar's `like` glob operator; the application-level evaluator changed to fail closed (deny) on any policy-rule evaluation error, not only the two cases this report identified.
2026-08-12	Independently re-verified the fix: cloned the patched `main` , re-ran both original PoCs against the real `cedar-policy` 0.1.4 engine — both now return `allowed: False` with zero evaluation errors (previously `allowed: True` with `type error: expected set, got string`).

CVE status (H2). GHSA-jqc6-6pxv-g2ww remains in `triage` state pending the maintainer's formal publish/close action on GitHub; the underlying fix is merged and independently re-verified regardless of that administrative step.

The FidusGate maintainer's independently-chosen fix further validates our recommendation, and generalizes it differently than wshobson did. Facing the identical H2 pattern, the maintainer converged on exactly the root-cause fix this paper recommends — replacing `.contains()` with `like` on `String` -typed attributes — across every occurrence in the file (20+ sites), not only the two reported here, confirming the pattern was understood as systematic rather than as two isolated bugs. The chosen defense-in-depth differs from wshobson's: FidusGate already shipped a schema typing these attributes as `String` (§5.6.3), so the missing layer was not `cedar validate` but a fail-closed application evaluator, which is what the maintainer added instead. Two independent maintainers, facing the same hazard class, converged on two different but individually sufficient defense-in-depth strategies — authoring-time schema validation (wshobson) and runtime fail-closed evaluation (FidusGate) — stronger support for this paper's general defense-in-depth framing (§6) than either case alone would provide.

Could a PDP-level "error in forbid → DENY" toggle fix the H1/H2 class instead? A `forbid` -specific fail-closed mode — where a `forbid` rule that errors is treated as DENY rather than discarded — would eliminate H1/H2 without any policy changes, and is the first alternative a reviewer is likely to propose. We evaluated it and find it not equivalent to our fix, for three reasons. First, Cedar's error-ignorance rule was explicitly designed to support *policy migration*: older policies may evaluate against newer schemas that add attributes the old policies do not reference, and silent discard allows gradual migration without breaking existing permit rules; a per- `forbid` fail-

closed toggle is backward-incompatible with this use case. Second, a `forbid`-fail-closed mode is strictly weaker than our fix: it converts H1/H2 from fail-open to fail-closed, but does not close the OWA attestation gap (§5.3) — a syntactically valid `forbid when { context.role == "attacker" }` still fails open against an adversary who simply omits the `role` attribute (context attributes are OWA; an absent attribute yields `undefined`, and `undefined == "attacker"` evaluates to `false`, so the forbid does not fire). Third, the Cedar team confirmed (`cedar-policy/cedar#2428`, 2026-07-03) that error-ignorance is documented, correct, intended behavior — the correct upstream fix is schema-based validated mode, not a semantics change. Our annotation + SMT-lint fix addresses both the type-error class (H1/H2) and the value-level OWA class simultaneously, at policy-authoring time, without requiring any PDP semantics change.

References

[1] J. W. Cutler, C. Disselkoen, A. Eline, S. He, K. Headley, M. Hicks, K. Hietala, J. Kastner, A. Mamat, M. McCutchen, N. Rungta, E. Torlak, and A. Wells. "Cedar: A New Language for Expressive, Fast, Safe, and Analyzable Authorization." *Proc. ACM Program. Lang.* (PACMPL), vol. 8, no. OOPSLA1, Article 93, April 2024. arXiv:2403.04651.

[2] M. Gibbs. "Practical Agentic AI Governance, Compliance, and Runtime Security." 2026.

[3] A. Karpathy. *autoresearch* (software artifact). GitHub, 2026. https://github.com/karpathy/autoresearch

[4] K. Fisler, S. Krishnamurthi, L. A. Meyerovich, and M. C. Tschantz. "Verification and change-impact analysis of access-control policies." In *Proc. ICSE*, 2005.

[5] C. Lu et al. "The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery." *arXiv:2408.06292*, 2024.

[6] K. Greshake et al. "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection." In *Proc. AISec*, 2023.

[7] Y. Liu et al. "Prompt Injection Attacks and Defenses in LLM-Integrated Applications." *arXiv:2310.12815*, 2023.

[8] F. Perez and I. Ribeiro. "Ignore Previous Prompt: Attack Techniques for Language Models." In *NeurIPS ML Safety Workshop*, 2022.

[9] Amazon Web Services. "Automated Reasoning checks for Amazon Bedrock Guardrails." *AWS Documentation*, 2024.

[10] National Institute of Standards and Technology. "AI Risk Management Framework (AI RMF 1.0)." 2023.

[11] European Parliament. "EU AI Act, Regulation 2024/1689." *Official Journal of the EU*, 2024.

[12] A. Squicciarini et al. "Enforcement of Separation of Duty Constraints in ABAC Systems." *IEEE TDSC*, 2008.

[13] Anthropic. "Model Context Protocol (MCP) Specification." 2024–2025.

[14] C. Disselkoen, A. Eline, S. He, K. Headley, M. Hicks, K. Hietala, J. Kastner, A. Mamat, M. McCutchen, N. Rungta, B. Shah, E. Torlak, and A. Wells. "How We Built Cedar: A Verification-Guided Approach." In *Companion Proc. ACM Int. Conf. Found. Softw. Eng.* (FSE), pp. 646–651, July 2024. arXiv:2407.01688.

[15] T. Shi, J. He, Z. Wang, H. Li, L. Wu, W. Guo, and D. Song. "Progent: Programmable Privilege Control for LLM Agents." *arXiv:2504.11703*, April 2025.

[16] N. Abaev, D. Klimov, G. Levinov, D. Mimran, Y. Elovici, and A. Shabtai. "AgentGuardian: Learning Access Control Policies to Govern AI Agent Behavior." *arXiv:2601.10440*, January 2026.

[17] Z. Ji, D. Wu, W. Jiang, P. Ma, Z. Li, Y. Gao, S. Wang, and Y. Li. "Taming Various Privilege Escalation in LLM-Based Agent Systems: A Mandatory Access Control Framework (SEAgent)." *arXiv:2601.11893*, January 2026.

[18] N. Palumbo, S. Choudhary, J. Choi, P. Chalasani, M. Christodorescu, and S. Jha. "Policy Compiler for Secure Agentic Systems (PCAS)." *arXiv:2602.16708*, February 2026.

[19] (Anonymous). "MiniScope: A Least Privilege Framework for Authorizing Tool Calling Agents." *arXiv:2512.11147*, December 2025.

[20] B. Hall, O. Ungaro, and W. Eiers. "CloudFix: Automated Policy Repair for Cloud Access Control Policies Using Large Language Models." *arXiv:2512.09957*, December 2025.

[21] B. Cook, K. Khazem, D. Kroening, S. Tasiran, M. Tautschnig, and M. R. Tuttle. "Semantic-based Automated Reasoning for AWS Access Policies using SMT." In *Proc. FMCAD* (Formal Methods in Computer Aided Design), pp. 1–9, October 2018. IEEE. https://ieeexplore.ieee.org/document/8602994/

[22] I. Shevrin and O. Margalit. "Detecting Multi-Step IAM Attacks in AWS Environments via Model Checking." In *Proc. USENIX Security Symposium*, 2023. https://www.usenix.org/conference/usenixsecurity23/presentation/shevrin

[23] (Multiple Authors). "Agentic AI as Cybersecurity Attack Surface: A Systematic Survey of Attack Vectors and Security Risks." *arXiv:2602.19555*, February 2026.

[24] P. McCann. "Effect-Transparent Governance for Agentic AI Systems." *arXiv:2605.01030*, May 2026.

[25] N. Betser et al. "AgenTRIM: Agentive Tool Right-sizing via Instrumented Minimization." *arXiv:2601.12449*, January 2026.

[26] S. Kim et al. "Prompt Flow Integrity: Controlling Information Flow in LLM Agents." *arXiv:2503.15547*, March 2025.

[27] M. Vatsa, B. Hall, and W. Eiers. "LLM-Based Synthesis and Repair of Access Control Policies." *arXiv:2510.20692*, October 2025.

[28] E. Grädel and V. Tannen. "Provenance Analysis and Semiring Semantics for First-Order Logic." *arXiv:2412.07986*, December 2024.

[29] Y. Guan. "AEX: Non-Intrusive Multi-Hop Attestation and Provenance for LLM APIs." *arXiv:2603.14283*, March 2026.

[30] H. Errico. "Autonomous Action Runtime Management (AARM): A System Specification for Securing AI-Driven Actions at Runtime." *arXiv:2602.09433*, February 2026.

[31] T. Bodea, M. Misono, J. Pritzi, P. Sabanic, T. Sommer, H. Unnibhavi, D. Schall, N. Santos, D. Stavrakakis, and P. Bhatotia. "Trusted AI Agents in the Cloud." *arXiv:2512.05951*, December 2025.

[32] S. Nagabhushanaradhya. "OpenID Connect for Agents (OIDC-A) 1.0: A Standard Extension for LLM-Based Agent Identity and Authorization." *arXiv:2509.25974*, September 2025.

[33] J. Mavračić. "Policy Cards: Machine-Readable Runtime Governance for Autonomous AI Agents." *arXiv:2510.24383*, October 2025.

[34] V. P. Bhardwaj. "Formal Analysis and Supply Chain Security for Agentic AI Skills." *arXiv:2603.00195*, February 2026.

[35] A. Metere. "Methods for Formal Verification of Agent Skills: Three Layers Toward a Mechanically Checkable Capability-Containment Proof." *arXiv:2605.23951*, May 2026.

[36] Cloud Security Alliance. "MCP Security Crisis: Systemic Design Flaws in AI Agent Infrastructure." CSA Research Note, May 2026. https://labs.cloudsecurityalliance.org/research/csa-research-note-mcp-security-crisis-20260504-csa-styled/

[37] Microsoft. "Introducing the Agent Governance Toolkit: Open-Source Runtime Security for AI Agents." Microsoft Open Source Blog, April 2026. https://opensource.microsoft.com/blog/2026/04/02/introducing-the-

agent-governance-toolkit-open-source-runtime-security-for-ai-agents/ — Cedar supported as a pluggable policy backend alongside OPA/Rego, https://github.com/microsoft/agent-governance-toolkit.

[38] Anonymous Authors. "[title omitted for double-blind review]." A concurrent, independently-anonymized submission characterizing a dynamically-verified negation-on-undefined fail-open hazard taxonomy and corpus audit in OPA/Gatekeeper Kubernetes admission control, structurally analogous to H1/H2 of this paper. 19th ACM Workshop on Artificial Intelligence and Security (AISec 2026), 2026.

[39] Anonymous Author(s). "[title omitted for double-blind review]." The authors' own separate submission to IEEE TPS-ISA 2026, presenting a peer-prediction auditor committee and an Abstention Frontier theorem for automated LLM red-teaming; cited here as related but separate work. 2026.

Appendix A: Cedar Policy Listings

A.1 Baseline Policy (π_0) — Key Permit Rules

```
// Reader: read and list data Resources only.
permit (
  principal is Agent,
  action in [Action::"read", Action::"list"],
  resource is Resource
)
when { principal.role == "reader" };

// Writer: read, write, list own resources only.
permit (
  principal is Agent,
  action in [Action::"read", Action::"write", Action::"list"],
  resource is Resource
)
when {
  principal.role == "writer" &&
  resource.owner == principal.id
};

// Admin: delegate to non-admin agents.
permit (
  principal is Agent,
  action == Action::"delegate",
  resource is Agent
)
when {
  principal.role == "admin" &&
  resource != principal
};

// Executor: execute Tools only.
permit (
  principal is Agent,
  action == Action::"execute",
  resource is Tool
)
when { principal.role == "executor" };
```

π_0 has no `forbid` rules. R(π) = 0.2000. V1 is closed by Cedar's default-deny; V8 (over-blocking check) is also closed: baseline permits correctly allow all four legitimate-access requests (FP rate = 0%).

A.2 Partial Policy (π_1) — Structural Forbids Added

```
// Block scope overflow: reader must not write, execute, or delegate.
forbid (principal is Agent, action in
  [Action::"write", Action::"execute", Action::"delegate"], resource)
when { principal.role == "reader" };

// Block delegation by any non-admin role.
forbid (principal is Agent, action == Action::"delegate", resource is Agent)
when { principal.role != "admin" };

// Block path traversal: ASCII literal and URL-encoded variants.
forbid (principal is Agent, action, resource is Resource)
when {
  resource.path like "*..*"      ||
  resource.path like "*%2e%2e*"  ||
  resource.path like "*%2E%2E*"
};
// (+ 4 additional encoded variants in full policy)
```

π_1 adds seven structural forbids (F1a/F1b scope overflow, F2a/F2b/F2c delegation, F3/F4 literal and encoded path traversal; abbreviated above). V5/V6/V7 remain open because OWA allows context-attribute forgery; V3.2 (Unicode U+FF0E) remains open for the same reason (B=0.5, contributing 0.5 weight). V8 remains closed (FP rate = 0%): the structural forbids do not over-block any legitimate access. $R(\pi, T_8) = 0.4750 = 9.5/20$.

A.3 Final Policy (π_2) — Attestation-Dependent Controls

```
// Writer permit: now requires PEP ownership attestation and path safety.
permit (
  principal is Agent,
  action in [Action::"read", Action::"write", Action::"list"],
  resource is Resource
)
when {
  principal.role == "writer" &&
  resource.owner == principal.id &&
  context.ownership_verified == true &&   // PEP: external registry
  context.path_safe == true               // PEP: path normalization
};

// Admin permit: requires cryptographic role attestation.
permit (
  principal is Agent, action == Action::"delegate", resource is Agent)
when {
  principal.role == "admin" &&
  resource != principal &&
  (resource.role == "reader" || resource.role == "writer"
   || resource.role == "executor") &&
  context.role_attested == true           // PEP: signed identity assertion
};

// Executor permit: PEP path safety + tool capability scope verification.
permit (
  principal is Agent, action == Action::"execute", resource is Tool)
when {
  principal.role == "executor" &&
  context.path_safe == true &&
  context.tool_scope_verified == true     // PEP: tool capability registry
```

```
};

// Belt-and-suspenders: deny when attestation absent.
forbid (principal is Agent, action in
  [Action::"read", Action::"write", Action::"list"], resource is Resource)
when {
  principal.role == "writer" &&
  context.ownership_verified == false
};
```

π_2 adds 4 attestation conditions to permits and 4 complementary negative-condition forbids. R(π) = 1.0000. All 8 vectors closed (V1–V7 attack vectors, V8 over-blocking check; FP rate = 0%).

A.4 Composition Artifacts (R_MCP, R_chain)

The multi-actor and delegation-chain results of §5.4–§5.5 are produced by two additional artifacts in the repository. `mcp_hardened_policy.cedar` extends π_2 with the V_cva cross-vendor forbid rule `forbid when { context.attestation_source != context.resource_authority }`; this is the policy deployed on both nodes of Graph B to reach $R_{MCP} = 1.0000$. The rule is only well-typed because `agent_schema.cedarschema` extends the Cedar context type with `attestation_source: String` and `resource_authority: String` — typed attestation provenance that the original boolean `ownership_verified` flag cannot express. Both artifacts, together with the per-state logs `rmcp_experiment_v1.csv`, `rchain_experiment_v1.csv`, and the 768-state exhaustive enumeration, are included in the reproducibility package.

Appendix B: Formal Proof Reproduction and Artifact Verification

B.1 Building and Checking the Lean Proofs

`lean/SecurityClaim1.lean` and `lean/SecurityClaim2_EntityStore.lean` are both standalone (no Mathlib dependency) and pinned to a specific toolchain via `lean/lean-toolchain`:

```
leanprover/lean4:v4.32.0
```

Reproduction, using `elan` (the Lean version manager):

```
cd lean/
elan install leanprover/lean4:v4.32.0   # or: elan toolchain install $(cat lean-toolchain)
lean SecurityClaim1.lean
echo $?    # expect 0
lean SecurityClaim2_EntityStore.lean
echo $?    # expect 0
```

A successful check compiles with exit code 0 and prints no `sorry` / `admit` warnings. `SecurityClaim1.lean`'s load-bearing theorems are `attestation_gap` and `no_policy_achieves_full_blockage`; `SecurityClaim2_EntityStore.lean`'s are `no_universal_entity_forgery` (the main result — entity-store attributes are not universally forgeable, unlike context) and the supporting `entityAttr_depends_only_on_resource` / `entity_forgery_is_selection_not_fabrication`. `SecurityClaim1.lean` was verified against both `v4.31.0` and `v4.32.0` in prior sessions of this project; both files were re-verified fresh in this session (exit code 0, zero `sorry`, on `v4.32.0`), and `#print axioms` confirms every theorem depends only on Lean's standard core axioms (`propext` for `SecurityClaim1.lean`'s theorems; `propext`, `Classical.choice`, `Quot.sound` for `no_universal_entity_forgery` — no `sorryAx` or other non-standard axiom in either file). `lean-toolchain` pins `v4.32.0` as canonical. §5.3 states the exact scope both proofs establish: a simplified, context-only-plus-entity-store model (`Value := String` in both files), not a machine-checked model of Cedar's full type universe.

B.2 Result Artifact Checksums

The following SHA-256 checksums pin the exact result files this paper cites to specific bytes, so a reviewer or replicator can confirm they are inspecting the same artifact this paper's numbers were computed from:

FILE	SHA-256
`results/sprint32_live_exploit_confirmed.json`	`2c7e7e41cc6136111030b8e51f352c1a97f6171c1c80bdaf0e`
`results/sprint43b_repo_stratified.json`	`1bd393577ee6adba43eb3ccb1cd632501051ffb5f39b2afbe5`
`results/sprint45f_enriched_confirm.json`	`b40da938ea19e2f5b29c491f43f4fefe26c73bc221ad3fe7a1`
`results/sprint46_schema_ci_stratification.json`	`3d6c1a721c7fa6761389ab6591d6fce353a09865d929950641`
`results/sprint47_weight_sensitivity.json`	`13668d779b0478055afea66fe8ed774a3ff81c9b296ec63202`
`results/sprint48_validator_baseline.json`	`89850c2882a832a6cd368a59ed3116a247fdfa9ecbb1e9d347`
`results/sprint49_validator_corpus_catchrate.json`	`27e7569a7ff52785b95ebb534f6d54a0ab7258aa208edcb425`
`results/sprint50_h2_contains_audit.json`	`0d0d0d1a4e65c2167de7461d4826a74d02430c27f834e69980`
`results/sprint51_broadened_authorship_search.json`	`eba73acae5c9c2d23324d84bcfee672cbf862849a40d3fa21a`
`results/sprint52_h2_schema_crossref.json`	`38062bcd5e0d881dd7a0ecfd627a64d7ddc9594322740fe7f8`
`results/sprint53_h2_schema_json_resolver.json`	`eec64d4f48b96b176764ef49daea5e68eae54e28213aff2e87`
`results/sprint54_independent_annotator_blind_study.json`	`9893261ce709e1f7d1b4e31756081a6af800a2957b5161c4dc`
`lean/SecurityClaim1.lean`	`ea039d4e8f46e9c3892cd56adae72ecbcd07d520c3b101d190`

Regenerate with `sha256sum <path>` from the repository root; a mismatch means the artifact has changed since this paper's numbers were computed and the citing claim should be re-verified against the new file before being trusted.

B.3 Classification Regexes

The two operator-misuse patterns (§4.2) that drive the §5.6/§5.6.1/§5.6.2 corpus audits are detected syntactically, but with an important asymmetry in reliability. The H1 (`in`-on-String) pattern, `context.<attr> in [<string literal>, ...]`, is distinguished from safe usage (entity-UID `in` lists, e.g. `principal in [Role::"admin"]`) by requiring the right-hand side to be a string or string-list literal rather than an entity-type reference; because Cedar's `in` operator is never semantically valid against a literal string list regardless of the attribute's type, the regex reliably identifies a genuine type-error *pattern* wherever it matches. That is a narrower claim than "no false positives," however: §5.6.1's file-by-file review of the 14 regex hits found 5 are non-deployment artifacts (vendored copies of Cedar's own test fixture, or mirror repos carrying only a self-labeled test file) rather than production misuse — the regex is a reliable *pattern* detector, not a reliable *deployment* detector, and distinguishing the two required the same manual, per-file provenance check applied to H2 below, not just eyeballing that the syntax matches. The H2 (`.contains()`-on-String) pattern, `context.<attr>.contains(<string literal>)`, has no such property: `.contains()` is Cedar's legitimate Set-membership operator, so the regex can only enumerate *candidates* (receiver is a bare `context.<attr>`), not confirm misuse — §5.6.2 manually reviewed all 10 regex-flagged H2 candidates and found zero true positives; a co-located `.cedarschema` definitively confirmed correct `Set<String>` typing for 2 of the 10, and the remaining 8 are unresolved-but-consistent with correct usage (naming and multi-literal usage pattern), not affirmatively confirmed. As noted in §6.4, a purely syntactic oracle cannot rule out false negatives from computed attribute names or string concatenation at scale for either pattern; for H2 specifically, it also cannot rule out false positives without schema information, which is why we report §5.6.2 as a manually-verified negative result rather than a raw regex count.

B.4 Engine Version Pins

All dynamic confirmations in this paper use one of three pinned engine versions, stated explicitly at each point of use rather than assumed: **cedar-python 0.1.4** (PyO3 binding over cedar-policy Rust 4.8.2) for the primary Rust PDP evaluations; **`@cedar-policy/cedar-wasm` v4.11.2** (official WebAssembly package, AWS Cedar team, wasm-bindgen build) for cross-engine

replication (§6.5); and **OPA 0.68.0** for the Rego language replication (§5.8). Registry-hash verification for each, fetched directly from the official registry APIs rather than computed locally (so a reviewer can cross-check independently of trusting this paper's build):

PACKAGE	REGISTRY	ARTIFACT	SHA-256 (OR REGISTRY-N
`cedar-python` 0.1.4	PyPI JSON API (`pypi.org/pypi/cedar-python/0.1.4/json`)	`cedar_python-0.1.4.tar.gz` (sdist)	`6006a0db7fc314bb0d15`
`cedar-python` 0.1.4	PyPI JSON API	`cedar_python-0.1.4-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl` (the wheel this paper's dynamic confirmations actually run under, Python 3.12 Linux x86_64)	`6eca6ab21cca34c79a87`
`@cedar-policy/cedar-wasm` 4.11.2	npm registry (`registry.npmjs.org/@cedar-policy/cedar-wasm/4.11.2`)	package tarball	integrity `sha512-WirTj7pA v14vHz6nEcxAQjR6oXtm2 6775027f3b4640c4d172`
OPA 0.68.0	GitHub Releases (`open-policy-agent/opa`, tag `v0.68.0`)	`opa_linux_amd64` binary	`afaea29d8db862035fae`

Any of these can be independently re-fetched and re-hashed via the registry URLs above; a mismatch means either the upstream artifact changed or a local build/install step introduced drift from what this paper's dynamic confirmations actually ran against.